

UNIT 1

INDEX

1. ENTERPRISE
2. BUSINESS PROCESS
3. DIGITAL TRANSFORMATION [IT/OT CONVERGENCE]
4. INTRODUCTION TO FULL STACK DEVELOPMENT
5. DESIGN THINKING

WHAT IS AN ENTERPRISE?

An Enterprise is an industrial undertaking, or a business concern, or any other establishment, which is engaged in production of goods and services to fulfil the demand of the customer.

Entrepreneurship involves using person initiative to identify some need and taking the necessary steps to satisfy that need. It involves taking both financial and personal risks.

FORMS OF AN ENTERPRISE

1] SOLE PROPRIETORSHIP

Sole proprietorship is the simplest and oldest form of enterprise. It is also known as a sole trader or simply proprietorship. It is the type of business entity which is owned and run by one individual, which means 100% ownership and profits stay with the owner and entire the decision taking authority.

- Single ownership
- No separate legal entity of the firm
- Unlimited liability
- One-man control
- Undivided risk

2] PARTNERSHIP FIRM

A group of persons can join their hands to form a partnership to fulfil the needs of organization. Partnership, the form of enterprise, over comes the limitations of a one- person business such as need of more capital, better managerial skills and specialization.

The partnership may come into existence either as a result of expansion of the sole proprietorship firm or by means of an agreement between two or more people to join hands to form a partnership firm.

- Two or more persons enter into the partnership agreement,
- A written or oral agreement,
- Mutual consent to join hands among the partners,
- No separate legal entity of the firm.

3] LIMITED LIABILITY PARTNERSHIP FIRM

Limited Liability Partnership is a hybrid form of organization which contains the features of both, the traditional partnership firm and company.

It is an alternative corporate business form which offers benefits of limited liability to the partners at lower compliances costs. The liabilities of its partners are limited.

- It is a corporate body which has its separate legal entity,
- It enjoys the perpetual succession unlike traditional form of partnership,
- However, unlike partnership firm, there is no limit of maximum members,
- Partners assume limited liability,
- If partners agree, LLP can have a common seal.

4] JOINT STOCK COMPANY

A joint stock company is a group of individuals organized to conduct a business for a common motive of making profits.

The individuals have a joint stock of capital represented by shares owned in the company. These shares are transferable by the members without the consent of the group.

WHAT IS FEASIBILITY STUDY?

Feasibility study is a systematic analysis that involves finding and documenting potential solutions to a particular business idea.

The feasibility study is done to check whether the business plan is compatible with the accessible technology or not.

1. Market Feasibility Analysis:

Market feasibility analysis deals with understanding the potential market and market share, market factors like market growth potential and the location of competitors. Market survey is the most popular method to evaluate the market feasibility. The market feasibility study involves the analysis of the nature of market cost of production and overall cost of production, selling price and profits, market demand, target market, market share etc.

2. Technical Feasibility Analysis:

Technical feasibility analysis deals with analyzing the technical viability of the project. In this study, efforts are made to review the infrastructural facilities like land, building, transportation, availability of raw material, locational advantages and disadvantages, evaluation of technical knowhow, labour force requirement and project schedule.

3. Financial Feasibility Analysis:

This involves assessing the risk and uncertainty involved in executing the project. The purpose of this analysis is to present some measures to assess the financial viability of the project. The feasibility report should take into account the expected income generated, taxation liabilities as per the applicable tax laws, income tax rebates for priority industries, incentives for backward areas, expected depreciations in line with the industry it belongs to etc.

4. **Commercial Feasibility Analysis:**

Commercial feasibility analysis evaluates the commercial aspects of the arrangement of the project. It also examines the arrangement process of plant and machinery, procedure of selection of raw material suppliers, appropriate location for the plant and other commercial aspects of the project.

5. **Economic Analysis:**

Economic analysis involves evaluation of a project in terms of cost and benefit analysis. It is the most crucial aspect of evaluation of a business idea. It is the process of determining the efficiency of the project. It is also called Cost Analysis.

ENTERPRISE SETUP

To setup an enterprise, the entrepreneur should have complete knowledge of men, material, machinery, market, and products. A number of formalities like approval and clearance from government departments are to be completed before setting up an enterprise.

The setting of an enterprise involves the study of business opportunities, developments of a feasible business plan and identification, determination and arrangement of men, materials, machinery and market for products.

WAYS TO SETTING UP AN ENTREPRICE:

1. **Conduct market research:**

Market research will tell you if there's an opportunity to turn your idea into a successful business. It's a way to gather information about potential customers and businesses already operating in your area. Use that information to find a competitive advantage for your business.

2. **Write your business plan:**

Your business plan is the foundation of your business. It's a roadmap for how to structure, run, and grow your new business. You'll use it to convince people that working with you or investing in your company is a smart choice.

3. **Fund your business:**

Your business plan will help you figure out how much money you'll need to start your business. If you don't have that amount on hand, you'll need to either raise or borrow the capital. Fortunately, there are more ways than ever to find the capital you need.

4. **Pick your business location:**

Your business location is one of the most important decisions you'll make. Whether you're setting up a brick-and-mortar business or launching an online store, the choices you make could affect your taxes, legal requirements, and revenue.

5. **Choose a business structure:**

The legal structure you choose for your business will impact your business registration requirements, how much you pay in taxes, and your personal liability.

WHAT IS BUSINESS?

Business is described as an organization or entity engaged with industrial, commercial, or professional activities.

The term Business also refers to the organized activities and efforts of individuals to build and sell goods and services for profit.

WHAT IS A BUSINESS PROCESS?

A business process is defined as, a collection of business tasks and activities that when performed by people or systems in a structured course, produce an outcome that contributes to the business goals.

INTRODUCTION TO BUSINESS ACTIVITIES: -

1] Operating Activities:

Operating activities refer to all those business activities that are directly or indirectly related to the provision of goods and services. As such they have a direct impact on cash flow, and eventually on income.

2] Investing Activities:

Investing activities refers to all those activities that aimed to be capitalized for more than a year. This includes capital expenditure such as purchase of long term assets or real estate.

3] Financing Activities:

Financing activities refer to all those activities that fund the business but are not directly related to the revenues from goods and services. Common financing activities include bonds, loans, and share issues.

CLASSIFICATION OF BUSINESS ACTIVITIES

INDUSTRY: -

The industry sector is defined as a sector where raw material gets transformed into beneficial products. An industry may create capital goods or consumer goods such as cloth, radio, bread, butter, etc. The industry can be classified into three categories namely:

1] Primary Industry

Primary industry is known as extractive industries. It involves activity connected with the production of wealth directly from natural resources such as water, air, land, etc. The primary sector involves activities like processing and extraction of natural resources etc.

2] Secondary Industry

The industry that uses raw materials as input and produces finished products as output is known as the secondary industry.

3] Tertiary industry

Tertiary industries are regarded as providing services that promote the flow of services and goods. This industry helps in the actions of the primary and secondary sectors.

WHAT IS BUSINESS PROCESS AUTOMATION?

Business process automation is a way to increase organizational efficiency by leveraging digital technologies to carry out business processes with minimal human intervention. By shifting the responsibility of completing routine tasks from humans to machines, business process automation allows employees to spend more time focusing on value-added activities.

WHAT ARE THE BENEFITS OF BUSINESS PROCESS AUTOMATION?

1. Time savings: Business process automation saves your employees a great deal of time and allows them to work smarter, not harder. By charging machines with completing recurrent, rule-based tasks, employees can free up time to focus on work that requires critical thinking, creativity, or the human touch.

2. Increased profits: While routine processes are necessary for the functioning of every company, they do not create value for your customers. Therefore, manually performing these processes often leads to lost profit. With business process automation, organizations can increase their profitability by delegating mundane tasks to machines and spending more time on activities that add value to their customers.

3. Higher productivity: Organizations that rely on technology to automate processes also see an increase in productivity. The main reason is that machines can handle multiple tasks simultaneously, thus speeding up processes. By relying on technologies to help with some of their tasks, employees will also get more work done in the same amount of time.

4. Greater efficiency: Business process automation helps companies fulfil tedious tasks with minimal resources. Companies get faster and more reliable results with lower costs, thus improving efficiency significantly.

5. Error minimization: Even the most capable and skilled employees make mistakes. Lack of attention, absent-mindedness, forgetfulness, and multitasking can all hinder the performance of your employees — especially when performing routine tasks. Unlike humans, computers don't forget. Plus, they never get tired or distracted. Therefore, automating processes helps companies decrease the likelihood of human error dramatically.

WHAT IS DIGITAL TRANSFORMATION?

Digital transformation is the integration of digital technology into all areas of a business, fundamentally changing how you operate and deliver value to customers. It's also a cultural change that requires organizations to continually challenge the status quo, experiment, and get comfortable with failure.

Historically, information technology (IT) and operational technology (OT) teams in manufacturing worked in siloes. While recent digital transformations have brought them closer together through IT/OT convergence, many companies still hesitate to fully integrate their networks, data, and processes.

Several use cases showcase IT/OT convergence powering digital transformation on modern shop floors:

1. **Predictive Maintenance**- Industrial IoT sensors collect real-time telemetry data from production equipment. IT systems aggregate and analyze the operational data using cloud analytics and artificial intelligence. The predictive insights on emerging equipment faults feedback to OT teams. They schedule precise maintenance during planned downtime instead of relying on imprecise schedules. Avoiding unexpected outages boosts productivity and asset usage.

2. **Augmented Reality** – OT technicians utilize augmented reality (AR) devices to streamline equipment inspections, maintenance, and repairs. High-fidelity 3D renderings overlay sensors, parts, and monitoring capabilities. Hands-free access to manuals and remote guidance enable faster issue resolution. IT networking and application support integrate the AR systems with backend data sources and content repositories.

3. **Smart Inventory** - Sensors, timers, and production signals feed into IT inventory management software. Dynamic dashboards provide real-time visibility for OT staff to improve material flows. Inventory optimization reduces waste, frees up working capital, and improves throughput.

HOW TO FACILITATE IT/OT CONVERGENCE

- **Safety and reliability.**

Keeping people safe and happy are essential goals for both IT and OT teams. If safety is the main concern, it can be easier to find solutions that work for both sides. For example, reliability is often more valued than a shiny new digital tool that takes too long to learn and slows down shop floor data latency.

- **A desire to pioneer their industry.**

Energize teams by encouraging them to be the first to tackle problems that other companies haven't figured out how to solve. Working together, IT/OT can do what's never been done before. Industry has thrived on innovative discoveries. Keeping the spirit of industry alive with innovation can have altruistic and improved performance wins.

- **Building together.**

Gain employee trust by working on culture shifts, showcasing the power of collaboration. Team silos may be holding the company back from achieving true success. Having "common goals" (which are not mutually divided for accountability), can make IT/OT teams more combative than collaborative. Identify leaders who can work to bridge cultural gaps and who are trusted by both sides.

- **Improved security.**

Without IT and OT convergence, your company runs the risk of increased security issues. When there are more resources dedicated to supporting staff, (e.g. training and monitoring), best practices can become scalable. Having a shared vision to protect the company's data can encourage understanding and mutual respect.

HOW TECHNOLOGY HAS IMPACTED DIGITAL TRANSFORMATION

- **Improved efficiency:**

Technologies such as analytics, machine learning, cloud computing, robotic process automation (RPA) and artificial intelligence allow businesses to automate repetitive tasks and improve efficiency. This technology can result in cost savings and improved productivity.

- **Data-driven decision making:**

The internet of things (IoT) automation, artificial intelligence, analytics, big data analytics, and machine learning enable businesses to collect and analyze data from various sources. This enables predictive analytics, real-time monitoring and decision making, improving efficiency and reducing costs. Furthermore, it will enable companies to make data-driven decisions, which can lead to better outcomes.

- **Increased flexibility:**

The emergence of technologies such as cloud computing, big data, software as a service, and mobile technology allow employees to access data and applications from anywhere,

enabling more flexible work arrangements and allowing companies to be more responsive to changing market conditions.

- **Enhanced security:**

As businesses become more digitized, cybersecurity has become increasingly important. The digital technologies such as automation, artificial intelligence, analytics, machine learning, blockchain and encryption can help protect sensitive data and prevent cyber-attacks.

THE TOP DIGITAL TRANSFORMATION SUCCESS STORIES

Microsoft

Microsoft have been leaders in software innovation since the dawn of home computing in the late 1980s. But faced with the advent of cloud-based software systems in the early 2000s, their long-established model suddenly seemed dated.

Their digital strategy was to completely overhaul their delivery systems, using their proprietary cloud solution. They launched Microsoft Azure in 2008. Azure has now become the 2nd biggest cloud provider in the world, providing a vast range of digital services for individuals and companies of all sizes.

As market leaders, Microsoft could have chosen to behave like Blockbuster video. Instead, they made a shrewd decision to completely update their digital strategy. If they never launched that radical change program, they could have been history.

Our article on Microsoft Azure gives you the full lowdown on this valuable product.

Nike

Nike, the global sportswear leader, showcases the excellent results that digital transformation initiatives can create.

Nike's entire approach to branding, sales, and technology has changed.

They now focus on becoming a robust direct-to-consumer brand, reducing their retail partnerships, and increasing their work to create online sales. Programs like Nike Training Club collect detailed consumer data and use advanced analytics to make sense of the patterns. Smaller physical stores now do a lot of work to promote the Nike brand.

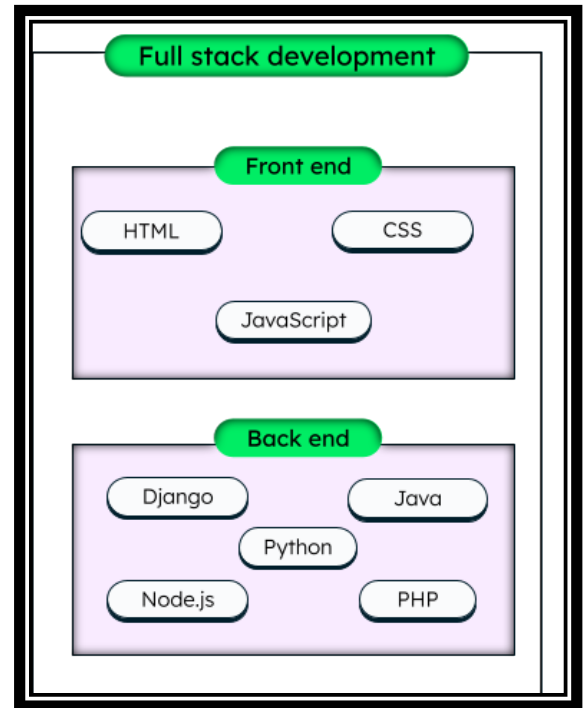
Behind the scenes, they use headless commerce to develop multiple online sales channels, with particular expansions into mobile apps and technology.

Like IKEA, Nike has dominated their market section for a long time. Their success in digital transformation means that they will continue to dominate. Instead of sticking with what they know, they've taken risks that have paid off. As a result, they've become a trend-setter in digital infrastructure, who are even used as a case study for teaching up-and-coming IT change leaders.

FULL STACK DEVELOPMENT

Full stack development is the process of developing both the frontend and backend of applications. Any application has a frontend (user-facing) component and a backend (database and logic) component. The frontend contains the user interface and code related to user interactions with the application. The backend contains all the code required for the application to run, including integrations with data systems, communicating with other applications, and processing data.

Software developers require slightly different skills, tools, and software knowledge for frontend and backend development. However, full-stack development combines both disciplines—meaning teams can build applications from start to finish in an integrated and cohesive manner. A full-stack application is a software application that encompasses both frontend and backend in a single code base.



WHAT ARE THE BENEFITS OF FULL-STACK DEVELOPMENT?

Organizations adopting full-stack development strategies experience the following benefits.

1)Efficient resource utilization

Traditional setups require separate developers for frontend, backend, database management, and sometimes even for server operations. Full-stack developers are adept at handling multiple aspects of a project, from user interface design to server-side scripting and database management. This significantly reduces the number of specialized roles that an organization needs to fill, making project management simpler and more streamlined.

2)Flexibility and speed

Full-stack developers have the skills to create minimum viable products (MVPs) at a much faster rate, given their ability to handle all layers of application development. This is particularly beneficial in agile or lean development frameworks where rapid iteration is key.

3)Enhanced problem-solving

The ability to identify problems across a project is enhanced when you have full-stack developers on board. Their comprehensive understanding of the system architecture and multiple coding languages enables them to identify bottlenecks, conflicts, or inefficiencies that specialized developers might miss. They can also switch easily between frontend and backend

development, providing flexibility in resource allocation—this is particularly useful during emergencies.

WHAT ARE THE TECHNOLOGIES USED IN FULL-STACK DEVELOPMENT?

1]Frontend technology

Frontend technology (or client software) focuses on the client-facing side of development. You code any parts of the graphical user interface (GUI) or the features that users interact with using frontend technology.

The most common frontend languages are:

- Hypertext Markup Language (HTML)
- Cascading Style Sheets (CSS)
- JavaScript

2] Backend technology

Backend technology (or server software) coordinates the exchange of information between the frontend and the server running a webpage. It allows an application to communicate with the main server. The most common backend languages are Hypertext Preprocessor (PHP), Ruby, Java, and Python.

1] API layer

The API layer receives interactions from the frontend and then communicates these to the storage layer. This layer acts as a bridge between the frontend and the backend.

2] Storage layer

The storage layer manages and stores any application data. It will communicate with databases to write and read data, while providing access to data. A notification layer will send notifications from the frontend to the backend and vice versa. This allows an application to communicate and trigger responses.

3] Business logic layer

The business logic layer is the main core of the backend. Backend developers configure processing logic in this layer, changing the response a certain API request delivers.

WHAT ARE FULL-STACK DEVELOPMENT FRAMEWORKS?

A framework is a collection of reusable software components that reduces the time it takes to make new applications. It includes resources like code libraries, compilers, debuggers, and APIs. Using frameworks helps to improve code quality, reduce development time, and adhere to software security.

1] Django

Django is a high-level Python web framework that encourages rapid development and clean, pragmatic design. Django follows the *batteries-included* philosophy and provides almost everything developers might want to do "out of the box." Because it's written in Python, it's easy to integrate with other Python libraries. It's also a good choice for projects that may involve machine learning, data analysis, and other complex tasks. Django handles much of the configuration itself, allowing developers to focus solely on application development. It's widely used in building robust and scalable web applications.

2] Spring Boot

Spring Boot is an extension of the Spring framework, aimed at simplifying the initial setup and development process. Written in Java, Spring Boot focuses on building production-ready applications quickly by eliminating much of the boilerplate code and configuration that characterizes Java development. It's highly flexible and compatible with almost all kinds of application requirements, making it a good fit for building enterprise-level applications.

WHAT ARE SOME POPULAR STACKS IN FULL-STACK DEVELOPMENT?

Below are common stacks that you may find in full-stack development.

1] LAMP stack

Linux, Apache, MySQL, and PHP make up the LAMP stack. Applications built using LAMP architecture work as follows.

2] MEAN stack

The MEAN stack is a LAMP alternative that consists of MongoDB, Express.js, Angular, and Node.js. It is based on Java technologies.

3] LEMP stack

LEMP uses four open-source technologies—Linux, Nginx, MySQL, and PHP. The Nginx web server receives HTTP requests and forwards them to the PHP component.

ADVANTAGES OF FULL STACK DEVELOPMENT IN THE CLOUD

- **Scalability**: Cloud computing platforms offer scalable infrastructure resources that can handle fluctuations in application traffic. Full-stack development in the cloud allows developers to easily scale applications up or down based on demand, ensuring optimal performance and cost efficiency.
- **Cost efficiency**: By leveraging cloud resources, full-stack developers can reduce infrastructure costs. They only pay for the resources they use, avoiding the upfront investment in hardware. Additionally, the pay-as-you-go cloud model allows developers to cost-effectively scale their applications, eliminating the need for over-provisioning.
- **Faster development and deployment**: Cloud platforms provide preconfigured environments and services that speed up the development process. Full-stack developers can leverage cloud-based development tools, continuous integration/continuous deployment (CI/CD) pipelines, and automated testing frameworks to streamline the entire software delivery lifecycle and reduce time to market.
- **Improved collaboration**: Full-stack development in the cloud supports collaboration between developers and other stakeholders. Cloud-based project management tools, version control systems, and communication platforms enable seamless collaboration and ensure effective teamwork, even in distributed or remote settings.

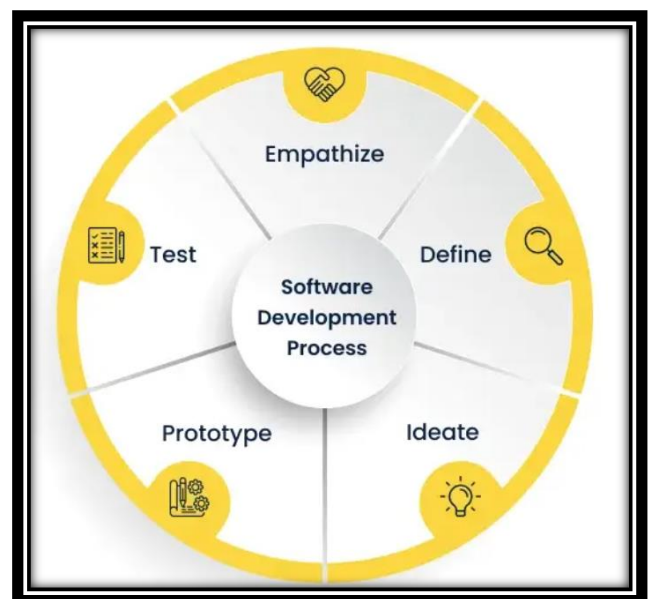
DESIGN THINKING

WHAT IS DESIGN THINKING SOFTWARE DEVELOPMENT?

Design thinking is a human-centered approach to problem-solving that can be used in a variety of fields, including software development.

It focuses on understanding the needs of users and developing solutions that meet those.

Design thinking is important in software development because it helps developers to create products that are both user-friendly and effective.



THE DESIGN THINKING IN SOFTWARE DEVELOPMENT PROCESS

Design thinking principles in the software development lifecycle consists of five key stages.

1. Empathize

In this initial stage, developers aim to understand users deeply. They gather insights into users' needs, desires, and challenges. This understanding forms the foundation for the entire design process.

2. Define

After gaining a comprehensive understanding of users, the next step is to define the specific problem that needs solving. This phase narrows the focus on what the software should address and the user pain points it should alleviate.

3. Ideate

This stage encourages developers to brainstorm and generate as many potential solutions to the defined problem as possible. The emphasis is on creativity and thinking outside the box.

4. Prototype

With a range of ideas in hand, developers create prototypes. These are simplified, early versions of the software that allow them to test and visualize their ideas. Prototypes can range from basic sketches to interactive digital models.

5. Test

Once a prototype is ready, it's tested by real users. Their feedback is gathered and used to refine the prototype. This iterative testing helps developers refine their ideas and create a more user-friendly solution.

BENEFITS OF DESIGN THINKING IN SOFTWARE DEVELOPMENT



UNIT-2

INDEX

1] AGIL METHODOLOGY

- **Epics**
- **Risk Management**
- **Creating user stories for the Epic**
- **Creating Acceptance Criteria**
- **Sprint Planning**
- **Backlog Refinement**
- **Sprint Demo**
- **Burn down charts**
- **Sprint retrospective**

2] Design principles

3] Architectural patterns

4] Design methods for security

5] Design methods for Data stores

6] Data base design

7] Design principles for – UI / UX

AGILE METHODOLOGY

Agile Software Development is an iterative and incremental approach to software development that emphasizes the importance of delivering a working product quickly and frequently.

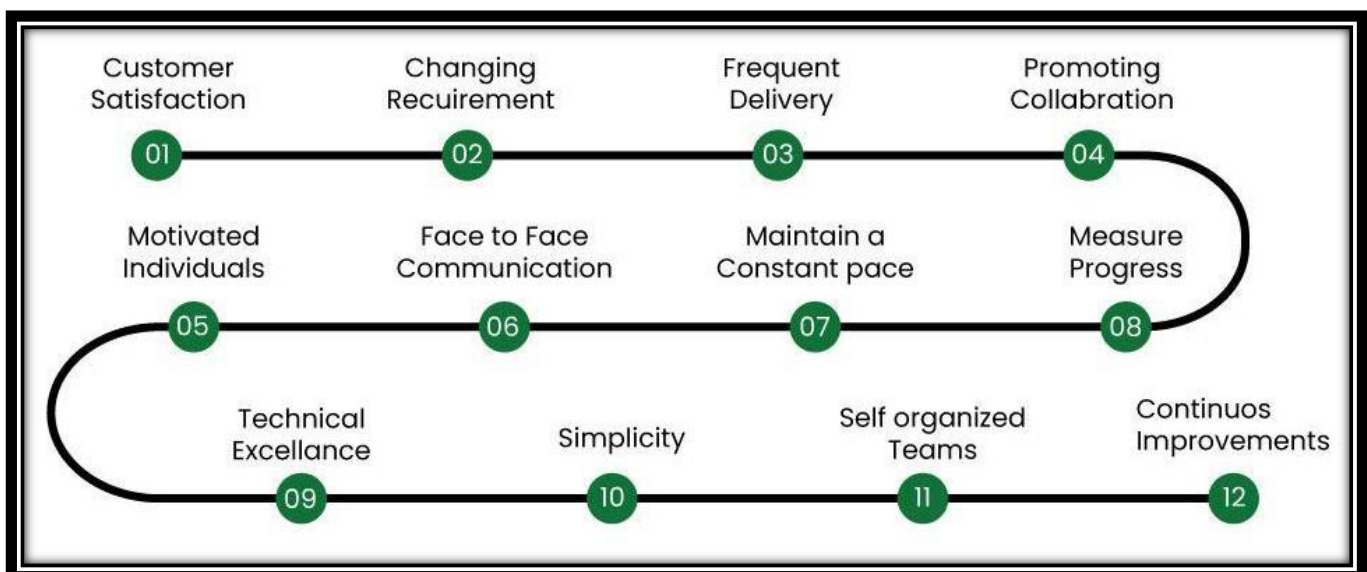
It involves close collaboration between the development team and the customer to ensure that the product meets their needs and expectations.

Agile methodology values flexibility, collaboration, and customer satisfaction.

WHY AGILE IS USED?

1. **Creating Tangible Value:** Agile places a high priority on creating tangible value as soon as possible in a project.
2. **Concentrate on Value-Added Work:** Agile methodology promotes teams to concentrate on producing functional and value-added product increments, hence reducing the amount of time and energy allocated to non-essential tasks.
3. **Agile as a Mindset:** Agile represents a shift in culture that values adaptability, collaboration, and client happiness.
4. **Quick Response to Change:** Agile fosters a culture that allows teams to respond swiftly to constantly shifting priorities and requirements.
5. **Regular Demonstrations:** Agile techniques place a strong emphasis on regular demonstrations of project progress.
6. **Cross-Functional Teams:** Agile fosters self-organizing, cross-functional teams that share information effectively, communicate more effectively and feel more like a unit.

12 PRINCIPLES OF AGILE SOFTWARE DEVELOPMENT



THE AGILE SOFTWARE DEVELOPMENT PROCES

1. **Requirements Gathering:** The customer's requirements for the software are gathered and prioritized.
2. **Planning:** The development team creates a plan for delivering the software, including the features that will be delivered in each iteration.
3. **Development:** The development team works to build the software, using frequent and rapid iterations.
4. **Testing:** The software is thoroughly tested to ensure that it meets the customer's requirements and is of high quality.
5. **Deployment:** The software is deployed and put into use.
6. **Maintenance:** The software is maintained to ensure that it continues to meet the customer's needs and expectations.

Advantages AGILE SOFTWARE DEVELOPMENT

1. **Increased collaboration and communication:** Agile Software Development Methodology emphasize collaboration and communication among team members, stakeholders, and customers. This leads to improved understanding, better alignment, and increased buy-in from everyone involved.
2. **Flexibility and adaptability:** Agile methodologies are designed to be flexible and adaptable, making it easier to respond to changes in requirements, priorities, or market conditions. This allows teams to quickly adjust their approach and stay focused on delivering value.
3. **Improved quality and reliability:** Agile methodologies place a strong emphasis on testing, quality assurance, and continuous improvement. This helps to ensure that software is delivered with high quality and reliability, reducing the risk of defects or issues that can impact the user experience.
4. **Enhanced customer satisfaction:** Agile methodologies prioritize customer satisfaction and focus on delivering value to the customer. By involving customers throughout the development process, teams can ensure that the software meets their needs and expectations.
5. **Increased team morale and motivation:** Agile methodologies promote a collaborative, supportive, and positive work environment. This can lead to increased team morale, motivation, and engagement, which can in turn lead to better productivity, higher quality work, and improved outcomes.

AGILE EPICS: -

An agile epic is a body of work that can be broken down into specific tasks (called user stories) based on the needs/requests of customers or end-users. Epics are an important practice for agile and DevOps teams.

An epic is a large body of work that can be broken down into a number of smaller stories, or sometimes called “Issues” in Jira. Epics often encompass multiple teams, on multiple projects, and can even be tracked on multiple boards.

Epics are almost always delivered over a set of sprints. As a team learns more about an epic through development and customer feedback, user stories will be added and removed as necessary.

ROAD MAP

Roadmaps in Jira Software are team-level roadmaps useful for planning large pieces of work several months in advance at the Epic level within a single project.

COST ESTIMATION

Cost estimation in software engineering is the process of predicting the resources (money, time, and people) necessary to finish a project within the defined scope.

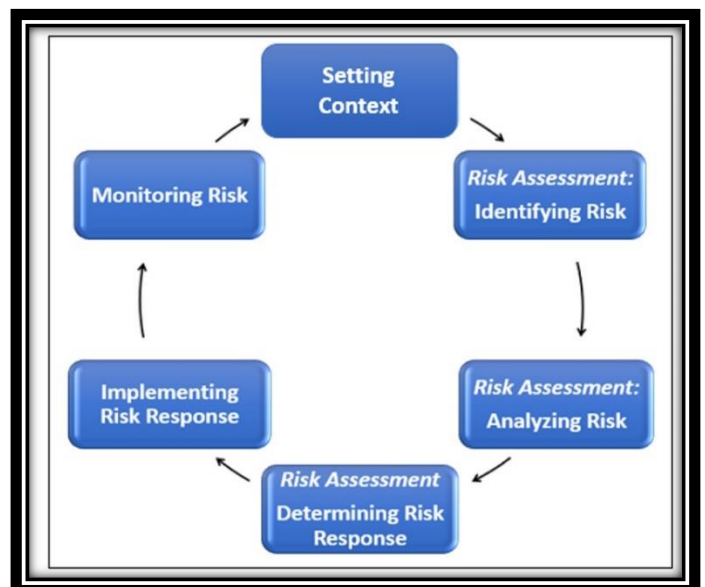
What is AGILE RISK MANAGEMENT?

There are three main classifications of risks which can affect a software project:

1. Project risks: Project risks concern different forms of budgetary, schedule, personnel, resource, and customer-related problems.

2. Technical risks: Technical risks concern potential method, implementation, interfacing, testing, and maintenance issue. Most technical risks appear due to the development team's insufficient knowledge about the project.

3. Business risks: This type of risks contains risks of building an excellent product that no one needs, losing budgetary or personnel commitments, etc.

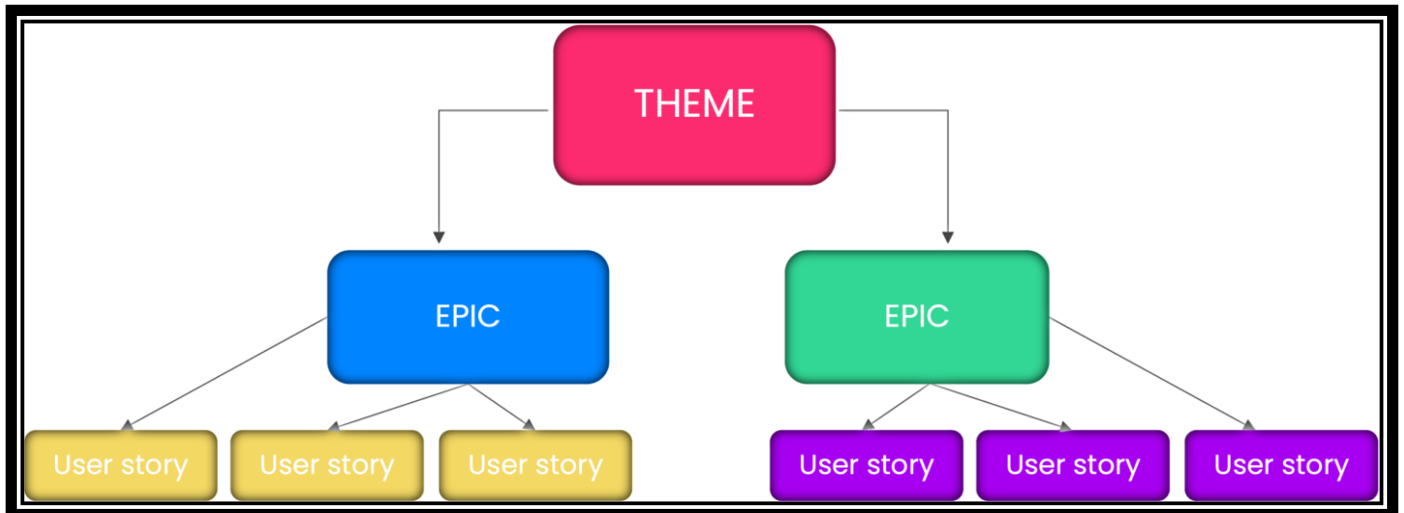


What are USER STORIES?

A user story is the smallest unit of work in an **Agile framework**. It's an end goal, not a feature, expressed from the software user's perspective.

A user story is an informal, general explanation of a software feature written from the perspective of the end user or customer.

The purpose of a user story is to articulate how a piece of work will deliver a particular value back to the customer.



BENEFITS of USER STORIES

- To keep the focus on the user
- To enable collaboration
- To drive creative solutions.
- To create momentum.

What are the ACCEPTANCE CRITERIA?

Acceptance criteria (AC) are the conditions a software product must meet to be accepted by a user or a customer.

They are unique for each user story and define the feature behavior from the end-user's perspective.

Good acceptance criteria should possess specific qualities.

1] Clarity: They should be straightforward and easy to understand for all team members, avoiding any confusion.

2] Conciseness: The criteria should communicate the necessary information without unnecessary detail.

3] Testability: Each criterion must be verifiable, allowing testers to clearly determine whether it has been met.

4] Result-oriented: The focus should be on delivering results that gratify the customer, emphasizing the end benefit or value.

What is SCRUM?

Scrum is an agile project management framework, that helps teams structure and manage their work through a set of values, principles, and practices.

What is SPRINT?

A sprint is a time-boxed period when a team works on a set amount of work to achieve specific goals.

WHAT IS SPRINT PLANNING?

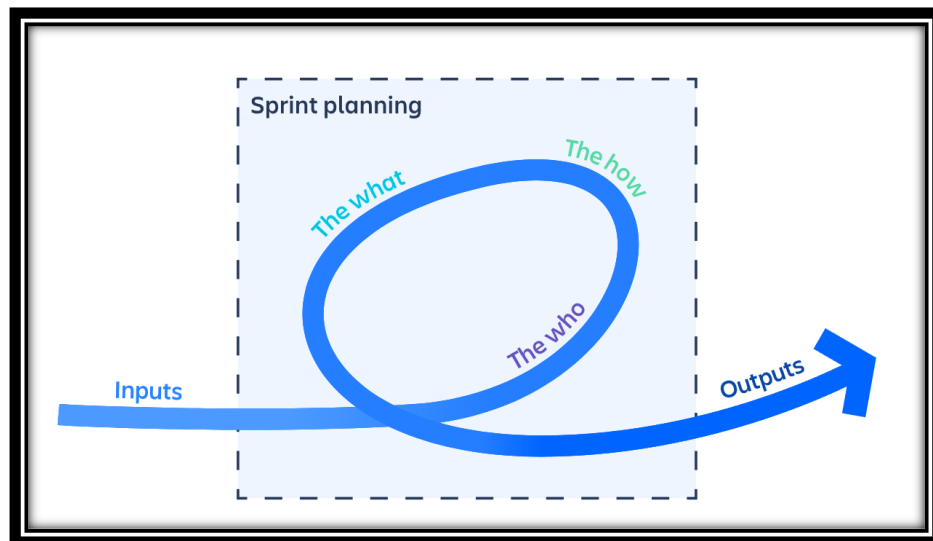
Sprint planning is an event in scrum that kicks off the sprint. The purpose of sprint planning is to define what can be delivered in the sprint and how that work will be achieved. Sprint planning is done in collaboration with the whole scrum team.

In scrum, the sprint is a set period of time where all the work is done. However, before you can leap into action you have to set up the sprint. You need to decide on how long the time box is going to be, the sprint goal, and where you're going to start. The sprint planning session kicks off the sprint by setting the agenda and focus. If done correctly, it also creates an environment where the team is motivated, challenged, and can be successful. Bad sprint plans can derail the team by setting unrealistic expectations.

- **The What** – The product owner describes the objective (or goal) of the sprint and what backlog items contribute to that goal. The scrum team decides what can be done in the coming sprint and what they will do during the sprint to make that happen.
- **The How** – The development team plans the work necessary to deliver the sprint goal. Ultimately, the resulting sprint plan is a negotiation between the development team and product owner based on value and effort.
- **The Who** – You cannot do sprint planning without the product owner or the development team. The product owner defines the goal based on the value that they

seek. The development team needs to understand how they can or cannot deliver that goal. If either is missing from this event, it makes planning the sprint almost impossible.

- **The Inputs** – A great starting point for the sprint plan is the product backlog as it provides a list of ‘stuff’ that could potentially be part of the current sprint. The team should also look at the existing work done in the increment and have a view to capacity.
- **The Outputs** – The most important outcome for the sprint planning meeting is that the team can describe the goal of the sprint and how it will start working toward that goal. This is made visible in the sprint backlog.



What is BACKLOG REFINEMENT?

Backlog refinement is the process of reviewing, ranking, and editing the product backlog.

Backlog refinement is an important tool in the product development process, because, it helps the development team build only the features and functionalities that the customer wants and the business needs.

Some of the activities that occur during this refinement of the backlog include:

- removing **user stories** that no longer appear relevant,
- creating new user stories in response to newly discovered needs,
- re-assessing the relative priority of stories,
- assigning estimates to stories that have yet to receive one,
- correcting estimates in light of newly discovered information
- splitting user stories that are high priority but too coarse-grained to fit in an upcoming iteration.

WHAT IS A SPRINT DEMO?

A sprint demo, an integral event in Agile development, is a meeting where the development team showcases the work accomplished during a sprint.

This event occurs at the end of each iteration and is planned during the sprint planning session using the product and sprint backlogs.

The sprint demo's purpose is to provide transparency, gather feedback from stakeholders, and ensure the product aligns with the desired goals and user needs.

What is a BURNDOWN CHART?

A burndown chart shows the amount of work that has been completed in an epic or sprint, and the total work remaining.

Burndown charts are used to predict the team's likelihood of completing their work in the time available.

- If you notice that the team consistently finishes work early, this might be a sign that they aren't committing to enough work during sprint planning.
- If they consistently miss their forecast, this might be a sign that they've committed to too much work.
- If the burndown chart shows a sharp drop during the sprint, this might be a sign that work has not been estimated accurately, or broken down properly.

WHAT IS A SPRINT RETROSPECTIVE?

A sprint retrospective is a review conducted after a sprint that plays a key role in the Agile methodology. A sprint retrospective aims to determine what went well and where team had problems and identify areas where team can improve.

DESIGN PRINCIPLES

1. **Availability** refers to the percentage of time that the infrastructure, system, or solution remains operational under normal circumstances in order to serve its intended purpose
2. **Performance** is an indicator of how well a software system or component meets its requirements for timeliness. Timeliness is measured in terms of response time or throughput. The response time is the time required to respond to a request

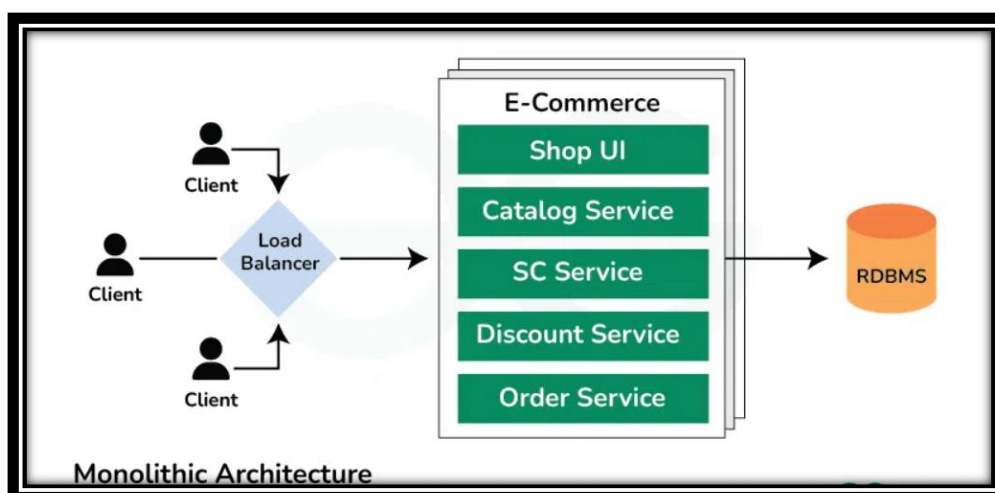
3. **Consistency** defined as the requirement that a series of measurements of the same project carried out by different raters using the same method should produce similar results, is one of the most important aspects to be taken into account in the measurement methods of the software.
4. **Scalability** is the measure of a system's ability to increase or decrease in performance and cost in response to changes in application and system processing demands.
5. **Manageability** How efficiently and easily a software system can be monitored and maintained to keep the system performing, secure, and running smoothly.
6. **Software cost** estimation is the process of predicting the effort required to develop a software system.

ARCHITECTURE

1) MONOLITHIC ARCHITECTURE

A monolithic architecture is a traditional approach to designing software where an entire application is built as a single, indivisible unit. In this architecture, all the different components of the application, such as the user interface, business logic, and data access layer, are tightly integrated and deployed together.

- This means that any changes or updates to the application require modifying and redeploying the entire monolith.
- Monolithic architectures are often characterized by their simplicity and ease of development, especially for small to medium-sized applications.
- However, they can become complex and difficult to maintain as the size and complexity of the application grow.



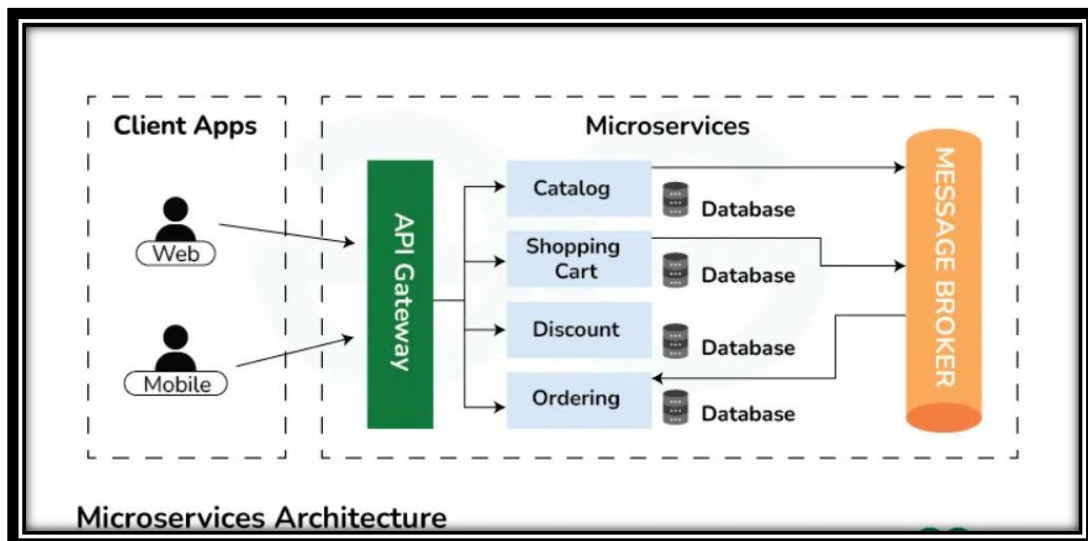
Advantages of using a Monolithic Architecture

- **Simplicity**
- **Development Speed**
- **Deployment**
- **Debugging**

2] MICROSERVICES ARCHITECTURE?

In a microservices architecture, an application is built as a collection of small, independent services, each representing a specific business capability. These services are loosely coupled and communicate with each other over a network, often using lightweight protocols like HTTP or messaging queues.

- Each service is responsible for a single functionality or feature of the application and can be developed, deployed, and scaled independently.
- The Microservice architecture has a significant impact on the relationship between the application and the database.
- Instead of sharing a single database with other microservices, each microservice has its own database. It often results in duplication of some data, but having a database per microservice is essential if you want to benefit from this architecture, as it ensures loose



coupling.

Advantages of using a MICROSERVICES ARCHITECTURE

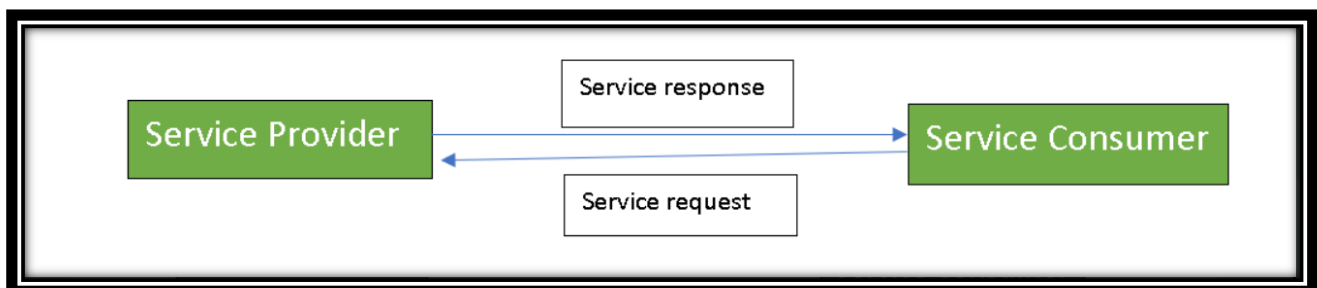
- **Scalability**

- **Flexibility**
- **Resilience**
- **Agility**
- **Easier Maintenance**
- **Technology Diversity**

3] SERVICE-ORIENTED ARCHITECTURE

Service-Oriented Architecture (SOA) is a stage in the evolution of application development and integration. It defines a way to make software components reusable using the interfaces.

- SOA allows users to combine a large number of facilities from existing services to form applications.
- SOA encompasses a set of design principles that structure system development and provide means for integrating components into a coherent and decentralized system.
- SOA-based computing packages functionalities into a set of interoperable services, which can be integrated into different software systems belonging to separate business



domains.

1. **Service provider:** The service provider is the maintainer of the service and the organization that makes available one or more services for others to use. To advertise services, the provider can publish them in a registry, together with a service contract that specifies the nature of the service, how to use it, the requirements for the service, and the fees charged.
2. **Service consumer:** The service consumer can locate the service metadata in the registry and develop the required client components to bind and use the service.

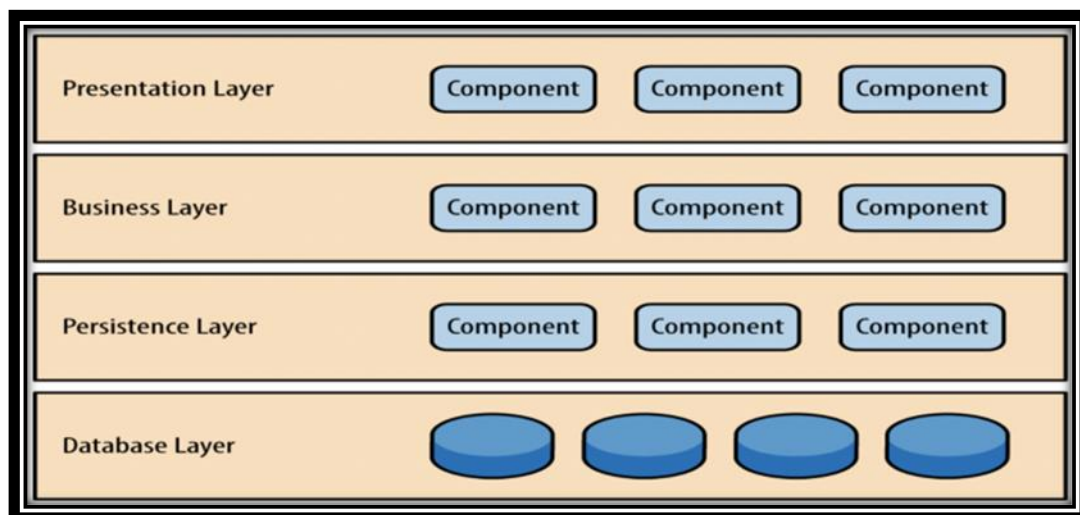
Services might aggregate information and data retrieved from other services or create workflows of services to satisfy the request of a given service consumer. This practice is known as service orchestration.

Advantages of SOA

- **Service reusability**
- **Easy maintenance**
- **Platform independent**
- **Availability**
- **Reliability**
- **Scalability**

4] LAYERED ARCHITECTURE

Layered Architecture is a software design pattern that is widely used in modern software development. It is a logical and structured approach to software design that separates different functional modules of an application into four separate horizontal layers, each with a specific set of responsibilities. This separation of concerns makes the code more modular, maintainable, and scalable, and enables easier testing and debugging.



1] Presentation Layer

The presentation layer is the topmost layer of the architecture, responsible for handling the user interface and displaying data to the user. This layer interacts with the application layer to retrieve the data and provides a visual representation of the data to the user. In web applications, this layer is often implemented using HTML, CSS, and JavaScript.

2] Application Layer

The application layer is responsible for handling business logic and coordinating interactions between different components. This layer receives input from the presentation layer and processes it before passing it down to the domain layer. It is also responsible for

communicating with external systems and services. Think of this layer as the maestro in an orchestra, that guides the musicians on what, when and how to play their instrument, but who doesn't actually play any instrument himself.

3] Domain Layer

The domain layer contains the business logic and rules of the application. This layer represents the core of the application and defines how the application processes data and interacts with the external world. It is responsible for ensuring the consistency and validity of the data and defines the behavior of the application. **4] Infrastructure Layer**

The infrastructure layer is responsible for handling external dependencies and providing services to the other layers. This layer interacts with databases, file systems, and other external systems. It also provides services such as logging, caching, and authentication to the other layers. By separating infrastructure concerns from the rest of the application, it becomes easier to maintain and replace external dependencies without affecting the core functionality of the application.

Advantages of Layered Architecture

1. **Modular structure**: By dividing the application into different layers, developers can isolate specific functionalities and easily modify or replace them without affecting other parts of the system. This allows for more agile development and easier maintenance.
2. **Separation of concerns**: Each layer is responsible for a specific set of tasks, and there is limited communication between the layers. This helps to ensure that the code is organized and easy to understand, making it easier to test and debug.
3. **Code reusability**: Because each layer has a well-defined interface, it is possible to reuse code across different applications or even different layers of the same application. This can save developers time and effort by reducing the amount of code they need to write and maintain.

WHAT IS APPLICATION SECURITY?

Application security describes security measures at the application level that aim to prevent data or code within the app from being stolen or hijacked.

It encompasses the security considerations that happen during application development and design, but it also involves systems and approaches to protect apps after they get deployed.

Application security is the process of developing, adding, and testing security features within applications to prevent security vulnerabilities against threats such as unauthorized access and modification.

Types of application security

- **Authentication**
- **Authorization**
- **Encryption**
- **Logging**
- **Application security testing**

TOKEN?

- Token-based authentication is a protocol that generates encrypted security tokens.
- It enables users to verify their identity to websites, which then generates a unique encrypted authentication token.
- That token provides users with access to protected pages and resources for a limited period of time without having to re-enter their username and password

A Token is a computer-generated code that acts as a digitally encoded signature of a user. They are used to authenticate the identity of a user to access any website or application network.

A token is classified into two types,

- **Physical token:** A Physical token use a tangible device to store the information of a user. Here, the secret key is a physical device that can be used to prove the user's identity. Two elements of physical tokens are hard tokens and soft tokens.
 - **Web token:** The authentication via web token is a fully digital process. Here, the server and the client interface interact upon the user's request. The client sends the user credentials to the server and the server verifies them, generates the digital signature, and sends it back to the client.
1. **User:** A person who intends to access the network carrying his/her username & password.
 2. **Client-server:** A client is a front-end login interface where the user first interacts to enroll for the restricted resource.
 3. **Authorization server:** A backend unit handling the task of verifying the credentials, generating tokens, and send to the user.
 4. **Resource server:** It is the entry point where the user enters the access token. If verified, the network greets users with a welcome note.

COOKIES?

Cookies are small pieces of data that are stored on a user's device when they visit a website. These data files are typically stored in the user's web browser. Depending on the browser and platform, cookies can be found in different locations. Commonly, cookies are stored in a specific folder or directory on the user's computer or device.

Types of Cookies:

1] Session cookies: Session cookies are also known as temporary cookies which are present as long as the user browser is open. Session cookies are deleted once the browser is closed and the user's session is inactive (time-based).

2] Persistent cookies: Persistent cookies are also known as permanent/long-term cookies. Persistent cookies can last longer than session cookies. Persistent cookies are stored on the user's device for a specific period (for less than 6 months). Persistent cookies are most often used for long-term tracking and remembering user preferences.

3] First-party cookies: First-party cookies are set by the website that you are currently visiting. First-party cookies are generally used to provide a good user experience, collect the analytics data, remember language settings etc.

4] Third-party cookies: Third-party cookies are set by the domains that you are not visiting. Third-party cookies are mostly used for cross-site tracking and advertising purposes these cookies collect data about your browsing habits and serve ads according to it.

SAML

SAML is an acronym used to describe the **Security Assertion Markup Language** (SAML).

Its primary role in online security is that it enables you to access multiple web applications using one set of login credentials.

It works by passing authentication information in a particular format between two parties, usually an identity provider (IdP) and a web application.

How it Works

SAML is an open standard used for authentication. Based upon the Extensible Markup Language (XML) format, web applications use SAML to transfer authentication data between two parties - the identity provider (IdP) and the service provider (SP).

The technology industry created SAML to simplify the authentication process where users needed to access multiple, independent web applications across domains.

ASSIGNMENT: -

- 1] OpenID,
- 2] Third party access,
- 3] Multi factor authentication.

WHAT IS ENCRYPTION?

Encryption is a way of scrambling data so that only authorized parties can understand the information. In technical terms, it is the process of converting human-readable plaintext to incomprehensible text, also known as cipher text. In simpler terms, encryption takes readable data and alters it so that it appears random.

How does encryption work?

Encryption is a mathematical process that alters data using an encryption algorithm and a key. Imagine if Alice sends the message "Hello" to Bob, but she replaces each letter in her message with the letter that comes two places later in the alphabet. Instead of "Hello," her message now reads "Jgnnq." Fortunately, Bob knows that the key is "2" and can decrypt her message back to "Hello."



Design methods for DATASTORES

Big Data includes huge volume, high velocity, and extensible variety of data.

1. **Structured data –**

Structured data is data whose elements are addressable for effective analysis. It has been organized into a formatted repository that is typically a database. It concerns all data which can be stored in database SQL in a table with rows and columns. They have relational keys and can easily be mapped into pre-designed fields. Today, those data are most processed in the development and simplest way to manage information. *Example:* Relational data.

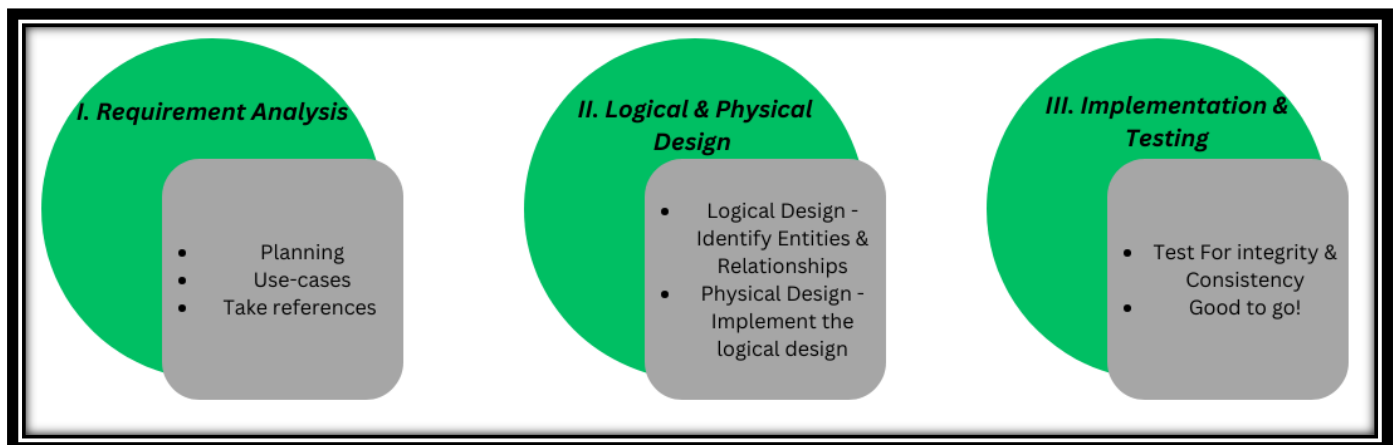
2. Semi-Structured data –

Semi-structured data is information that does not reside in a relational database but that has some organizational properties that make it easier to analyze. With some processes, you can store them in the relation database (it could be very hard for some kind of semi-structured data), but Semi-structured exist to ease space. *Example: XML data.*

3. Unstructured data –

Unstructured data is a data which is not organized in a predefined manner or does not have a predefined data model, thus it is not a good fit for a mainstream relational database. So for Unstructured data, there are alternative platforms for storing and managing, it is increasingly prevalent in IT systems and is used by organizations in a variety of business intelligence and analytics applications. *Example: Word, PDF, Text, Media logs.*

DATABASE DESIGN LIFECYCLE



1. Requirement Analysis

It's very crucial to understand the requirements of our application so that you can think in productive terms. And imply appropriate integrity constraints to maintain the data integrity & consistency.

2. Logical & Physical Design

1. **Logical Data Model Design:** This phase consists of coming up with a high-level design of our database based on initially gathered requirements to structure & organize our data accordingly. A high-level overview on paper is made of the database without considering the physical level design, this phase proceeds by **identifying** the kind of data to be stored and what relationship will exist among those data.

2. **Physical Design of Data Model:** This phase involves the implementation of the logical design made in the previous stage. All the relationships among data and integrity constraints are implemented to maintain consistency & generate the actual database.

3. Testing

Finally, after implementing the physical design of the database, we're ready to input the data & test our integrity. This phase involves testing our database for its integrity to see if something got left out or, if anything new to add & then integrating it with the desired application.

WHAT IS UI DESIGN?

UI (User interface) Design is the interface of the website or a digital product through which a user interacts. The main goal of UI is to catch and grab user attention by making the user interface look more pleasing and aesthetic. Nowadays UI is very important for any organization to scale up because there are lots of businesses offering similar products or services in the market are available.

Importance of UI Design

- **User Experience:** A well designed UI can make customers to navigate easily and complete their tasks which leads to positive user experience.
- **Engagement:** A good UI can make users stay on the platform or make them visit more often.
- **Branding:** UI communicates the product's personality and features to the users, so visually appealing UI can make brands recognizable by the users.
- **Efficiency:** A well designed UI can reduce time and efforts which increases user efficiency which leads to user satisfaction.

WHAT IS UX DESIGN?

UX (User Experience) Design is a concept in design which helps to design a better user experience for a product. Its user centered design concept makes the website or digital product easy to navigate, design according to user behavior, to match user expectations of the product to engage users and accomplish the organizations goals.

Importance of UX Design

- **Increased engagement:** A well-designed UX can increase user engagement with the product which leads users to use it frequently.

- **Improved Usability:** UX design focuses on creating products that are easy to use and navigate which can improve user satisfaction.
- **Better brand perception:** A positive user experience is very important to improve the user's perception of the brand.
- **Competitive Advantage:** A better User Experience can differentiate a product or a website from its competitors.
- **Reduced development cost:** A user centered design approach can help to identify issues during the development process. Which can reduce the costs of fixing those errors later on.

Principles of User Interface and User Experience (UI/UX) Design

1. **Simplicity:** Simplicity is one of the most important principles of Ui. The design should be easy to use and navigate, requiring minimal effort.
2. **User-centered Design:** The UI should be designed in such a way that focuses user's needs and preferences first. It should be easily accessible by all users including users with disabilities.
3. **Visibility:** The UI should provide a clear and visible interface highlighting the important tasks need to performed by the user.
4. **Consistency:** Consistency is necessary for a good UI design. Consistent design elements like colors, typography and layout helps users to navigate properly also enhances the brand's authority.
5. **Feedback:** Feedback is a very important component of good UI design as it helps users understand the result of actions performed by them in form of visual, sound effect or animations. One example for feedback in UI design is the message user gets on successfully submitting a form.
6. **Clarity:** Clarity is a fundamental principle of UX design. It helps users to complete their goals. The clarity of a web-page or a digital product should be considered more important than a visually appealing design.
7. **Accessibility:** Accessibility is a very important principle of UX design which ensures the services or website is usable by everyone including those with disabilities. text-to-speech and keyboard navigation are examples of accessibility.
8. **Improved Usability:** UX design focuses on easy to use and navigate which can improve user satisfaction and loyalty.

UNIT-3

INDEX

1| DevOps

2| DevOps Practices

1. Continuous Integration
2. Continuous Delivery
3. Continuous deployment
4. Microservices
5. monitoring
6. Infrastructure as Code
7. Configuration Management

3| VERSION CONTROL SYSTEM

4| Git

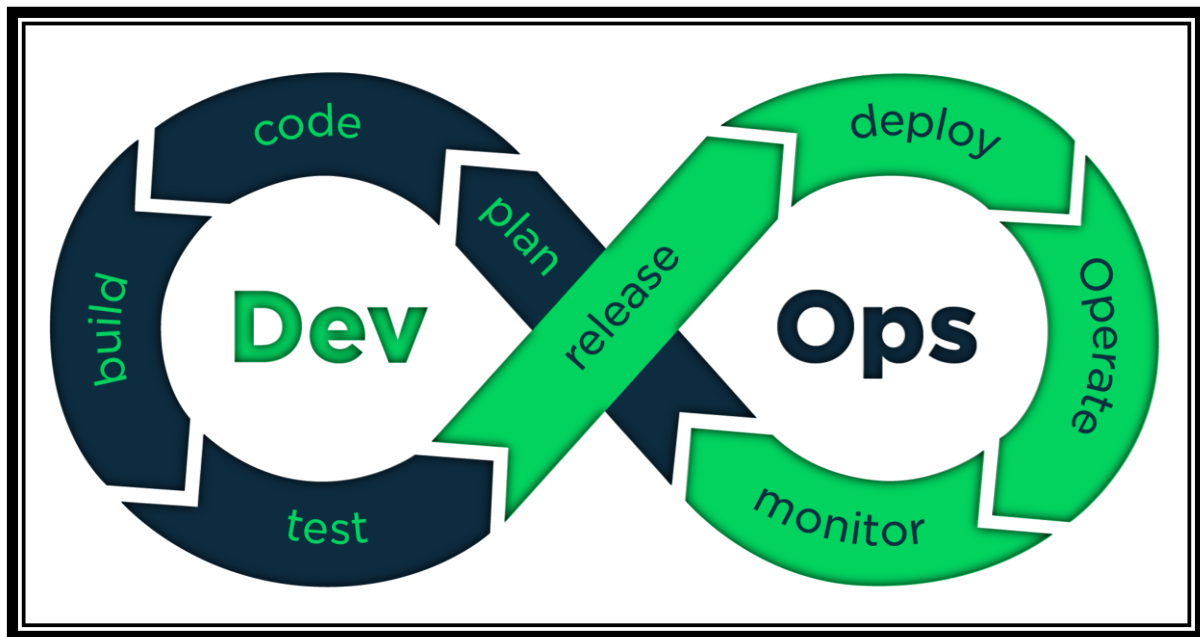
5|cloud computing

DevOps

DevOps is the combination of cultural philosophies, practices, and tools that increases an organization's ability to deliver applications and services at high velocity. [or],

DevOps is an evolving philosophy and framework that encourages faster, better application development and faster release of new or revised software features or products to customers.

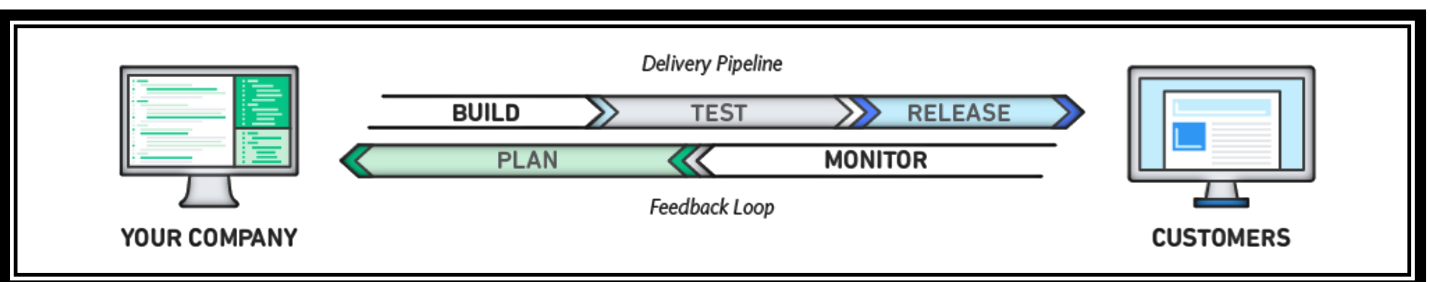
The practice of DevOps encourages smoother, continuous communication, collaboration, integration, visibility, and transparency between application development teams (Dev) and their IT operations team (Ops).



How DevOps Works

Under a DevOps model, development and operations teams are no longer siloes(independent). Sometimes, these two teams are merged into a single team where the engineers work across the entire application lifecycle, from development and test to deployment to operations, and develop a range of skills not limited to a single function.

In some DevOps models, quality assurance and security teams may also become more tightly integrated with development and operations and throughout the application lifecycle.



Benefits of DevOps

- Faster, better product delivery
- Faster issue resolution and reduced complexity
- Greater scalability and availability
- More stable operating environments
- Better resource utilization
- Greater automation
- Greater visibility into system outcomes
- Greater innovation

DevOps Practices

1. Continuous Integration
2. Continuous Delivery
3. Continuous deployment
4. Microservices
5. monitoring
6. Infrastructure as Code
7. Configuration Management

1] what is Continuous Integration? [C I]

Continuous integration is a DevOps software development practice, where developers regularly merge their code changes into a central repository, after which automated builds and tests are run.

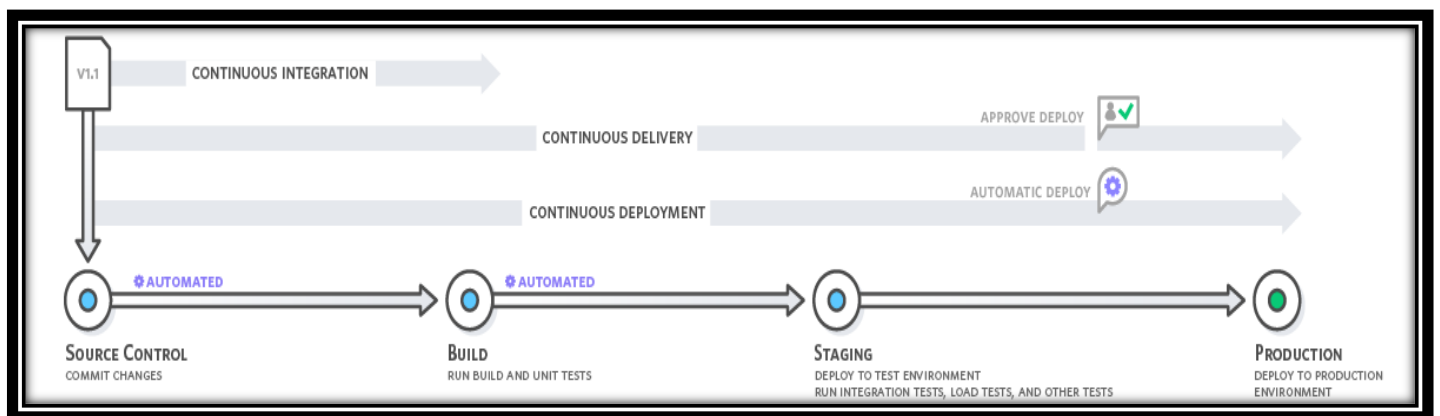
Continuous integration most often refers to the build or integration stage of the software release process and entails both an automation component and a cultural component.

The key goals of continuous integration are to find and address bugs quicker, improve software quality, and reduce the time it takes to validate and release new software updates.

Why is Continuous Integration Needed?

In the past, developers on a team might work in isolation for an extended period of time and only merge their changes to the master branch once their work was completed. This made merging code changes difficult and time-consuming, and also resulted in bugs accumulating for a long time without correction. These factors made it harder to deliver updates to customers quickly.

With continuous integration, developers frequently commit to a shared repository using a version control system such as Git. Prior to each commit, developers may choose to run local unit tests on their code as an extra verification layer before integrating. A continuous integration service automatically builds and runs unit tests on the new code changes to immediately surface any errors.



Continuous integration refers to the build and unit testing stages of the software release process. Every revision that is committed triggers an automated build and test.

Continuous Integration Benefits

1. Improve Developer Productivity
2. Find and Address Bugs Quicker
3. Deliver Updates Faster

2] what is Continuous Delivery?

Continuous delivery is a software development practice where code changes are automatically prepared for a release to production.

A pillar of modern application development, continuous delivery expands upon continuous integration by deploying all code changes to a testing environment and/or a production environment after the build stage.

Continuous delivery lets developers automate testing beyond just unit tests so they can verify application updates across multiple dimensions before deploying to customers. These tests may include UI testing, load testing, integration testing, API reliability testing, etc.

This helps developers more thoroughly validate updates and pre-emptively discover issues. With the cloud, it is easy and cost-effective to automate the creation and replication of multiple environments for testing, which was previously difficult to do on-premises.

Continuous Delivery Benefits

1] Automate the Software Release Process

2] Improve Developer Productivity

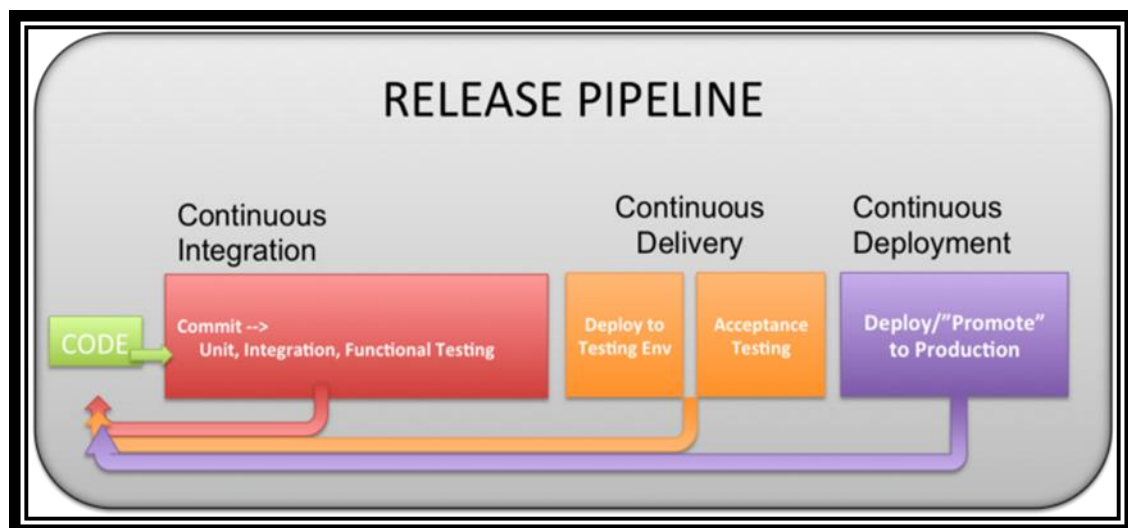
3] Find and Address Bugs Quicker

4] Deliver Updates Faster

3] Continuous Deployment (CD).

CD practice automates the release of new or changed code into production.

A company doing continuous deployment might release code or feature changes several times per day. The use of container technologies, such as Docker and Kubernetes, can enable continuous deployment by helping to maintain consistency of the code across different deployment platforms and environments.

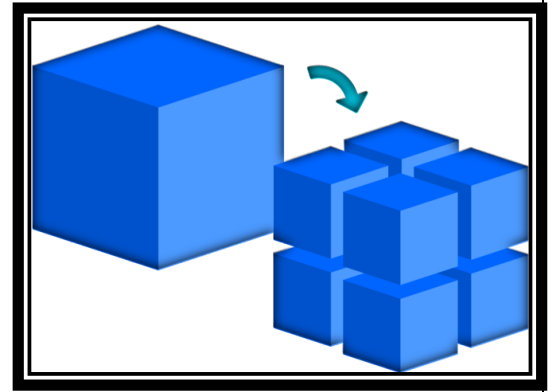


4] What are Microservices?

A microservice is a function responsible for a single piece of logic. Multiple microservices combine to create distributed applications, such as Jira Software.

There are three types of microservices:

1. **Domain microservices** loosely couple services with related functionality.
2. **Integration microservices** facilitate interaction between unrelated applications.
3. **Unit-of-work microservices** handle singular functions.



Microservices interact with each other using application programming interfaces (APIs). Developers working on individual services can learn the internal workings of the other microservices, which is another advantage over monolithic architecture.

Microservices-based architecture has several distinguishing features. Developers can develop and deploy their services independently using the languages and technologies that suit their components.

Communication between microservices is API-based, allowing access to data from various sources without modification. Individual services can scale based on demand, saving costs and ensuring availability. These attributes make microservices-based distributed applications flexible and easy to maintain.

Benefits of microservices

- 1] Resilience and fault isolation
- 2] Agility
- 3] Technology diversity
- 4] Improved maintenance

What is microservices architecture used for?

- **E-commerce sites** like eBay use separate microservices for shopping carts, mobile apps, and messaging.
- **Financial institutions** like Bank of America use microservices to communicate with outside services. They also use them to handle functions such as user authorizations and displaying transactions.
- **Social media platforms** like Instagram and Facebook use microservices to display users' news feeds, messages, notifications, and friend networks.

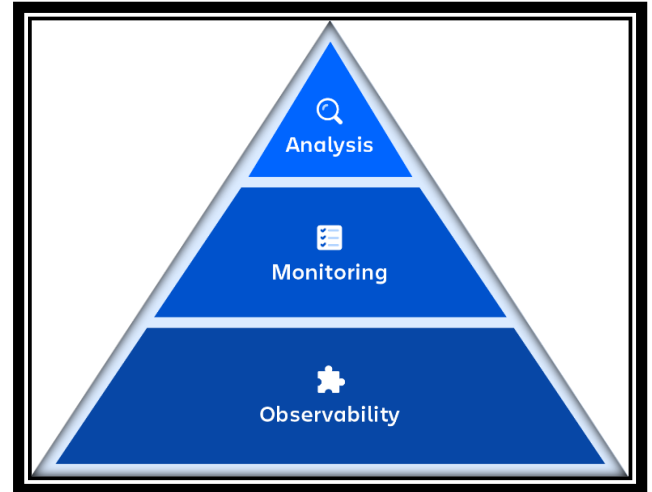
5] What is DevOps Monitoring?

DevOps monitoring entails overseeing the entire development process from planning, development, integration and testing, deployment, and operations.

It involves a complete and real-time view of the status of applications, services, and infrastructure in the production environment.

Features such as real-time streaming, historical replay, and visualizations are critical components of application and service monitoring.

DevOps monitoring allows teams to respond to any degradation in the customer experience, quickly and automatically.



The importance of DevOps monitoring

- 1] Frequent code changes demand visibility
- 2] Automated collaboration
- 3] Experimentation
- 4] Change management
- 5] Dependent system monitoring

6] What is Infrastructure as code? [IaC]: -

Infrastructure as code is an IT infrastructure management process that applies best practices from DevOps software development to the management of cloud infrastructure resources.

Applicable infrastructure resources are virtual machines, networks, load balancers, databases, and other networked applications.

It is a form of configuration management that codifies an organization's infrastructure resources into text files. These infrastructure files are then committed to a version control system like Git. The version control repository enables feature branch and pull request workflows, which are foundational dependencies of CI/CD.

7] What is Configuration Management?

Configuration management occurs when a configuration platform is used to automate, monitor, design and manage otherwise manual configuration processes. System-wide changes take place across servers and networks, storage, applications, and other managed systems.

An important function of configuration management is defining the state of each system. By orchestrating these processes with a platform, organizations can ensure consistency across integrated systems and increase efficiency.

There are primary components that go into the comprehensive configuration management required for DevOps:

1] Artifact Repository

An artifact repository is meant to store machine files. This can include binaries, test data, and libraries. Effectively, it's a database for files that people don't generally use. In DevOps, artifacts, like binaries, are a natural result of continuous integration. DevOps developers are always pushing out builds which, in turn, create artifact files that need to be stored, but not necessarily accessed.

2] Source Code Repository

Conversely, the source code repository is a database of source code which developers use. This database serves as a container for all the working code. Source code aside, it stores a number of useful components including various scripts and configuration files.

3] Configuration Management Database

A configuration management database or (CMDB) is a relational database that spans across multiple systems and applications related to configuration management, including services, servers, applications, and databases to name a few.

What is VERSION CONTROL SYSTEM?

Version control systems are a category of software tools that helps in recording changes made to files by keeping a track of modifications done in the code.

A version control system is a kind of software that helps the developer team to efficiently communicate and manage(track) all the changes that have been made to the source code along with the information like who made and what changes have been made. A separate branch is created for every contributor who made the changes and the changes aren't merged into the original source code unless all are analyzed as soon as the changes are green signaled they merged to the main source code. It not only keeps source code organized but also improves productivity by making the development process smooth.

Basically Version control system keeps track on changes made on a particular software and take a snapshot of every modification. Let's suppose if a team of developer add some new functionalities in an application and the updated version is not working properly so as the version control system keeps track of our work so with the help of version control system we can omit the new changes and continue with the previous version.

Benefits of the version control system:

1. Enhances the project development speed by providing efficient collaboration,
2. Leverages the productivity, expedites product delivery, and skills of the employees through better communication and assistance,
3. Reduce possibilities of errors and conflicts meanwhile project development through traceability to every small change,
4. Employees or contributors of the project can contribute from anywhere irrespective of the different geographical locations through this **VCS**,
5. For each different contributor to the project, a different working copy is maintained and not merged to the main file unless the working copy is validated. The most popular example is **Git, Helix core, Microsoft**
6. Helps in recovery in case of any disaster or contingent situation,
7. Informs us about Who, What, When, Why, changes have been made.

Types of Version Control Systems:

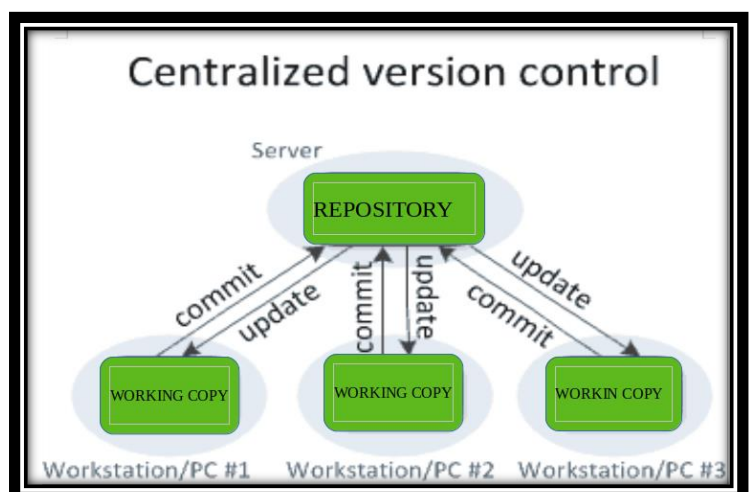
1] Local Version Control Systems: It is one of the simplest forms and has a database that kept all the changes to files under revision control. RCS is one of the most common VCS tools. It keeps patch sets (differences between files) in a special format on disk. By adding up all the patches it can then re-create what any file looked like at any point in time.

2] Centralized Version Control Systems:

Centralized version control systems contain just one repository globally and every user need to commit for reflecting one's changes in the repository. It is possible for others to see your changes by updating.

Two things are required to make your changes visible to others which are:

- **You commit**
- **They update**

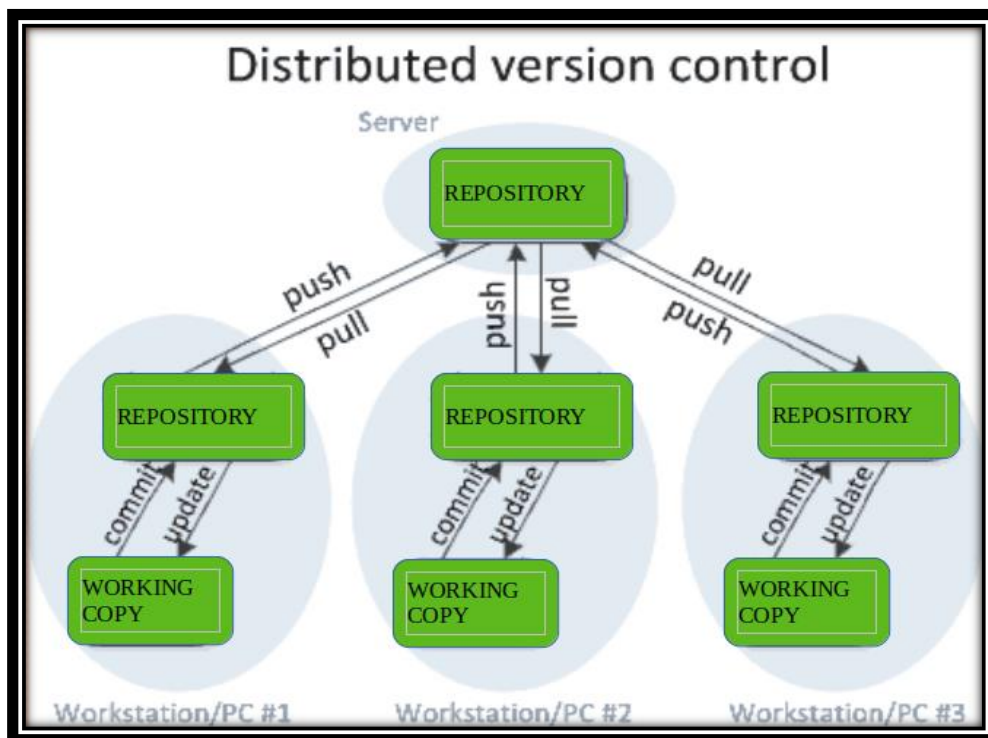


3] Distributed Version Control Systems:

Distributed version control systems contain multiple repositories. Each user has their own repository and working copy. Just committing your changes will not give others access to your changes. This is because commit will reflect those changes in your local repository and you need to push them in order to make them visible on the central repository. Similarly, when you update, you do not get others' changes unless you have first pulled those changes into your repository.

To make your changes visible to others, 4 things are required:

- **You commit**
- **You push**
- **They pull**
- **They update**



What is Git?

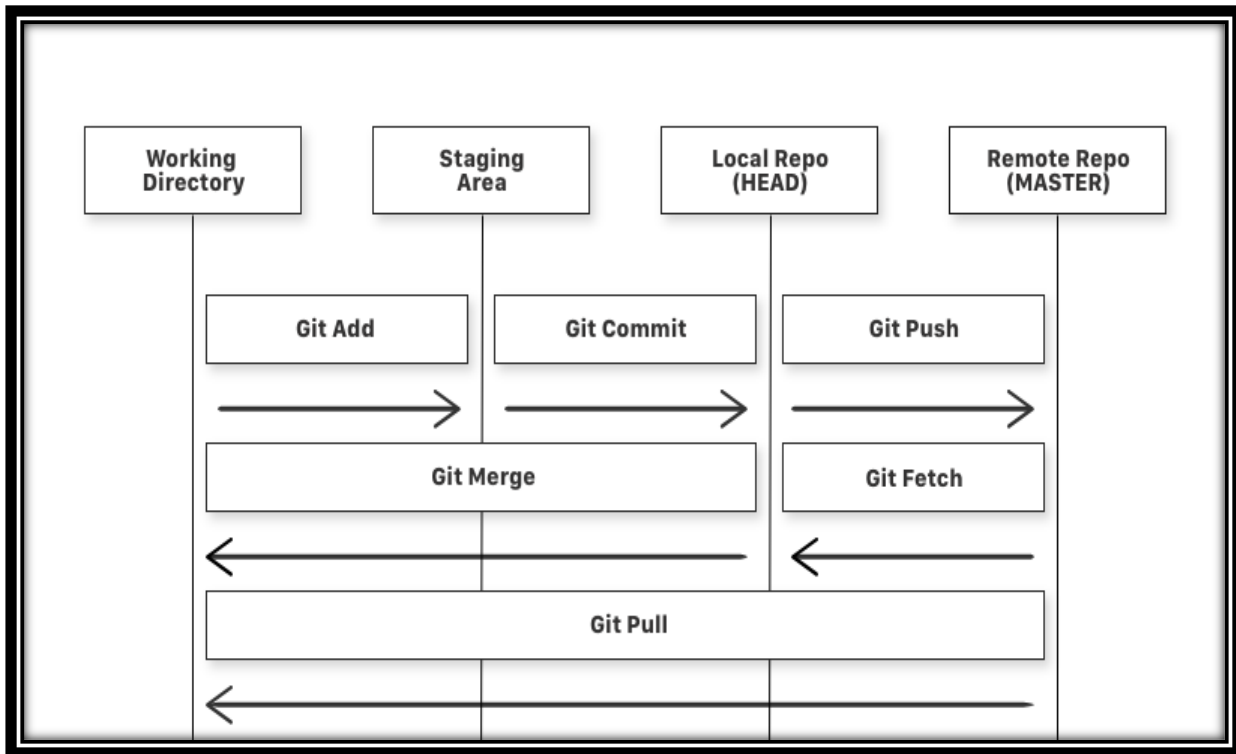
Git is a Distributed version-control system for tracking changes in computer files and coordinating work on those files among multiple people. It does not necessarily rely on a central server to store all the versions of a project's files.

Instead, every user "clones" a copy of a repository and has the **full** history of the project on their own hard drive.

This clone has *all* of the metadata of the original while the original itself is stored on a self-hosted server or a third party hosting service like GitHub.

There are four fundamental elements in the Git Workflow.

1. Working Directory,
2. Staging Area,
3. Local Repository
4. Remote Repository.



Git commands

- **git add** is a command used to add a file that is in the working directory to the staging area.
- **git commit** is a command used to add all files that are staged to the local repository.
- **git push** is a command used to add all committed files in the local repository to the remote repository. So in the remote repository, all files and changes will be visible to anyone with access to the remote repository.
- **git fetch** is a command used to get files from the remote repository to the local repository but not into the working directory.
- **git merge** is a command used to get the files from the local repository into the working directory.
- **git pull** is command used to get files from the remote repository directly into the working directory. It is equivalent to a git fetch and a **git merge**.

▪ Creating and switching to new branches

New Branches

The git branch command can be used to create a new branch. When you want to start a new feature, you create a new branch off main using git branch new_branch . Once created you can then use git checkout new_branch to switch to that branch.

▪ Switching between branches

You can use the git switch – command to undo any changes you make and return to your previous branch. If you instead want to keep your changes and continue from here, you can use git switch -c <new-branch-name> to create a new branch from this point.

▪ Merging local branches together

To merge branches locally, use git checkout to switch to the branch you want to merge into. This branch is typically the main branch. Next, use git merge and specify the name of the other branch to bring into this branch. This example merges the jeff/feature1 branch into the main branch.

GitHub

Account creation and configuration

You typically configure your global username and email address after installing Git.

- 1.Toet your global username/email configuration:
- 2.Open the command line.
- 3.Set your username: git config –global user.name “FIRST_NAME LAST_NAME”
- 4.Set your email address: git config –global user.email MY_NAME@example.com

Create and push to repositories

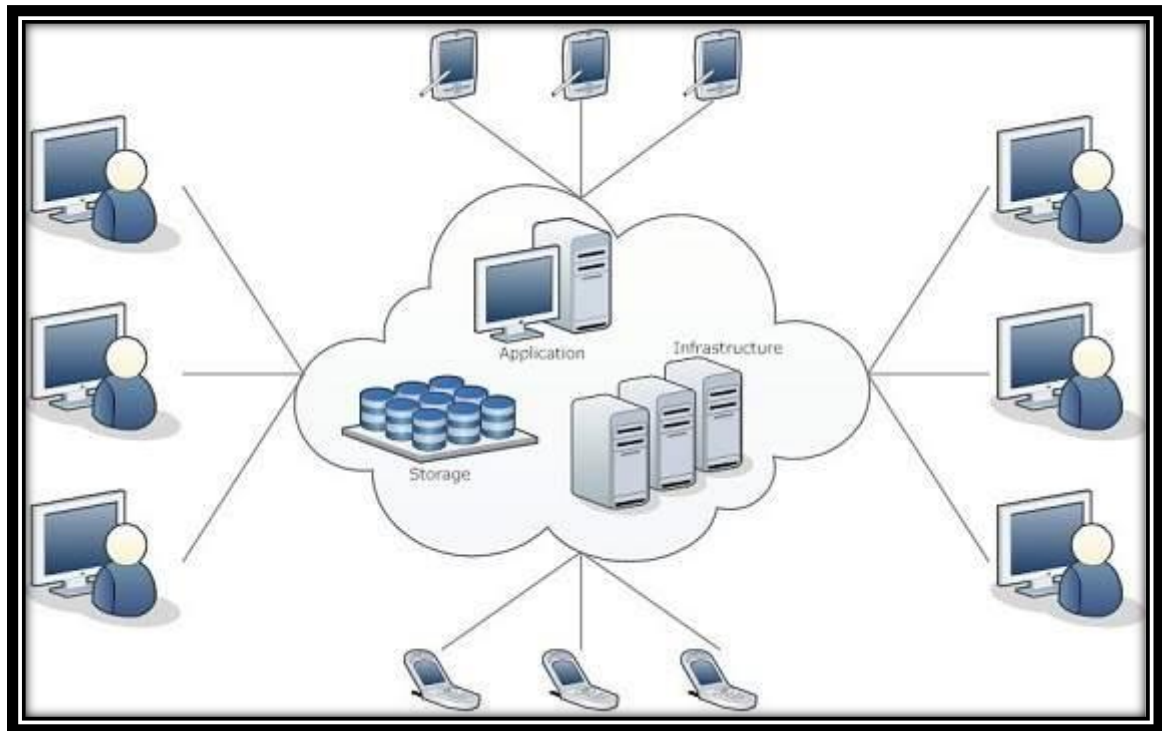
- Create a new repository on GitHub.com.
- Open Terminal Git Bash.
- Change the current working directory to your local project.
- Use the init command to initialize the local directory as a Git repository,
- Add the files in your new local repository.
- Commit the files that you’ve staged in your local repository.

CLOUD BASICS

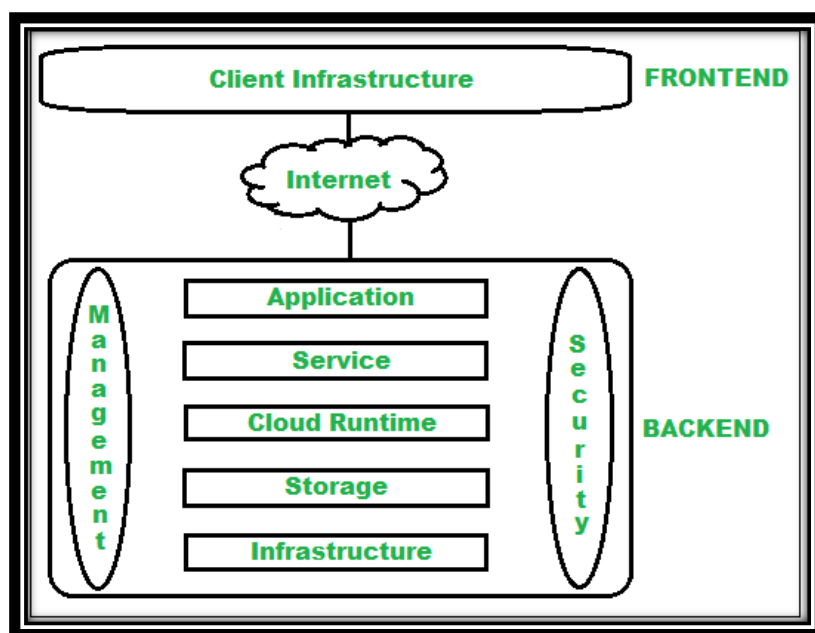
What is Cloud Computing?

Cloud Computing refers to **manipulating, configuring, and accessing** the hardware and software resources remotely. It offers online data storage, infrastructure, and application.

Cloud computing offers **platform independency**, as the software is not required to be installed locally on the PC. Hence, the Cloud Computing is making our business applications **mobile** and **collaborative**.



Cloud Computing Architecture:



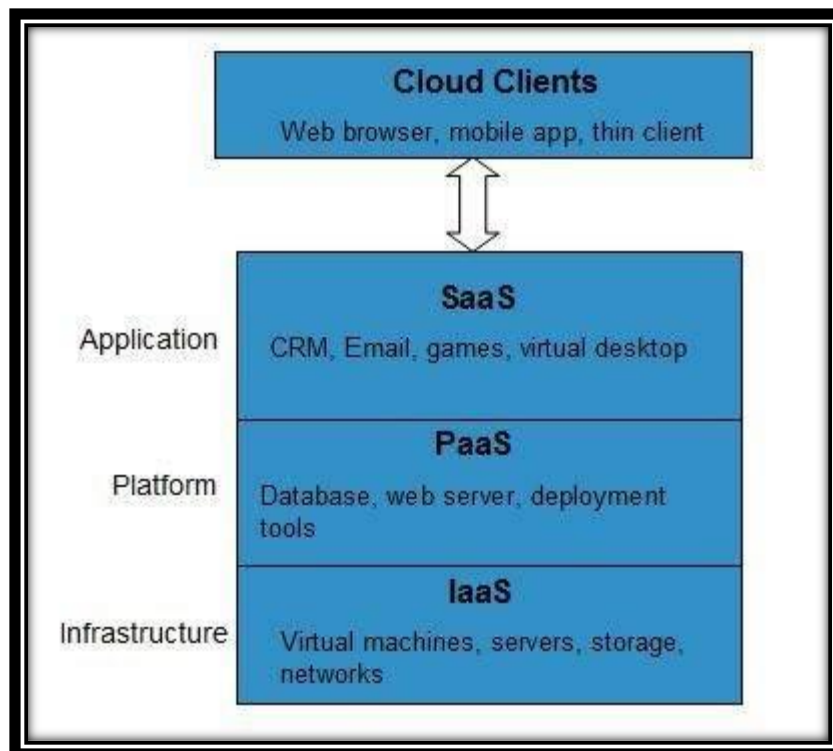
Front End:

The client uses the front end, which contains a client-side interface and application. Both of these components are important to access the Cloud computing platform. The front end includes web servers (Chrome, Firefox, Opera, etc.), clients, and mobile devices.

Back End:

The backend part helps you manage all the resources needed to provide Cloud computing services. This Cloud architecture part includes a security mechanism, a large amount of data storage, servers, virtual machines, traffic control mechanisms, etc.

1] Services models



A] SaaS (Software as a Service)

SaaS or software as a service is a software distribution model in which applications are hosted by a vendor or service provider and made available to customers over a network (internet). SaaS is becoming an increasingly prevalent delivery model as underlying technologies that supports **Service Oriented Architecture (SOA) or Web Services**. Through internet this service is available to users anywhere in the world.

B] PaaS (Platform as a Service)

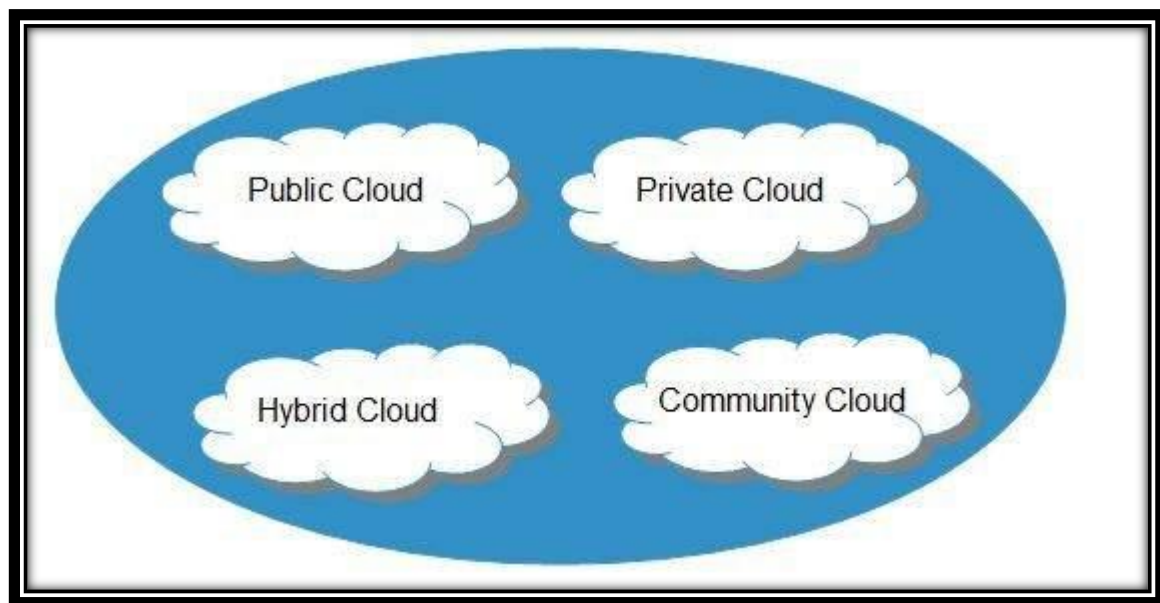
Platform as a service, is referred as PaaS, it provides a platform and environment to allow developers to build applications and services. This service is hosted in the cloud and accessed by the users via internet.

C] IaaS (Infrastructure as a Service)

IaaS is one of the fundamental service model of cloud computing alongside PaaS. It provides access to computing resources in a virtualized environment “the cloud” on internet. It provides computing infrastructure like virtual server space, network connections, bandwidth, load balancers and IP addresses. The pool of hardware resource is extracted from multiple servers and networks usually distributed across numerous data centres. This provides redundancy and reliability to IaaS.

2| Deployment Models

Deployment models define the type of access to the cloud, i.e., how the cloud is located? Cloud can have any of the four types of access: Public, Private, Hybrid, and Community.



Public Cloud

The **public cloud** allows systems and services to be easily accessible to the general public. Public cloud may be less secure because of its openness.

Private Cloud

The **private cloud** allows systems and services to be accessible within an organization. It is more secured because of its private nature.

Community Cloud

The **community cloud** allows systems and services to be accessible by a group of organizations.

Hybrid Cloud

The **hybrid cloud** is a mixture of public and private cloud, in which the critical activities are performed using private cloud while the non-critical activities are performed using public cloud.

Benefits of Cloud Computing Architecture:

- Makes overall cloud computing system simpler.
- Improves data processing requirements.
- Helps in providing high security.
- Makes it more modularized.
- Results in better disaster recovery.
- Gives good user accessibility.

VIRTUALIZATION AND CLOUD COMPUTING

The main enabling technology for Cloud Computing is Virtualization.

Virtualization is the partitioning of a single physical server into multiple logical servers. Once the physical server is divided, each logical server behaves like a physical server and can run an operating system and applications independently. Many popular companies like VMware and Microsoft provide virtualization services. Instead of using your PC for storage and computation, you can use their virtual servers. They are fast, cost-effective, and less time-consuming.

Network Virtualization: It is a method of combining the available resources in a network by splitting up the available bandwidth into channels. Each channel is independent of others and can be assigned to a specific server or device in real time.

Storage Virtualization: It is the pooling of physical storage from multiple network storage devices into what appears to be a single storage device that is managed from a central console. Storage virtualization is commonly used in storage area networks (SANs).

Server Virtualization: Server virtualization is the masking of server resources like processors, RAM, operating system, etc., from server users. Server virtualization intends to increase resource sharing and reduce the burden and complexity of computation from users.

UNIT-4

INDEX

1] HTML, CSS, JavaScript...

2] JS objects

3]JSON

4] ES6

5] Type Script

BASICS OF HTML

- HTML stands for hypertext markup language.
- HTML is the standard markup language for creating web pages.
- HTML describes the structure of a web page.
- HTML consists of a series of elements.
- HTML elements tell the browser how to display the content.
- HTML elements label pieces.

A simple HTML document example:

```
<!DOCTYPE HTML>
<html>
<head>
<title>page title</title>
</head>
<body>
<h1>This is my first Heading</h1>
<p>My first paragraph</P>
</body>
</html>
```

- The <!DOCTYPE HTML> declaration defines that this document is an HTML5 document.
- HTML5 is a markup language used for structuring and presenting content on the world wide web. It is the fifth & final major HTML version that is a world wide web consortium recommend. The current satisfaction. The current specification is known as the HTML living standard.
- The <html> element is the root element of an HTML page
- The <head> element contains meta information about the HTML page.
- The <title> element specifies a title for the HTML page (which is shown in the browser's title bar or in the page's tab).
- The <body> element defines the document's body. & it is a container for all the visible contents. Such as heading, Paragraphs, images, hyperlinks, tables, lists etc.
- The <h1> elements defines a large heading
- The <p> element defines a paragraph.

What is an HTML element?

An HTML element is defined by a start tag, some content & an end tag.

- `<br>` - **Break line**
- `<strong>` - **Important text**
- `<i>` - **Italic text**
- `<em>` - **emphasized text**
- `<mark>` - **marked text**
- `<small>` - **smaller text**
- `<del>` - **deleted text**
- `<ins>` - **inserted text**
- `<sub>` - **subscript text**
- `<sup>` - **superscript text**
- `<center>` - **center**
- `<hr />` - **horizontal line**
- `<b>` - **bold**
- `<button>` **for button**
- **Etc.....**

HTML Attributes

HTML attributes provide additional information about HTML elements.

- All HTML elements can have **attributes**
- Attributes provide **additional information** about elements
- Attributes are always specified in **the start tag**
- Attributes usually come in name/value pairs like: **name="value"**

The href Attribute

The `<a>` tag defines a hyperlink. The href attribute specifies the URL of the page the link goes to:

Example

```
<a href="https://www.w3schools.com">Visit W3Schools</a>
```

The src Attribute

The tag is used to embed an image in an HTML page. The src attribute specifies the path to the image to be displayed:

Example

```

```

The width and height Attributes

The tag should also contain the width and height attributes, which specify the width and height of the image (in pixels):

Example

```

```

The alt Attribute

The required alt attribute for the tag specifies an alternate text for an image, if the image for some reason cannot be displayed. This can be due to a slow connection, or an error in the src attribute, or if the user uses a screen reader.

Example

```

```

```

```

The style Attribute

The style attribute is used to add styles to an element, such as color, font, size, and more.

Example

```
<p style="color:red;">This is a red paragraph.</p>
```

The lang Attribute

You should always include the lang attribute inside the <html> tag, to declare the language of the Web page. This is meant to assist search engines and browsers.

The following example specifies English as the language:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<body>
```

```
...
```

```
</body>
```

```
</html>
```

The title Attribute

The title attribute defines some extra information about an element.

The value of the title attribute will be displayed as a tooltip when you mouse over the element:

Example

```
<p title="I'm a tooltip">This is a paragraph.</p>
```

The <div> Element

The <div> element is used as a container for other HTML elements.

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<style>
```

```
div {
```

```
    background-color: #FFF4A3;
```

```
}
```

```
</style>
```

```
<body>
```

```
<h1>HTML DIV Example</h1>
```

```
<div>
```

```
    <h2>London</h2>
```

```
    <p>London is the capital city of England.</p>
```

```
    <p>London has over 13 million inhabitants.</p>
```

```
</div>
```

```
<p>The yellow background is added to demonstrate the footprint of the DIV element.</p>
```

```
</body>
```

```
</html>
```

HTML **class** Attribute

The HTML class attribute is used to specify a class for an HTML element.

Multiple HTML elements can share the same class.

The class attribute is often used to point to a class name in a style sheet. It can also be used by a JavaScript to access and manipulate elements with the specific class name.

In the following example we have three <div> elements with a class attribute with the value of "city". All of the three <div> elements will be styled equally according to the. city style definition in the head section:

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

.city {
  background-color: tomato;
  color: white;
  border: 2px solid black;
  margin: 20px;
  padding: 20px;
}

</style>

</head>

<body>

<div class="city">
  <h2>London</h2>
  <p>London is the capital of England.</p>
</div>

<div class="city">
  <h2>Paris</h2>
  <p>Paris is the capital of France.</p>
```

```
</div>
<div class="city">
<h2>Tokyo</h2>
<p>Tokyo is the capital of Japan.</p>
</div>
</body>
</html>
```

HTML id Attribute

The HTML id attribute is used to specify a unique id for an HTML element.

You cannot have more than one element with the same id in an HTML document.

Example

```
<!DOCTYPE html>
<html>
<head>
<style>
#myHeader {
  background-color: lightblue;
  color: black;
  padding: 40px;
  text-align: center;
}
</style>
</head>
<body>
<h2>The id Attribute</h2>
<p>Use CSS to style an element with the id "myHeader":</p>
<h1 id="myHeader">My Header</h1>
```

```
</body>
```

```
</html>
```

MORE EXAMPLES:

Example 1:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>welcome to mysure</title>
```

```
</head>
```

```
<body>
```

```
<h1>ultismart infotech</h1>
```

```
<p>ultismart infotech llp is it solutions,product development,outsourcing...</p>
```

```
</body>
```

```
</html>
```

Example 2: style

```
<!DOCTYPE HTML>
```

```
<html>
```

```
<head>
```

```
<title> USING HTML STYLE</title>
```

```
</head>
```

```
<body>
```

```
<h1>HTML STYLE</h1>
```

```
<p>I AM AN ARMY GIRL</p>
```

```
<p style="color:red";>in the bangtan girl</p>
```

```
<p style="color:blue";>in the bangtan world</p>
```

```
<p style="color:orange";>my life is fantastic</p>
```

```
<p style="color:purple";>full of lifestick</p>
```

```
<p style="font-size:50px";>self-love is for important in this world</p>
```

```
</body>
```

```
</html>
```

Example 3: background colour

```
<!DOCTYPE HTML>
<html>
<head>
<title>html style </title>
</head>
<body>
<h1>HTML STYLE BACKGROUND COLOR</h1>
<body style="background-color:powderblue;">
</body>
</html>
```

Example 4: external link

```
<!DOCTYPE HTML>
<html>
<head>
<title>HTML HYPER LINK</title>
</head>
<body>
<h1>HTML HYPER LINK</h1>
  <a href="https://www.ultismartinfotech.com">visit this link</a>
</body>
</html>
```

Example 5: image source

```
<!DOCTYPE HTML>
<html>
<head>
<title>HTML IMAGE</title>
</head>
<body>
<h1>html image</h1>

```

```
</body>
</html>
```

Example 6: Headings

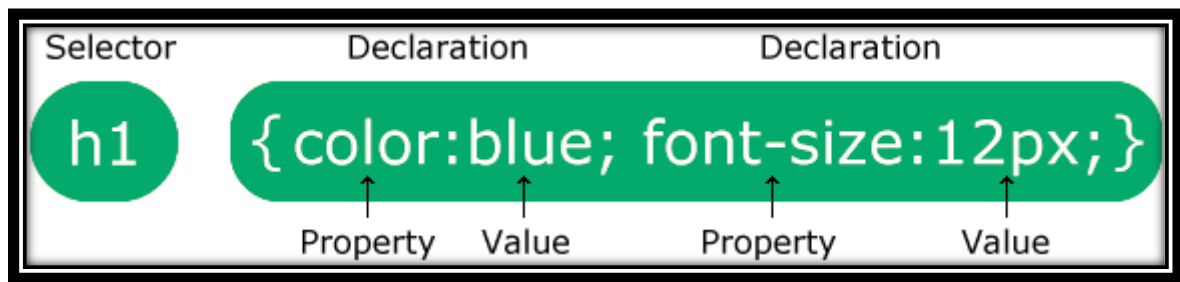
```
<!DOCTYPE HTML>
<html>
<head>
<title>this is my first webpage</title>
</head>
<body background-color:"pink";>
<h1>welcome to mysure</h1>
<h2>welcome to mysure</h2>
<h3>welcome to mysure</h3>
<h4>welcome to mysure</h4>
<h5>welcome to mysure</h5>
<h6>welcome to mysure</h6>
<pre>
<p>welcome to imagination world<br>where they can full fill our big dreams.in my
imagination world,<br>i love myself more then others<br>there is no restriction and no
rules</p>
</pre>
</body>
</html>
```

CSS

- CSS stands for Cascading Style Sheets
- CSS describes how HTML elements are to be displayed on screen, paper, or in other media
- CSS saves a lot of work. It can control the layout of multiple web pages all at once
- External style sheets are stored in CSS files

CSS Syntax

A CSS rule consists of a selector and a declaration block.



The selector points to the HTML element you want to style.

The declaration block contains one or more declarations separated by semicolons.

Each declaration includes a CSS property name and a value, separated by a colon.

Multiple CSS declarations are separated with semicolons, and declaration blocks are surrounded by curly braces.

Three Ways to Insert CSS

1) External CSS

With an external style sheet, you can change the look of an entire website by changing just one file

Each HTML page must include a reference to the external style sheet file inside the <link> element, inside the head section.

```
<html>
```

```
<head>
```

```
<link rel="stylesheet" href="mystyle.css">
```

```
</head>
```

```
<body>
```

```
<h1>This is a heading</h1>
```

```
<p>This is a paragraph.</p>
```

```
</body>
```

```
</html>
```

mystyle.css

```
body {  
    background-color: lightblue;  
}  
  
h1 {  
    color: navy;  
    margin-left: 20px;  
}
```

2] Internal CSS

An internal style sheet may be used if one single HTML page has a unique style. The internal style is defined inside the <style> element, inside the head section.

```
<html>  
<head>  
<style>  
body {  
    background-color: linen;  
}  
  
h1 {  
    color: maroon;  
    margin-left: 40px;  
}  
</style>
```

```
</head>
<body>
<h1>This is a heading</h1>
<p>This is a paragraph.</p>
</body>
</html>
```

3| Inline CSS

An inline style may be used to apply a unique style for a single element.

To use inline styles, add the style attribute to the relevant element. The style attribute can contain any CSS property.

```
<html>
<body>
<h1 style="color:blue;text-align:center;">This is a heading</h1>
<p style="color:red;">This is a paragraph.</p>
</body>
</html>
```

CSS Selectors [5]

1| The CSS element Selector

The element selector selects HTML elements based on the element name.

```
<html>
<head>
<style>
p {
  text-align: center;
```

```
color: red;
}
</style>
</head>
<body>
<p>Every paragraph will be affected by the style.</p>
<p id="para1">Me too!</p>
<p>And me!</p>
</body>
</html>
```

2] The CSS **id** Selector

The id selector uses the id attribute of an HTML element to select a specific element.

To select an element with a specific id, write a hash (#) character, followed by the id of the element.

```
<html>
<head>
<style>
#para1 {
  text-align: center;
  color: red;
}
</style>
</head>
<body>
<p id="para1">Hello World!</p>
<p>This paragraph is not affected by the style.</p>
```

```
</body>
```

```
</html>
```

3] The CSS **class** Selector

The class selector selects HTML elements with a specific class attribute.

To select elements with a specific class, write a period (.) character, followed by the class name.

```
<html>
```

```
<head>
```

```
<style>
```

```
.center {
```

```
  text-align: center;
```

```
  color: red;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1 class="center">Red and center-aligned heading</h1>
```

```
<p class="center">Red and center-aligned paragraph.</p>
```

```
</body>
```

```
</html>
```

4] The CSS **Universal** Selector

The universal selector (*) selects all HTML elements on the page.

```
<!DOCTYPE html>
```

```
<html><head>
```

```
<style>
```

```
* {  
  text-align: center;  
  color: blue;  
}  
</style></head>  
<body>  
<h1>Hello world!</h1>  
<p>Every element on the page will be affected by the style.</p>  
<p id="para1">Me too!</p>  
<p>And me!</p>  
</body></html>
```

5] The CSS **Grouping Selector**

The grouping selector selects all the HTML elements with the same style definitions.

Look at the following CSS code (the h1, h2, and p elements have the same style definitions):

```
<html>  
<head>  
<style>  
h1, h2, p {  
  text-align: center;  
  color: red;  
}  
</style>  
</head>  
<body>  
<h1>Hello World!</h1>  
<h2>Smaller heading!</h2>
```

<p>This is a paragraph.</p>

</body></html>

Data types

DataTypes	Description	Example
String	represents textual data	'hello', "hello world!" etc
Number	an integer or a floating-point	3, 3.234, 3e-2 etc.
BigInt	an integer with arbitrary precision	900719925124740999n , 1n etc.
Boolean	Any of two values: true or false	true and false
undefined	a data type whose variable is not initialized	let a;
null	denotes a null value	let a = null;
Symbol	unique and immutable	let value = Symbol('hello');
Object	key-value pairs of collection of data	let student = { };

Types of JavaScript Operators

There are different types of JavaScript operators:

- Arithmetic Operators
- Assignment Operators
- Comparison Operators
- Logical Operators
- Conditional Operators

What is JavaScript?

- An object-oriented computer programming language commonly used to create interactive effects within web browsers.
- JavaScript is a **text-based programming language used both on the client-side and server-side**.
- Where HTML and CSS are languages that give structure and style to web pages, JavaScript gives web pages interactive elements that engage a user
- JavaScript programs can be inserted almost anywhere into an HTML document using the `<script>` tag.

JavaScript is a lightweight, cross-platform, single-threaded, and interpreted compiled programming language. It is also known as the scripting language for webpages. It is well-known for the development of web pages, and many non-browser environments also use it.

Syntax:

`<script>`

`// JavaScript Code`

`</script>`

Example

`<!DOCTYPE html>`

```

<html lang="en">
<head>
  <title>
    Basic Example to Describe JavaScript
  </title>
</head>
<body>
  <script>
    console.log("Welcome to Gpt college");
  </script>
</body>
</html>

```

Keyword	Description
var	Used to declare a variable
let	Used to declare a block variable
const	Used to declare a block constant
if	Used to decide if certain block will get executed or not
switch	Executes a block of codes depending on different cases
for	Executes a block of statements till the condition is true
function	Used to declare a function
return	Used to exit a function
try	Used to handle errors in a block of statements
break	Used to terminate a loop

Keyword	Description
continue	Used to continue a loop

JavaScript Can Change HTML Content

One of many JavaScript HTML methods is `getElementById()`.

```
<html>
```

```
<body>
```

```
<h2>What Can JavaScript Do?</h2>
```

```
<p id="demo">JavaScript can change HTML content.</p>
```

```
<button type="button" onclick='document.getElementById("demo").innerHTML  
= "Hello JavaScript!">Click Me!</button>
```

```
</body>
```

```
</html>
```

JavaScript Statements

- In a programming language, the list of programming instructions are called **statements**.
- A JavaScript program is a list of programming statements.
- JavaScript statements are composed of: Values, Operators, Expressions, Keywords, and Comments.

Comments

Single line comments start with `//`

Multi-line comments start with `/*` and end with `*/`

Variables

Variables are containers for storing data (storing data values).

In this example, `x`, `y`, and `z`, are variables, declared with the `var` keyword:

```
var x = 5; var y = 6;
```

Declaring a JavaScript Variable

- Using **var**
- Using **let**
- Using **const**

```
<html>
<body>
<h1>JavaScript Variables</h1>
<p>In this example, x, y, and z are variables.</p>
<p id="demo"></p>
<script>
var x=5;
var y = 6;
var z = x + y;
document.getElementById("demo").innerHTML ="The value of z is: " + z;
</script>
</body></html>
```

JavaScript Functions

JavaScript functions are the blocks of code used to perform some particular operations. JavaScript function is executed when something calls it. It calls many times so the function is reusable.

Syntax:

```
function functionName( par1, par2, ....., parn ) {

    // Function code

}
```

Example

```
1] <html>
<body>
<script>
function msg(){
alert("hello! this is message");
}
</script>
<input type="button" onclick="msg()" value="call function"/>
</body>
</html>
```

```
2] <html>
<body>
<script>
function getcube(number){
alert(number*number*number);
}
</script>
<form>
<input type="button" value="click" onclick="getcube(4)"/>
</form>
</body></html>
```

JAVASCRIPT OUTPUT COMPONENTS

1. innerHTML [getElementById()]

2. console.log()
3. document.write()
4. window.alert()
5. window.print()
6. window.prompt()

JavaScript Objects

Objects are variables too. But objects can contain many values.

The values are written as **name:value** pairs (name and value separated by a colon).

Practice:

```
<script>
```

```
// Create an object:
```

```
const person = {  
  firstName: "John",  
  lastName: "Doe",  
  age: 50,  
  eyeColor: "blue"  
};
```

```
// Display some data from the object:
```

```
document.getElementById("demo").innerHTML =person.firstName + " is " +  
person.age + " years old.";
```

```
or
```

```
document.getElementById("demo").innerHTML =person["firstName"] + " " +  
person["age"];
```

```
</script>
```

Object Methods

Objects can also have **methods**.

Methods are **actions** that can be performed on objects.

Methods are stored in properties as **function definitions**.

Property	Property Value
firstName	John
lastName	Doe
age	50
fullName	function() {return this.firstName + " " + this.lastName;}

<script>

```
const person = {
  firstName: "John",
  lastName : "Doe",
  id    : 5566,
  fullName : function() {
    return this.firstName + " " + this.lastName;
  }
};

document.getElementById("demo").innerHTML = person.fullName();
</script>
```

Constructors:

A **constructor** in Javascript is a special method that is used to initialize objects.

Practice:

<script>

// Constructor function for Person objects

```
function Person(first, last, age, eye) {  
  this.firstName = first;  
  this.lastName = last;  
  this.age = age;  
}
```

// Create a Person object

```
const myFather = new Person("John", "Doe", 50, "blue");
```

// Display age

```
document.getElementById("demo").innerHTML =  
"My father is " + myFather.age + ".";  
</script>
```

Object Properties

The **name:values** pairs in JavaScript objects are called **properties**

Prototype Properties

All JavaScript objects inherit properties and methods from a prototype.

The prototype property allows you to add new methods to objects constructors:

Practice:

```
<script>  
function Person(first, last, age, eye) {  
  this.firstName = first;  
  this.lastName = last;  
}  
  
Person.prototype.nationality = "English";  
  
const myFather = new Person("John", "Doe", 50, "blue");
```

```
document.getElementById("demo").innerHTML =  
"The nationality of my father is " + myFather.nationality;  
</script>
```

JSON

- JSON stands for JavaScript Object Notation
- JSON is a text format for storing and transporting data
- JSON is "self-describing" and easy to understand
- JSON is a lightweight data-interchange format
- JSON is plain text written in JavaScript object notation
- JSON is used to send data between computers.
- JSON is language independent

Why Use JSON?

The JSON format is syntactically similar to the code for creating JavaScript objects.

JavaScript program can easily convert JSON data into JavaScript objects.

JSON data can easily be sent between computers, and used by any programming language.

JavaScript has a built in function for converting JSON strings into JavaScript objects:

JSON.parse()

JavaScript also has a built in function for converting an object into a JSON string:

JSON.stringify()

JSON Syntax Rules

JSON syntax is derived from JavaScript object notation syntax:

- Data is in name/value (key/value) pairs: A name/value pair consists of a field name (in double quotes), followed by a colon, followed by a value
"name":"John"
- Data is separated by commas

- Curly braces hold objects
- Square brackets hold arrays

JSON Values

In JSON, *values* must be one of the following data types:

- a string
- a number
- an object
- an array
- a boolean
- null

JSON.parse()

A common use of JSON is to exchange data to/from a web server.

When receiving data from a web server, the data is always a string.

Parse the data with JSON.parse(), and the data becomes a JavaScript object.

Practice:

```
<html>
```

```
<body>
```

```
<h2>Creating an Object from a JSON String</h2>
```

```
<p id="demo"></p>
```

```
<script>
```

```
const txt = '{"name":"John", "age":30, "city":"New York"}'
```

```
const obj = JSON.parse(txt);
```

```
document.getElementById("demo").innerHTML = obj.name + ", " + obj.age;
```

```
</script>
```

```
</body>
```

```
</html>
```

JSON.stringify()

A common use of JSON is to exchange data to/from a web server.

When sending data to a web server, the data has to be a string.

Convert a JavaScript object into a string with JSON.stringify().

Practice:

```
<html>
```

```
<body>
```

```
<h2>Create a JSON string from a JavaScript object.</h2>
```

```
<p id="demo"></p>
```

```
<script>
```

```
const obj = {name: "John", age: 30, city: "New York"};
```

```
const myJSON = JSON.stringify(obj);
```

```
document.getElementById("demo").innerHTML = myJSON;
```

```
</script>
```

```
</body>
```

```
</html>
```

ES6

ECMAScript (**E**uropean **C**omputer **M**anufacturers **A**ssociation **S**cript) is a scripting language based on JavaScript

ECMAScript 2015 was the second major revision to JavaScript.

ECMAScript 2015 is also known as ES6 and ECMAScript 6.

1| Arrow functions

Arrow functions allows a short syntax for writing function expressions.

You don't need the **function** keyword, the **return** keyword, and the **curly brackets**.

Practice:

```
<script>
const x = (x, y) => x * y;
document.getElementById("demo").innerHTML = x(5, 5);
</script>
```

2| Template strings

Template Literals use back-ticks (`) rather than the quotes (") to define a string:

Practice:

```
<script>
let text = `Hello world!`;
document.getElementById("demo").innerHTML = text;
</script>
```

Interpolation

Template literals provide an easy way to interpolate variables and expressions into strings.

The method is called string interpolation.

The syntax is:

```
${...}
```

Template literals allow variables in strings:

Practice:

```
<script>
let firstName = "John";
let lastName = "Doe";
let text = `Welcome ${firstName}, ${lastName}!`;
document.getElementById("demo").innerHTML = text;
```

</script>

3| JavaScript Object Prototypes

All JavaScript objects inherit properties and methods from a prototype.

Prototype Inheritance

The Object.prototype is on the top of the prototype inheritance chain:

Date objects, Array objects, and Person objects inherit from Object.prototype.

All JavaScript objects inherit properties and methods from a prototype:

- Array objects inherit from Array.prototype
- Person objects inherit from Person.prototype

4| The Spread (...) Operator

The ... operator expands an iterable (like an Array) into more elements:

Practice:

```
<script>
const cars1 = ["Saab", "Volvo", ..."BMW"];
const cars2 = ["Fiat", "Toyota"];
const combined = [cars1, ...cars2];
document.getElementById("demo").innerHTML = combined;
</script>
```

5| Map

Creating a Map from an Array

A Map holds key-value pairs where the keys can be any datatype.

A Map remembers the original insertion order of the keys.

A Map has a property that represents the size of the map

How to Create a Map

You can create a JavaScript Map by:

- Passing an Array to new Map()
- Create a Map and use Map.set()

new Map()

You can create a Map by passing an Array to the new Map() constructor:

Practice:

```
<script>
// Create a Map
const fruits = new Map([
  ["apples", 500],
  ["bananas", 300],
  ["oranges", 200]
]);
document.getElementById("demo").innerHTML = fruits.get("apples");
</script>
```

Map.set()

You can add elements to a Map with the **set()** method:

Practice:

```
<script>
// Create a Map
const fruits = new Map();
// Set Map Values
fruits.set("apples", 500);
```

```
fruits.set("bananas", 300);  
fruits.set("oranges", 200);  
document.getElementById("demo").innerHTML = fruits.get("apples");  
</script>
```

Map Methods

Method	Description
new Map()	Creates a new Map object
set()	Sets the value for a key in a Map
get()	Gets the value for a key in a Map
clear()	Removes all the elements from a Map
delete()	Removes a Map element specified by a key
has()	Returns true if a key exists in a Map
forEach()	Invokes a callback for each key/value pair in a Map
entries()	Returns an iterator object with the [key, value] pairs in a Map
keys()	Returns an iterator object with the keys in a Map
values()	Returns an iterator object of the values in a Map
size	Returns the number of Map elements

6| JavaScript Set

A JavaScript Set is a collection of unique values.

Each value can only occur once in a Set.

A Set can hold any value of any data type.

How to Create a Set

You can create a JavaScript Set by:

- Passing an Array to **new Set()**
- Create a new Set and use **add()** to add values
- Create a new Set and use **add()** to add variables

The new Set() Method

Pass an Array to the **new Set()** constructor:

```
<script>
// Create a Set
const letters = new Set(["a","b","c"]);
// Display set.size
document.getElementById("demo").innerHTML = letters.size;
</script>
```

Create a Set and add literal values:

```
<script>
// Create a Set
const letters = new Set();
// Add Values to the Set
letters.add("a");
letters.add("b");
letters.add("c");
```

```
// Display set.size  
document.getElementById("demo").innerHTML = letters.size;  
</script>
```

Set Methods

Method	Description
new Set()	Creates a new Set
add()	Adds a new element to the Set
delete()	Removes an element from a Set
has()	Returns true if a value exists
clear()	Removes all elements from a Set
forEach()	Invokes a callback for each element
values()	Returns an Iterator with all the values in a Set
keys()	Same as values()
entries()	Returns an Iterator with the [value,value] pairs from a Set

TypeScript

- TypeScript is a syntactic superset of JavaScript which adds static typing.
- This basically means that TypeScript adds syntax on top of JavaScript, allowing developers to add types.
- TypeScript being a "Syntactic Superset" means that it shares the same base syntax as JavaScript, but adds something to it.
- TypeScript uses compile time type checking. Which means it checks if the specified types match before running the code, not while running the code.
- JavaScript is better suited for small-scale applications, while TypeScript is better for larger applications.
- TypeScript is better than JavaScript in terms of language features, reference validation, project scalability, collaboration within and between teams, developer experience, and code maintainability.

Setting up development environment for TypeScript

Pre-requisite to install TypeScript

1. Text Editor or IDE
2. Node.js Package Manager (npm)
3. The TypeScript compiler

Ways to install TypeScript

There are two ways to install TypeScript:

1. Install TypeScript using Node.js Package Manager (npm).
2. Install the TypeScript plug-in in your IDE (Integrated Development Environment).

Install TypeScript using Node.js Package Manager (npm)

Step-1:

Install Node.js. It is used to setup TypeScript on our local computer.

To install Node.js on Windows, go to the following

link: <https://www.javatpoint.com/install-node.js>

To verify the installation was successful, enter the following command in the Terminal Window.

node -v

npm -v

Step-2:

Install TypeScript. To install TypeScript, enter the following command in the Terminal Window.

- `npm install typescript --save-dev` //As dev dependency
- `npm install typescript -g` //Install as a global module [or]
- `npm install -g typescript`
- `npm install typescript@latest -g` //Install latest if you have an older version

Step-3:

To verify the installation was successful, enter the command `$ tsc -v` in the Terminal Window.

Install Live server

```
npm install -g live-server
```

Create and run first program in TypeScript

- open command prompt
- go to d: drive(any drive)
- `d:\>mkdir typescript`
- `d:\>cd typescript`
- `d:\typescript> npm install typescript --save-dev`
- open visual studio code
- file-open folder-choose typescript folder from d:
- create new file- save it as types.ts(any name.ts)
- Write the below code and save it
- `console.log("Hello World");`
- go to command prompt and compile the program
- `tsc types.ts`
- run the program

- node types.js
- Observe the output

Basic Types

Built-in Data Type	keyword	Description
Number	number	It is used to represent both Integer as well as Floating-Point numbers
Boolean	boolean	Represents true and false
String	string	It is used to represent a sequence of characters
Void	void	Generally used on function return-types
Null	null	It is used when an object does not have any value
Undefined	undefined	Denotes value given to uninitialized variable
Any	any	If variable is declared with any data-type then any type of value can be assigned to that variable

Control flow statement

1] TypeScript If Statement:

If statement is used to execute a block of statements if specified condition is true.

```
if(condition){
    //Block of TypeScript statements.
}
```

2] TypeScript If Else Statement:

If else statement is used to execute either of two block of statements depends upon the condition. If condition is true then if block will execute otherwise else block will execute.

```
if(condition){  
  //Block of TypeScript statements1.  
}  
else{  
  //Block of TypeScript statements2.  
}
```

3] TypeScript **If Else If** Statement:

If else statement is used to execute one block of statements from many depends upon the condition. If condition1 is true then block of statements1 will be executed, else if condition2 is true block of statements2 is executed and so on. If no condition is true, then else block of statements will be executed.

```
if(condition1){  
  //Block of TypeScript statements1.  
}  
else if(condition2){  
  //Block of TypeScript statements2.  
}  
... else if(conditionn){  
  //Block of TypeScript statementsn.  
}  
else{  
  //Block of TypeScript statements.  
}
```

Example

```
var num:number = 2;  
if(num==1){  
  console.log("TypeScript Statement 1");  
}  
else if(num==2){
```

```
console.log("TypeScript Statement 2");  
}  
else if(num==3){  
console.log("TypeScript Statement 3");  
}  
else{  
console.log("TypeScript Statement n");  
}
```

TypeScript - Loops

1] for Loop

The for loop is used to execute a block of code a given number of times, which is specified by a condition.

```
for (first expression; second expression; third expression) {  
    // statements to be executed repeatedly  
}
```

```
for (let i = 0; i < 3; i++) {  
    console.log ("Block statement execution no." + i);  
}
```

2] for...of Loop

TypeScript includes the for...of loop to iterate and access elements of an array, list, or tuple collection.

```
let arr = [10, 20, 30, 40];  
for (var val of arr) {  
    console.log(val); // prints values: 10, 20, 30, 40  
}
```

3] for...in Loop

Another form of the for loop is for...in. This can be used with an array, list, or tuple.

```
let arr = [10, 20, 30, 40];  
for (var index in arr) {  
    console.log(index); // prints indexes: 0, 1, 2, 3  
    console.log(arr[index]); // prints elements: 10, 20, 30, 40  
}
```

4] while Loop

The while loop is another type of loop that checks for a specified condition before beginning to execute the block of statements. The loop runs until the condition value is met.

```
while (condition expression) {  
    // code block to be executed  
}  
  
let i: number = 2;  
while (i < 4) {  
    console.log( "Block statement execution no." + i )  
    i++;  
}
```

5] do..while loop

The do..while loop is similar to the while loop, except that the condition is given at the end of the loop. The do..while loop runs the block of code at least once before checking for the specified condition.

```
do {  
    // code block to be executed  
}  
while (condition expression);  
let i: number = 2;
```

```
do {  
    console.log("Block statement execution no." + i )  
    i++;  
} while ( i < 4)
```

Functions

In TypeScript, functions can be of two types: named and anonymous.

1] Named Functions

A named function is one where you declare and call a function by its given name.

```
function display() {  
    console.log("Hello TypeScript!");  
}  
display(); //Output: Hello TypeScript
```

Functions can also include parameter types and return type.

```
function Sum(x: number, y: number) : number {  
    return x + y;  
}  
Sum(2,3); // returns 5
```

2] Anonymous Function

- An anonymous function is one which is defined as an expression.
- This expression is stored in a variable.
- So, the function itself does not have a name.
- These functions are invoked using the variable name that the function is stored in.

```
let display = function() {  
    console.log("Hello TypeScript!");  
};  
display (); //Output: Hello TypeScript!
```

UNIT 5

INDEX

- 1] Introduction to React**
- 2] Introduction to JSX**
- 3] Components**
- 4] Props & State**
- 5] Handling Events**
- 6] Conditional Rendering**
- 7] Lists & Keys**
- 8] controlled components**
- 9] uncontrolled components**
- 10] Lifting State Up**

1] Introduction to React

React is an open source frontend JavaScript library for building complex UIs; it allows the programmers to create complex UIs from smaller components.

React is a declarative, efficient, and flexible JavaScript library for building user interfaces.

Setting up React development

Step 1: Install NodeJS.

Download Node.js and npm from <https://nodejs.org>.

Test that Node.js and npm were installed successfully by running the commands

```
node -v
```

```
npm -v.
```

Step 2:

```
npx create-react-app myapp
```

```
cd myapp
```

```
npm start
```

What is Node.js?

- Node.js is an open source server environment (free)
- Node.js runs on various platforms (Windows, Linux, Unix, Mac OS X, etc.)
- Node.js uses JavaScript on the server

What Can Node.js Do?

- Node.js can generate dynamic page content
- Node.js can create, open, read, write, delete, and close files on the server
- Node.js can collect form data
- Node.js can add, delete, modify data in your database

2] What is JSX?

- JSX stands for JavaScript XML.
- JSX allows us to write HTML in React.
- JSX makes it easier to write and add HTML in React.
- Consider this variable declaration:
- `const element = <h1>Hello, world! </h1>;`
- JSX is a XML syntax extension. Just like HTML, JSX tags can have a tag names, attributes, and children. If an attribute is wrapped in curly braces, the value is a JavaScript expression.

```
import React from 'react';
import ReactDOM from 'react-dom/client';

const myFirstElement = <h1>Hello React!</h1>

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(myFirstElement);
```

Expressions in JSX

With JSX you can write expressions inside curly braces { }

The expression can be a React variable, or property, or any other valid JavaScript expression. JSX will execute the expression and return the result

Ex1:

```
import React from 'react';
import ReactDOM from 'react-dom/client';

const name='niranjan';
const element=<h1>Hello, { name }</h1>;
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(element);
```

Ex2:

```
import React from 'react';
import ReactDOM from 'react-dom/client';

const myElement = <h1>React is {5 + 5} times better with JSX</h1>;
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(myElement);
```

Specifying Attributes with JSX

JSX allows us to use attributes with the HTML elements.

JSX uses camelcase convention for attributes.

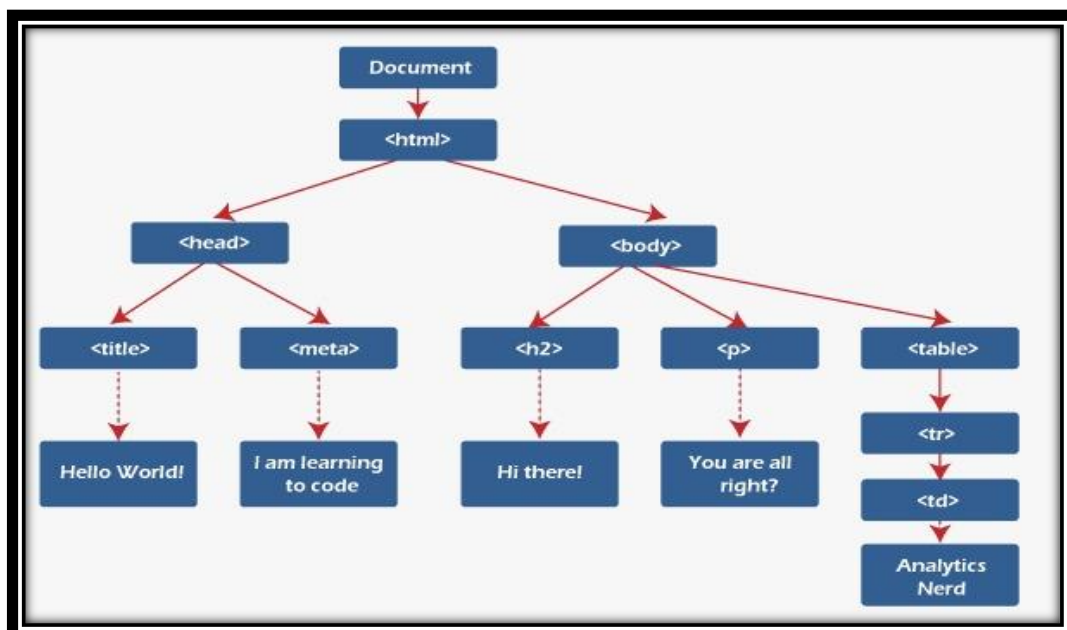
```
import React from 'react';
import ReactDOM from 'react-dom/client';
const myElement = <h1 className="myclass">Hello World</h1>;
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(myElement);
```

What is the DOM?

The DOM (Document Object Model) represents the web page as a tree structure.

Any piece of HTML that we write is added as a node, to this tree.

With JavaScript, we can access any of these nodes (HTML elements) and update their styles, attributes, and so on.



React-DOM

ReactDOM is a package that provides DOM specific methods that can be used at the top level of a web app to enable an efficient way of managing DOM elements of the web page.

Virtual DOM in React

Virtual DOM in React is a “virtual” representation of the actual DOM. It is nothing but an object created to replicate the actual DOM.

3] What are React Components?

- React components are independent and reusable code.
- They are the building blocks of any React application.
- Components serve the same purpose as JavaScript functions, but work individually to return JSX code as elements for our UI.
- Components usually come in two types, functional components and class components

Functional Components

```
import React from 'react';
import ReactDOM from 'react-dom/client';

function Car() {
  return <h2>Hi, I am a Car!</h2>;
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Car />);
```

Class Components

A class component must include the extends React.Component statement.

The component also requires a render() method, this method returns HTML.

```
import React from 'react';
import ReactDOM from 'react-dom/client';

class Car extends React.Component {
  render() {
    return <h2>Hi, I am a Car!</h2>;
  }
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Car />);
```

Rendering a Component

```
import React from 'react';
import ReactDOM from 'react-dom/client';

function Car() {
  return <h2>Hi, I am a Car!</h2>;
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Car />);
```

Composing Components

```
function Welcome(props) {
  return <h1>Hello, {props.name}</h1>;
}

function App() {
  return (
    <div>
```

```

    <Welcome name="Sara" />
    <Welcome name="Cahal" />
    <Welcome name="Edite" />
  </div> );
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);

```

Create your first React Component.

When creating a React component, the component's name *MUST* start with an upper case letter.

```

import React from 'react';
import ReactDOM from 'react-dom/client';

function Car() {
  return <h2>Hi, I am a Car!</h2>;
}const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Car />);

```

4] What are props?

Props is short for properties and they are used to pass data between React components. React's data flow between components is uni-directional (from parent to child only).

```

import React from 'react';
import ReactDOM from 'react-dom/client';

class ParentComponent extends React.Component {

```

```

render() {
  return (
    <ChildComponent name="First Child" />
  );
}
}

const ChildComponent = (props) => {
  return <p>{props.name}</p>;
};

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<ParentComponent />);

```

What is state?

React has another special built-in object called state, which allows components to create and manage their own data. So unlike props, components cannot pass data with state, but they can create and manage it internally.

```

import React from 'react';
import ReactDOM from 'react-dom/client';

class Test extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      id: 1,
      name: "test"
    };
  }

  render() {
    return (
      <div>
        <p>{this.state.id}</p>

```

```

        <p>{this.state.name}</p>
      </div>
    );
  }
}
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Test />);

```

Communication between components using Props

You can pass props down to child components.

```

import React from 'react';
class Parent extends React.Component {
  render() {
    const variable = 5;
    return (
      <div>
        <Child message="message for child" />
        <Child message={variable} />
      </div>
    );
  }
}
class Child extends React.Component {
  render() {
    return <h1>{this.props.message}</h1>
  }
}
export default Parent;

```

What is the React component lifecycle?

In React, components go through a lifecycle of events:

1. Mounting (adding nodes to the DOM)
2. Updating (altering existing nodes in the DOM)
3. Unmounting (removing nodes from the DOM)
4. Error handling (verifying that your code works and is bug-free)

1] Mounting

Mounting means putting elements into the DOM.

React has four built-in methods that gets called, in this order, when mounting a component:

1. constructor()
2. getDerivedStateFromProps()
3. render()
4. componentDidMount()

The render() method is required and will always be called, the others are optional and will be called if you define them.

2] Updating

A component is updated whenever there is a change in the component's state or props.

React has five built-in methods that gets called, in this order, when a component is updated:

1. getDerivedStateFromProps()
2. shouldComponentUpdate()
3. render()
4. getSnapshotBeforeUpdate()
5. componentDidUpdate()

3] Unmounting

The next phase in the lifecycle is when a component is removed from the DOM, or unmounting as React likes to call it.

React has only one built-in method that gets called when a component is unmounted:

- componentWillUnmount()

4] Error Handling

Error Handling methods are called when there is an error during rendering, in a lifecycle method, or in the constructor of any child component.

- `static getDerivedStateFromError()`
- `componentDidCatch()`

```
import React from 'react';
import ReactDOM from 'react-dom/client';

import { Component } from "react";

class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }

  static getDerivedStateFromError(error) {
    // Update state so the next render will show the fallback UI.
    return { hasError: true };
  }

  componentDidCatch(error, errorInfo) {
    // You can also log the error to an error reporting service
    logErrorToMyService(error, errorInfo);
  }

  render() {
    if (this.state.hasError) {
      // You can render any custom fallback UI
      return <h1>Something went wrong.</h1>;
    }

    return this.props.children;
  }
}

export default ErrorBoundary;
```

5] Handling Events

React events are written in camelCase syntax:

`onClick` instead of `onclick`.

React event handlers are written inside curly braces:

`onClick={shoot}` instead of `onClick="shoot()"`.

```
import React from 'react';
import ReactDOM from 'react-dom/client';

function Football() {
  const shoot = () => {
    alert("Great Shot!");
  }
  return (
    <button onClick={shoot}>Take the shot!</button>
  );
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Football />);
```

6] Conditional Rendering

`if` Statement

We can use the `if` JavaScript operator to decide which component to render.

```
import React from 'react';
import ReactDOM from 'react-dom/client';

function UserGreeting(props) {
  return <h1>Welcome back!</h1>;
}
```

```

}

function GuestGreeting(props) {
  return <h1>Please sign up.</h1>;
}

function Greeting(props) {
  const isLoggedIn = props.isLoggedIn;
  if (isLoggedIn) {
    return <UserGreeting />;
  }

  return <GuestGreeting />;
}

const root = ReactDOM.createRoot(document.getElementById('root'));
// Try changing to isLoggedIn={true}:
root.render(<Greeting isLoggedIn={false} />);

```

7] Lists

You can build collections of elements and include them in JSX using curly braces {}.

- The list is used to display data in an ordered format and is traversed using the map() function.
- lists are a continuous group of text or images.

```

import React from 'react';
import ReactDOM from 'react-dom/client';

function NumberList(props) {

```

```

const numbers = props.numbers;
const listItems = numbers.map((number) =><li>{number}</li>);
return (
  <ul>{listItems}</ul>
);
}
const numbers = [1, 2, 3, 4, 5];
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<NumberList numbers={numbers} />);

```

Keys

- Keys are used to identify which items have changed, are added, or are removed.
- Keys should be given inside the array to give the elements a stable identity.

```

import React from 'react';
import ReactDOM from 'react-dom';
function App(props) {
  const numbers = props.numbers;
  return (
    <div>
      <ul>
        {numbers.map((number) =>
          <li key={number.toString()}>{number}</li>
        )}
      </ul>
    </div>
  );
}

```

```
</ul>
</div>
);
}
const numbers = ['Indore', 'Mumbai', 'Pune', 'Hyderabad', 'Banglore'];
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App numbers={numbers} />);
export default App;
```

8] Controlled components

- In HTML, form elements such as <input>, <textarea>, and <select> typically maintain their own state and update it based on user input.
- In React, mutable state is typically kept in the state property of components, and only updated with setState().

9] Uncontrolled components

- In a controlled component, form data is handled by a React component. The alternative is uncontrolled components, where form data is handled by the DOM itself.
- To write an uncontrolled component, instead of writing an event handler for every state update, you can use a ref to get form values from the DOM.

10] Lifting State Up

In React, it is recommended to make a common pattern while using the same state in multiple components. “Lifting state up” is a common pattern used in React to share state

among multiple components to avoid complex and unnecessary patterns for state management.

Testing single page application

Filename: App.css

```
.App {  
  text-align: center;  
  background-color: green;  
}  
.input {  
  font-size: larger;  
  padding: 5px;  
  margin: 2px;  
}  
.btn {  
  color: white;  
  display: block;  
  margin: 10px auto;  
}  
.messages {  
  display: flex;  
  justify-content: center;  
}
```

Filename: App.js

```
import './App.css';  
import Form from './Form'  
function App() {  
  return (  
    <div className="App">  
      <Form />  
    </div>  
  );  
}  
export default App;
```

UNIT 6

Index

1] React Router

2] React Hooks

3] Recap

1| React Router

- React Router is a **standard library for routing in React**.
- It enables the navigation among views of various components in a React Application, allows changing the browser URL, and keeps the UI in sync with the URL
- **Route params** are parameters whose values are set dynamically in a page's URL.
- This allows a route to render the same component while passing that component the dynamic portion of the URL, so that it can change its data based on the parameter.
- Examples include things like IDs for products, books, movies, users etc

```
<Route exact path="/movie/:id" component={MovieDetailsContainer} />
```

React router key components

- routers, like **<BrowserRouter>** and **<HashRouter>**
 - <BrowserRouter>**: It is used for handling the dynamic URL.
 - <HashRouter>**: It is used for handling the static request.

```
function App() {  
  return <h1>Hello React Router</h1>;  
}
```

```
ReactDOM.render(  
  <BrowserRouter>  
    <App />  
  </BrowserRouter>,  
  document.getElementById("root")  
);
```

Add React Router

- To add React Router in your application, run this in the terminal from the root directory of the application:

```
Command: npm i -D react-router-dom
```

Create a folder name called pages. Within a pages create following files.

Index.js

```
import ReactDOM from "react-dom/client";
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Layout from "../pages/Layout";
import Home from "../pages/Home";
import Blogs from "../pages/Blogs";
import Contact from "../pages/Contact";
import NoPage from "../pages/NoPage";
```

```
export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Layout />} />
        <Route index element={<Home />} />
        <Route path="blogs" element={<Blogs />} />
        <Route path="contact" element={<Contact />} />
        <Route path="*" element={<NoPage />} />
      </Routes>
    </BrowserRouter>
  );
}
```

```
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

Blogs.js

```
const Blogs = () => {
  return <h1>Blog Articles</h1>;
};

export default Blogs;
```

Contact.js

```
const Contact = () => {
  return <h1>Contact Me</h1>;
};
```

```
export default Contact;
```

Home.js

```
const Home = () => {  
  return <h1>Home</h1>;  
};  
export default Home;
```

Layout.js

```
import { Outlet, Link } from "react-router-dom";
```

```
const Layout = () => {  
  return (  
    <>  
      <nav>  
        <ul>  
          <li>  
            <Link to="/">Home</Link>  
          </li>  
          <li>  
            <Link to="/blogs">Blogs</Link>  
          </li>  
          <li>  
            <Link to="/contact">Contact</Link>  
          </li>  
        </ul>  
      </nav>  
  
      <Outlet />  
    </>  
  )  
};  
  
export default Layout;
```

NoPage.js

```
const NoPage = () => {  
  return <h1>404</h1>;  
};
```

```
};  
export default NoPage;
```

a) Context

- Context provides a way to pass data through the component tree without having to pass props down manually at every level.
- In a typical React application, data is passed top-down (parent to child) via props.
- Context provides a way to share values between components without having to explicitly pass a prop through every level of the tree.

```
import React from 'react';  
import ReactDOM from 'react-dom/client';  
import { createContext, useContext } from "react";  
  
const Context = createContext("abc");  
  
export function Main() {  
  const value = "My Context Value";  
  
  return (  
    <Context.Provider value={value}>  
      <MyComponent />  
    </Context.Provider>  
  );  
}  
  
function MyComponent() {  
  const value = useContext(Context);  
  
  return <span>{value}</span>;  
}  
  
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(<MyComponent/>);
```

b) React Fragments

- Allow you to wrap or group multiple elements without adding an extra node to the DOM.
- This can be useful when rendering multiple child elements/components in a single parent component.

```
import React from 'react';
import ReactDOM from 'react-dom/client';
function Column() {
  return (
    <React.Fragment>
      <h1>JSS</h1>
      <h2>Polytechnic</h2>
    </React.Fragment>
  );
}
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Column/>);
```

c) React Higher-Order Component

- The React Higher-Order Components is an advanced technique that takes a component and returns a new component. It is used for reusing component logic.

```
function sub (x, y) {
  return x - y }
function higherOrderComp(x, subReference) {
  return subReference(x, 10) }
higherOrderComp(50, sub)
```

2] React Hooks

- Hooks (Receive or Steal) are the new feature introduced in the React 16.8 version. It allows you to use state and other React features without writing a class.

1]useState, 2]useEffect 3] useContext 4] useReducer

A] useState

- The React useState Hook allows us to track state in a function component.
- State generally refers to data or properties that need to be tracking in an application.
- To use the useState Hook, we first need to import it into our component.

Syntax: `import { useState } from "react";`

- We initialize our state by calling useState in our function component.
- useState accepts an initial state and returns two values:
 - i. The current state.
 - ii. A function that updates the state.

Initialization

```
import { useState } from "react";
function FavoriteColor() {
  const [color, setColor] = useState("");
}
```

Example for update and Read state

```
import { useState } from "react";
import ReactDOM from "react-dom/client";
function FavoriteColor() {
  const [color, setColor] = useState("red");
  return (
    <>
      <h1>My favorite color is {color}!</h1>
      <button
        type="button"
        onClick={() => setColor("blue")}
      >Blue</button>
    </>
  )
}
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<FavoriteColor />);
```

B) useEffect:

- The `useEffect` Hook allows you to perform side effects in your components.
- Some examples of side effects are: fetching data, directly updating the DOM, and timers.
- `useEffect` accepts two arguments. The second argument is optional.

Syntax: `useEffect(<function>, <dependency>)`

Example to set timer:

```
import { useState, useEffect } from "react";
import ReactDOM from "react-dom/client";

function Timer() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    setTimeout(() => {
      setCount((count) => count + 1);
    }, 1000);
  });

  return <h1>I have rendered {count} times!</h1>;
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Timer />);
```

Example to clear timer:

```
import { useState, useEffect } from "react";
import ReactDOM from "react-dom/client";

function Timer() {
  const [count, setCount] = useState(0);
  useEffect(() => {
    let timer = setTimeout(() => {
      setCount((count) => count + 1);
    }, 1000);
  });
}
```

```

    return () => clearTimeout(timer)
  }, []);
  return <h1>I've rendered {count} times!</h1>;
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Timer />);

```

C] useContext

- React Context is a way to manage state globally.
- It can be used together with the `useState` Hook to share state between deeply nested components more easily than with `useState` alone.

Create Context: You must Import `createContext` and initialize it:

```

import { useState, createContext } from "react";
import ReactDOM from "react-dom/client";
const UserContext = createContext()

```

Context Provider: Wrap child components in the Context Provider and supply the state value.

To use the Context in a child component, we need to access it using the `useContext` Hook.

```

import { useState, createContext, useContext } from "react";
import ReactDOM from "react-dom/client";
const UserContext = createContext();

```

```

function Component1() {
  const [user, setUser] = useState("Jesse Hall");
  return (
    <UserContext.Provider value={user}>
      <h1>`Hello ${user}!`</h1>
      <Component2 />
    </UserContext.Provider>
  );
}

```

```
function Component2() {
  return (
    <>
      <h1>Component 2</h1>
      <Component3 />
    </>
  );
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Component1 />);
```

D] useReducer

- The useReducer Hook is similar to the useState Hook.
- It allows for custom state logic.
- If you find yourself keeping track of multiple pieces of state.

Syntax: useReducer(<reducer>, <initialState>)

- The reducer function contains your custom state logic and the initialState can be a simple value but generally will contain an object.

Example:

```
import React, { useReducer } from "react";

// Defining the initial state and the reducer
const initialState = 0;
const reducer = (state, action) => {
  switch (action) {
    case "add":
      return state + 1;
    case "subtract":
      return state - 1;
    case "reset":
      return 0;
    default:
      throw new Error("Unexpected action");
  }
};
```

```

const App = () => {
  // Initialising useReducer hook
  const [count, dispatch] = useReducer(reducer, initialState);
  return (
    <div>
      <h2>{count}</h2>
      <button onClick={() => dispatch("add")}>
        add
      </button>
      <button onClick={() => dispatch("subtract")}>
        subtract
      </button>
      <button onClick={() => dispatch("reset")}>
        reset
      </button>
    </div>
  );
};

```

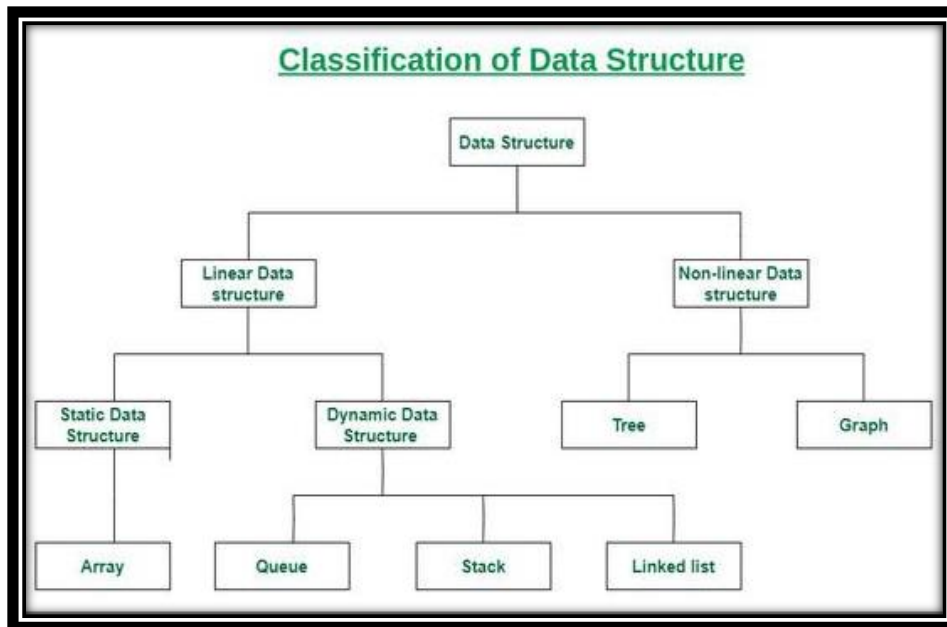
3| Recap

Object oriented concepts and design principles

1. DRY (Don't repeat yourself) - avoids duplication in code.
2. Encapsulate what changes - hides implementation detail, helps in maintenance
3. Open Closed design principle - open for extension, closed for modification
4. SRP (Single Responsibility Principle) - one class should do one thing and do it well
5. DIP (Dependency Inversion Principle) - don't ask, let framework give to you
6. Favor Composition over Inheritance - Code reuse without cost of inflexibility
7. LSP (Liskov Substitution Principle) - Sub type must be substitutable for super type
8. ISP (Interface Segregation Principle) - Avoid monolithic interface, reduce pain on client side
9. Programming for Interface - Helps in maintenance, improves flexibility
10. Delegation principle - Don't do all things by yourself, delegate it

Data Structure:

A data structure is **a specialized format for organizing, processing, retrieving and storing data.**



Database Concepts:

A database is an organized collection of structured information, or data, typically stored electronically in a computer system. A database is usually controlled by a database management system (DBMS).

- **Database Schema:** It is a design of the database.
- **Data Constraints:** Constraints are used to put some restriction on the table.
- **Data dictionary or Metadata:** Metadata is known as the data about the data
- **Database instance:** used to define the complete database environment and its components.
- **Query:** query is used to access data from the database.
- **Data manipulation:** manipulation can be done using three main operations that is Insertion, Deletion, and updation.
- **Data Engine:** used to create and manage various database queries.

Java and servlet basics

- Java is a popular programming language.
- Java is used to develop mobile apps, web apps, desktop apps, games and much more.

Servlets are the Java programs that run on the Java-enabled web server or application server. They are used to handle the request obtained from the webserver, process the request, produce the response, then send a response back to the webserver.

Java Collections (List, Set, Map)

A **List** in java extends the collection interface and represent an sequenced or ordered group of elements. It can contain duplicate elements.

It also defines some additional methods which it inherits from Collection interface.

```
import java.util.*;
public class Main {
    public static void main(String args[]){
        List<String> mySubjects = new ArrayList<>();
        mySubjects.add("Java");
        mySubjects.add("Spring");
        System.out.println("My Subjects:");
        for(String subject : mySubjects){
            System.out.println(subject);
        }
    }
}
```

Set

- A set represents a group or collection of items. Set has a special property that is unique items, it can not contain a duplicate item or element. It extends the collection interface.

```
import java.util.*;
public class Main {
    public static void main(String args[]){
        Set<String> mySubjects = new HashSet<>();
        mySubjects.add("Java");
        mySubjects.add("Spring");
        System.out.println("My Subjects:");
        for(String subject : mySubjects){
            System.out.println(subject);
        }
    }
}
```

```
}
```

Map

- It represents a group of special elements or objects. Every map element or object contains key and value pair. A map can't contain duplicate keys and one key can refer to at most one value.

```
import java.util.*;
public class Main {

    public static void main(String args[]){
        Map<Integer,String> mysubjects = new HashMap<Integer,String>();

        mysubjects.put(1,"Java");
        mysubjects.put(2,"Spring");
        for(Map.Entry subject : mysubjects.entrySet())
            System.out.println(subject.getKey()+" - "+subject.getValue());
    }
}
```

Xml introduction

- XML stands for eXtensible Markup Language
- XML is a markup language much like HTML
- XML was designed to store and transport data
- XML was designed to be self-descriptive

The Difference Between XML and HTML

XML and HTML were designed with different goals:

- XML was designed to carry data - with focus on what data is
- HTML was designed to display data - with focus on how data looks
- XML tags are not predefined like HTML tags are

XML Simplifies Things

- XML simplifies data sharing
- XML simplifies data transport
- XML simplifies platform changes
- XML simplifies data availability

Unit 7

Index

- 1] Apache Maven**
- 2] Spring Framework**
- 3] Inversion of control and Dependency injection**

Apache Maven

- Apache Maven is a software project management and comprehension tool.
- Based on the concept of a project object model (POM), Maven can manage a project's build, reporting and documentation from a central piece of information.
- Using maven we can build and manage any Java based project.

Project management tool

- Apache Maven is a build tool.
- This is a software project management tool that gives a new concept of the project object model (POM).
- Maven allows the developer to automate the handling of the creation of the original folder format, performing the assortment and testing and the packaging and deployment of the final output.
- It cuts down the considerable number of steps in the base process, and it makes it just one-step process to do a build.

pom.xml

- POM stands for Project Object Model. It is fundamental unit of work in Maven. It is an XML file that resides in the base directory of the project as pom.xml.
- The POM contains information about the project and various configuration detail used by Maven to build the project(s).
- POM also contains the goals and plugins. While executing a task or goal, Maven looks for the POM in the current directory. It reads the POM, gets the needed configuration information, and then executes the goal.

It should be noted that there should be a single POM file for each project.

- All POM files require the project element and three mandatory fields: groupId, artifactId,

groupId

This is an Id of project's group. This is generally unique amongst an organization or a project. For example, a banking group com.company.bank has all bank related projects.

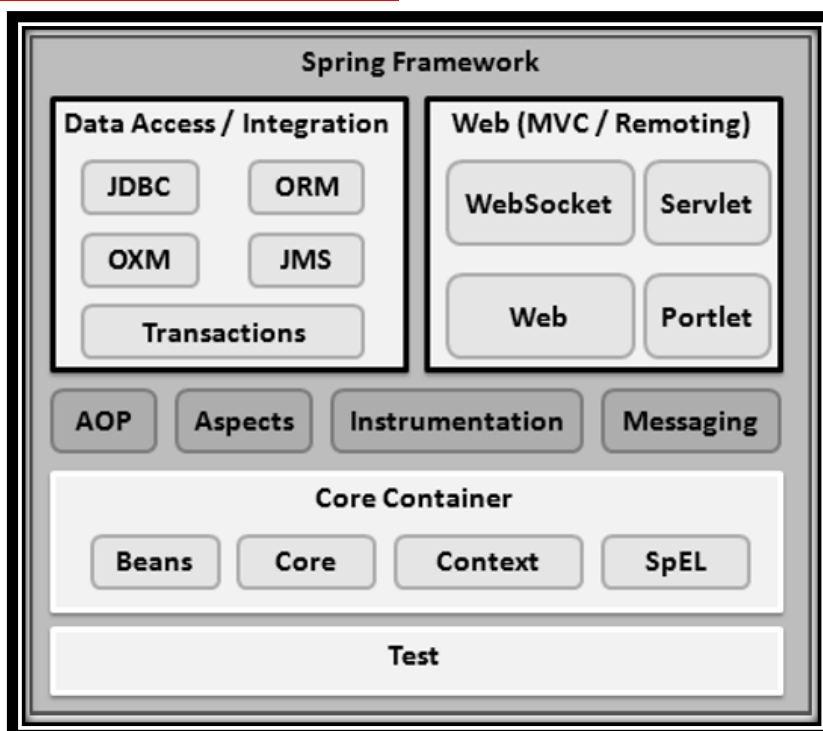
artifactId

This is an Id of the project. This is generally name of the project. For example, consumer-banking. Along with the groupId, the artifactId defines the artifact's location within the repository

What is Spring Framework?

- The Spring Framework is an open-source application framework that provides infrastructure support for developing Java applications.
- One of the most popular Java Enterprise Edition (Java EE) frameworks, Spring helps developers create high performing applications using plain old Java objects (POJOs).
- Spring is a *lightweight* framework. It can be thought of as a *framework of frameworks* because it provides support to various frameworks such as Struts, Hibernate, Tapestry, EJB, JSF, etc.

Spring Framework Architecture



Data Access/Integration

The Data Access/Integration layer consists of the JDBC, ORM, OXM, JMS and Transaction modules

- The **JDBC module** provides a JDBC-abstraction layer that removes the need for tedious JDBC related coding.
- The **ORM module** provides integration layers for popular object-relational mapping APIs, including JPA, JDO, Hibernate, and iBatis.
- The **OXM module** provides an abstraction layer that supports Object/XML mapping implementations for JAXB, Castor, XMLBeans, JiBX and XStream.
- The Java Messaging Service **JMS module** contains features for producing and consuming messages.
- The **Transaction module** supports programmatic and declarative transaction management for classes that implement special interfaces and for all your POJOs.

Web

The Web layer consists of the Web, Web-MVC, Web-Socket, and Web-Portlet modules

- The **Web module** provides basic web-oriented integration features such as multipart file-upload functionality and the initialization of the IoC container using servlet listeners and a web-oriented application context.
- The **Web-MVC module** contains Spring's Model-View-Controller (MVC) implementation for web applications.
- The **Web-Socket module** provides support for WebSocket-based, two-way communication between the client and the server in web applications.
- The **Web-Portlet module** provides the MVC implementation to be used in a portlet environment and mirrors the functionality of Web-Servlet module.

Core Container

The Core Container consists of the Core, Beans, Context, and Expression Language modules

- The **Core module** provides the fundamental parts of the framework, including the IoC and Dependency Injection features.

- The **Bean module** provides BeanFactory, which is a sophisticated implementation of the factory pattern.
- The **Context module** builds on the solid base provided by the Core and Beans modules and it is a medium to access any objects defined and configured. The ApplicationContext interface is the focal point of the Context module.
- The **SpEL module** provides a powerful expression language for querying and manipulating an object graph at runtime.

SpringBoot

- Spring Boot is an open source Java-based framework used to create a micro Service.
- Spring Boot provides a good platform for Java developers to develop a stand-alone and production-grade spring application that you can **just run**. You can get started with minimum configurations without the need for an entire Spring configuration setup

Advantages

Spring Boot offers the following advantages to its developers –

- Easy to understand and develop spring applications
- Increases productivity
- Reduces the development time

Why SpringBoot?

- It provides a flexible way to configure Java Beans, XML configurations, and Database Transactions.
- It provides a powerful batch processing and manages REST endpoints.
- In Spring Boot, everything is auto configured; no manual configurations are needed.
- It offers annotation-based spring application
- Eases dependency management
- It includes Embedded Servlet Container

Spring Annotations

Annotations are used to provide supplemental information about a program.

- Annotations start with '@'.
- Annotations help to associate *metadata* (information) to the program elements i.e. instance variables, constructors, methods, classes, etc.
- Annotations basically are used to provide additional information, so could be an alternative to XML and Java marker interfaces.

Spring Annotations List

@Service

Annotate all your service classes with @Service. All your business logic should be in Service classes.

@Service

```
public class CompanyServiceImpl implements CompanyService {  
...  
}
```

@Repository

Annotate all your DAO classes with @Repository. All your database access logic should be in DAO classes.

@Repository

```
public class CompanyDAOImpl implements CompanyDAO {  
...  
}
```

@Component

Annotate your other components (for example REST resource classes) with @Component

@Component

```
public class ContactResource {  
...  
}
```

@Autowired

Let Spring auto-wire other beans into your classes using @Autowired annotation.

@Service

```
public class CompanyServiceImpl implements CompanyService {
```

```
    @Autowired
```

```
    private CompanyDAO companyDAO;
```

```
    ...
```

```
}
```

@Transactional

Configure your transactions with @Transactional spring annotation.

@Service

```
public class CompanyServiceImpl implements CompanyService {
```

```
    @Autowired
```

```
    private CompanyDAO companyDAO;
```

```
    @Transactional
```

```
    public Company findByName(String name) {
```

```
        Company company = companyDAO.findByName(name);
```

```
        return company;
```

```
    }
```

```
    ...
```

```
}
```

@Scope

To change default behavior, use @Scope spring annotation.

@Component

```
@Scope("request")
```

```
public class ContactResource {
```

```
    ...
```

```
}
```

What is inversion of control?

- Spring IoC (Inversion of Control) Container is the core of Spring Framework. It creates the objects, configures and assembles their dependencies, manages their entire life cycle.
- Inversion of Control (IoC) is a design principle that allows classes to be loosely coupled and, therefore, easier to test and maintain.
- IoC refers to transferring the control of objects and their dependencies from the main program to a container or framework.

What is dependency injection?

- Dependency Injection is the main functionality provided by Spring IOC(Inversion of Control).
- The Spring-Core module is responsible for injecting dependencies through either Constructor or Setter methods.

Types of Spring Dependency Injection.

1. **Constructor Injection**: In the constructor injection, the injector supplies the service (dependency) through the client class constructor.
2. **Property Injection**: In the property injection (aka the Setter Injection), the injector supplies the dependency through a public property of the client class.
3. **Method Injection**: In this type of injection, the client class implements an interface which declares the method(s) to supply the dependency and the injector uses this interface to supply the dependency to the client class.

DI in spring Boot

Dependency Injection is the ability of an object to supply dependencies of another object.

It mainly has three classes involved:

- Client Class: This is the dependent class and is dependent on the Service class.
- Service Class: This class provides a service to the client class.

- **Injector Class:** This class is responsible for injecting the service class object into the client class

There are mainly three types of Dependency Injection:

- **Constructor Injection:** In this type of injection, the injector supplies dependency through the client class constructor.
- **Setter Injection / Property Injection:** In this type of injection, the injector method injects the dependency to the setter method exposed by the client.
- **Interface Injection:** In this type of injection, the injector uses Interface to provide dependency to the client class. The clients must implement an interface that will expose a setter method which accepts the dependency.

Autowiring

Autowiring feature of spring framework enables you to inject the object dependency implicitly. It internally uses setter or constructor injection.

Qualifier

@Qualifier is an annotation used to specify which Spring managed bean should be injected via @Autowired.

Bean

- Any object in the Spring framework that we initialize through Spring container is called Spring Bean.
- Any normal Java POJO class can be a Spring Bean if it's configured to be initialized via container by providing configuration metadata information.
- In Spring, the objects that form the backbone of your application and that are managed by the Spring IoC container are called beans.
- A bean is an object that is instantiated, assembled, and managed by a Spring IoC container.

Unit- 8

INDEX

1] Application Programming Interface (API)

2] API endpoints

3] Basics of REST

4] Spring REST

Application Programming Interface (API)?

Application Programming Interface (API) is a software interface that allows two applications or software's to interact, communicate and exchange data with each other without any user intervention.

API is a collection of software functions and procedures.

How an API works:

1. **A client application initiates an API call** to retrieve information—also known as a *request*. This request is processed from an application to the web server via the API's Uniform Resource Identifier (URI) and includes a request verb, headers, and sometimes, a request body.
2. **After receiving a valid request**, the API makes a call to the external program or web server.
3. **The server sends a *response*** to the API with the requested information.
4. **The API transfers the data** to the initial requesting application.

Why we need APIs?

- API helps two different software's to communicate and exchange data with each other.
- It helps you to embed content from any site or application more efficiently.
- APIs can access app components. The delivery of services and information is more flexible.
- Content generated can be published automatically.
- It allows the user or a company to customize the content and services which they use the most.
- Software needs to change over time, and APIs help to anticipate changes.

API types

There are mainly **4** main types of APIs:

1. **Open APIs:** These types of APIs are publicly available to use like OAuth APIs from Google. It has also not given any restriction to use them. So, they are also known as Public APIs.
2. **Partner APIs:** Specific rights or licenses to access this type of API because they are not available to the public.
3. **Internal APIs:** Internal or private. These APIs are developed by companies to use in their internal systems. It helps you to enhance the productivity of your teams.
4. **Composite APIs:** This type of API combines different data and service APIs.

Types of API Protocols

1] **SOAP** (Simple Objects Access Protocol)

- It is the oldest API protocol in use, emerging in 1998.
- SOAP uses XML files to transfer data between web services.
- These XML files are sent over HTTP/HTTPS transmissions as is common on the internet.
- SOAP also provides flexibility and enables data transmission over other protocols as well such as Transmission Control Protocol(TCP), Simple Mail Transport Protocol(SMTP), User Data Protocol (UDP), etc.

Messages in SOAP are encoded in XML specifically and have a proper defined format:

1. **Envelope:** Literally envelopes the entire message/data in tags.
2. **Header:** Defines all extra information that might be needed to process this data. This is an optional element.
3. **Body:** This is where we actually write the request for data required/ where all the requested data is added.
4. **Fault:** Defines all the errors that may arise in the data during/due to transmission and measures to handle them.

2] **REST (Representational State Transfer)**

- REST protocols overcome SOAP's dependency on XML by supporting data transmission in multiple formats such as JSON, HTML, Python, plain text as well as media files.
- REST APIs follow a client-server architecture and must be *stateless*.
- *Stateless* communication implies that no client data is stored between GET requests.
- These GET requests must be distinct and disconnected. REST assigns every operation a unique URL, so when the server receives a request, it knows which instructions to execute to fulfil the request.
- REST also supports caching. So, the browser can store the results obtained from the request locally and retrieve it periodically as needed, thereby increasing speed and efficiency.

A typical REST request has the following components:

1. **Endpoint:**

The destination URL from which data is being requested.

2. **Method:**

We use predefined methods such as GET, POST, PUT or DELETE to fetch the data. These methods vary from one other. Ex. in when using GET, the data is appended to the end of the URL string, whereas in POST, the data is sent along with the HTTP request.

3. **Headers:**

They define the request's details and dictate the proper format in which the response must be received.

4. **body (data):**

The actual data sent by the service.

Common API examples:

1. Twitter Bots.
2. Log-In Using google or Facebook.
3. Weather Snippers.
4. Pay with PayPal or paytm.
5. Google Maps.
6. Travel Booking.
7. E-Commerce.

What is API endpoint?

- An endpoint is one end of a communication channel. When an API interacts with another system, the touchpoints of this communication are considered endpoints. For APIs, an endpoint can include a URL of a server or service.
- Each endpoint is the location from which APIs can access the resources they need to carry out their function.

Why are API endpoints important?

- Endpoints help to depict the exact location of the resources to be accessed by API and also play a vital role in ensuring that the software which is interacting with the API is functioning correctly.
- The performance and productivity of APIs depend on its ability to interact and communicate with endpoints effectively.

API endpoint examples

- Twitter API Endpoint
- Spotify API Endpoint
- YouTube API Endpoint

HTTP / HTTPS

- HTTP stands for Hypertext Transfer Protocol.
- Hypertext Transfer Protocol is a set of rule which is used for transferring the files like, audio, video, image, text and other multimedia files on the WWW.
- HTTP is an application-level protocol. The communication usually takes place through TCP/IP sockets, but any reliable transport can also be used.
- This latest version is already in use on the web with the help of UDP (User Datagram Protocol) instead of TCP (Transmission Control Protocol) for the underlying transport protocol.
- HTTP is used to make communication between a variety of hosts and clients. It supports a mixture of network configuration.
- HTTP is a protocol that is used to transfer the hypertext from the client end to the server end, but HTTP does not have any security.

- Whenever a user opens their Web Browser, that means the user indirectly uses HTTP

HTTP working

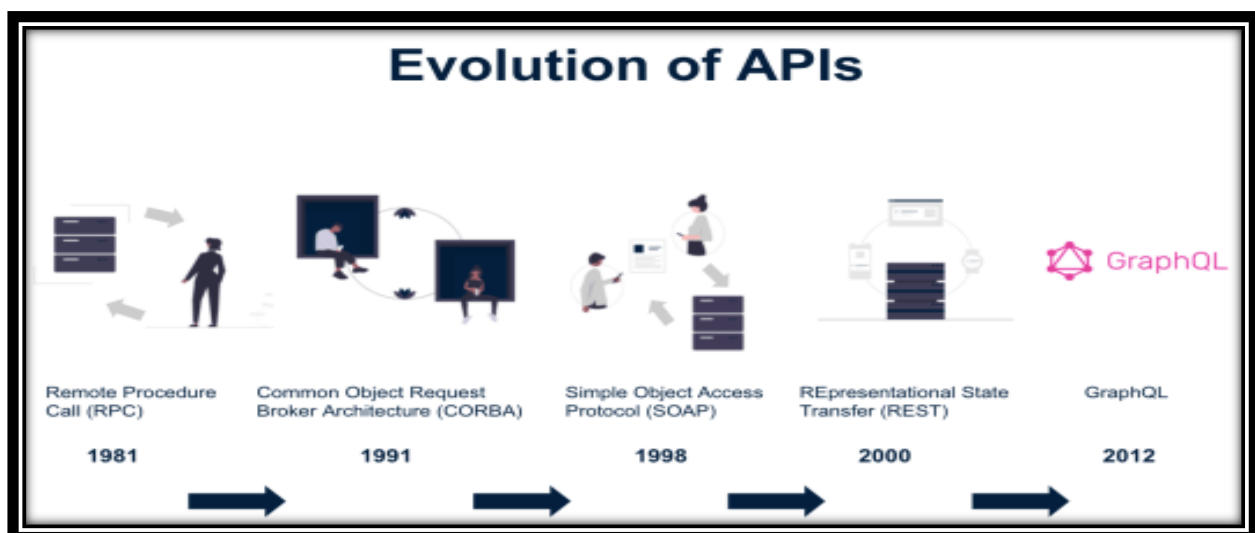
As a request-response protocol, HTTP gives users a way to interact with web resources such as HTML files by transmitting hypertext messages between clients and servers. HTTP clients generally use Transmission Control Protocol (TCP) connections to communicate with servers.

HTTP Method (GET, POST, PUT, DELETE)

1. **GET** requests a specific resource in its entirety
2. **POST** adds content, messages, or data to a new page under an existing web resource
3. **PUT** directly modifies an existing web resource or creates a new URI if need be
4. **DELETE** gets rid of a specified resource

Basics of **REST**

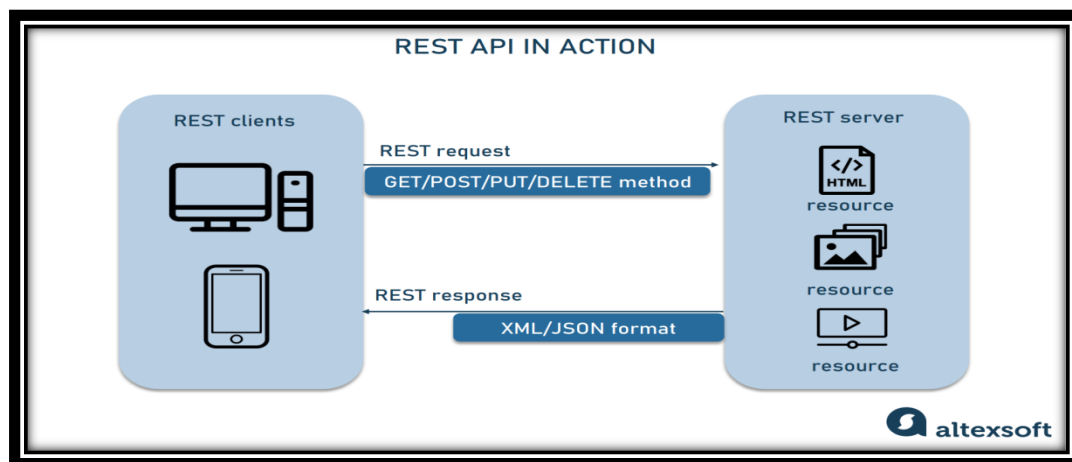
- **Representational State Transfer (REST)** is an architectural style that defines a set of constraints(rules) to be used for creating web services.
- **REST API** is a way of accessing web services in a simple and flexible way without having any processing.



- REST technology is generally preferred to the more robust Simple Object Access Protocol (SOAP) technology, simple and flexible making it more suitable for internet usage.
- It's used to fetch or give some information from a web service. All communication done via REST API uses only HTTP request.

Working

- A request is sent from client to server in the form of a web URL as HTTP GET or POST or PUT or DELETE request.
- After that, a response comes back from the server in the form of a resource which can be anything like HTML, XML, Image, or JSON.
- But now JSON is the most popular format being used in Web Services.



HTTP methods

Following four HTTP methods are commonly used in REST based architecture.

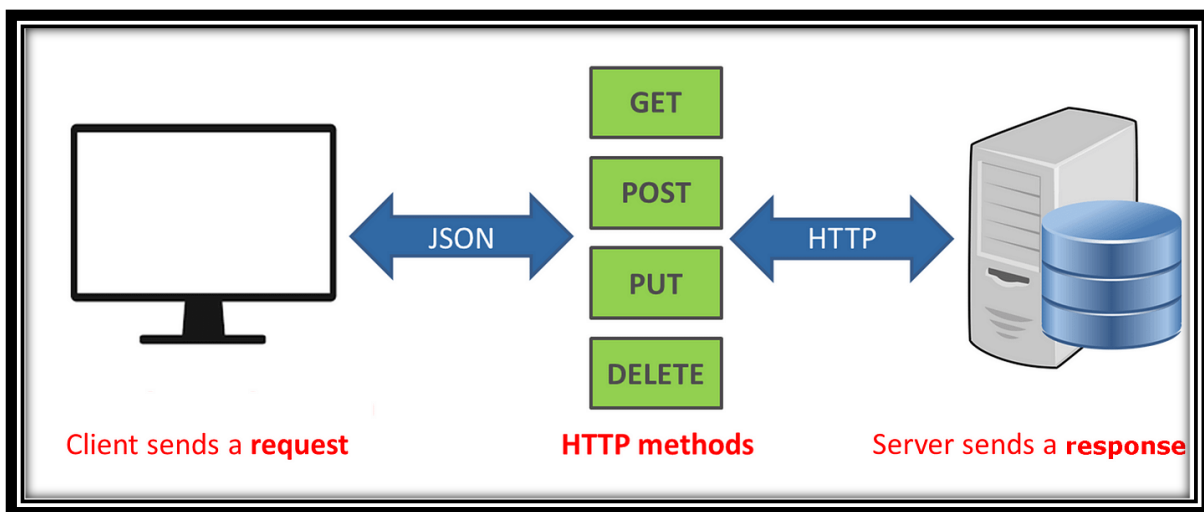
- GET – Provides a read only access to a resource.
- POST – Used to create a new resource.
- DELETE – Used to remove a resource.
- PUT – Used to update an existing resource or create a new resource.

REST ARCHITECTURAL STYLE

- REST is a software architectural style that defines the set of rules to be used for creating web services. Web services which follow the REST architectural style are known as RESTful web services. It allows requesting systems to access and manipulate web resources by using a uniform and predefined set of rules.
- Interaction in REST based systems happen through Internet's Hypertext Transfer Protocol (HTTP).
- A Restful system consists of a
 - client who requests for the resources.
 - server who has the resources.

Components

- POST—Creates a resource.
- GET—Retrieves one or more resources.
- PUT—Updates a resource.
- DELETE—Deletes a resource.



REST Architectural Views

Process View: Process view illustrates the interaction relationship among components. The interaction is in the form of data flow in a system.

Connector View: A connector view of architecture tells on the mechanics of the communication between components.

Data View: A data view of architecture tells the application state as information flows through the components.

Architectural Constraints of RESTful API

1. **Uniform Interface:** uniform way of interacting with a given server
2. **Stateless:** server would not store anything related to the session. In REST, the client must include all information for the server to fulfill the request.
3. **Cacheable:** Every response should include whether the response is cacheable or not and for how much duration responses can be cached at the client side. Client will return the data from its cache for any subsequent request and there would be no need to send the request again to the server.
4. **Client-Server:** REST application should have a client-server architecture. A Client is someone who is requesting resources and server is someone who holds the resources.
5. **Layered System:** A REST API can consist of multiple layers, eg., business logic, presentation, data access
6. **Code on Demand:** It is an optional feature. According to this, servers can also provide executable code to the client.

Properties of REST API

1. **Performance:** REST can support the Performance goal by using caches to keep available data close to where it is being processed.
2. **Scalability:** Scalability means to consistently provide service regardless of the increase or decrease of web users.
3. **Simplicity:** By separating the functionality within components in a system, we can induce the simplicity of the architectural styles.
4. **Modifiability:** Modifiability represents the ease at which changes can be incorporated into the architecture.
5. **Visibility:** Visibility refers to the ability of a component to monitor or mediate the interaction between two components.

6. **Portability:** Portability represents the ease with which a system can be moved from one deployed location to other. portability of components by moving program code with the data;

REST API Design Principles

- REST APIs are designed around *resources*, which are any kind of object, data, or service that can be accessed by the client.
- A resource has an *identifier*, which is a URI that uniquely identifies that resource. For example, the URI for a particular customer order might be:
`https://adventure-works.com/orders/1`
- Clients interact with a service by exchanging *representations* of resources. Many web APIs use JSON as the exchange format. For example, a GET request to the URI listed above might return this response body:
`{"orderId":1,"orderValue":99.90,"productId":1,"quantity":1}`
- REST APIs use a uniform interface, which helps to decouple the client and service implementations. For REST APIs built on HTTP, the uniform interface includes using standard HTTP verbs to perform operations on resources. The most common operations are GET, POST, PUT, PATCH, and DELETE.
- REST APIs use a stateless request model. HTTP requests should be independent and may occur in any order
- REST APIs are driven by hypermedia links that are contained in the representation.

Limitations of JDBC API

- Since a **Prepared Statement** object represents only one SQL statement at a time, we can execute only one statement by one prepared statement object.
- To prevent injection attacks it does not allow more than one value to a place holder.

Object relational Mapping – features and benefits

- ORM stands for **object-relational mapping**, where objects are used to connect the programming language on to the database systems, with the facility to work SQL and object-oriented programming concepts.

Features:

- **Idiomatic persistence:** It enables you to write the persistence classes using object oriented classes.
- **High Performance:** It has many fetching techniques and hopeful locking techniques.
- **Reliable:** It is highly stable and eminent. Used by many industrial programmers.

Benefits:

- It speeds up development time for teams.
- Decreases the cost of development.
- Handles the logic required to interact with databases.
- Improves security. ORM tools are built to eliminate the possibility of SQL injection attacks.
- You write less code when using ORM tools than with SQL.

JPA – Java Persistent API

- The Java Persistence API (JPA) is a specification of Java. It is used to persist data between Java object and relational database.
- JPA acts as a bridge between object-oriented domain models and relational database systems.
- As JPA is just a specification, it doesn't perform any operation by itself. It requires an implementation.
- So, ORM tools like Hibernate, TopLink and iBatis implements JPA specifications for data persistence.

Spring REST controller

In Spring applications, the Controller, Service, and Repository layers form a common architectural pattern that promotes separation of concerns and maintainability. Here's a breakdown of each layer:

Controller Layer:

Purpose: Handles incoming HTTP requests, processes them, and returns responses.

Responsibilities:

- Receives requests from clients (e.g., REST API calls).
- Validates input data.
- Invokes the appropriate service layer methods to perform business logic.
- Transforms data from the service layer into a format suitable for the response (e.g., JSON).
- Sends the response back to the client.

Annotations:

- `@RestController`: Marks the class as a REST controller.
- `@RequestMapping`, `@GetMapping`, `@PostMapping`, etc.: Define the endpoint mappings.
- `@RequestBody`: Maps request body data to method parameters.
- `@PathVariable`: Extracts values from the URL path.

Service Layer:

Purpose: Contains the core business logic of the application.

Responsibilities:

- Implements business rules and operations.
- Orchestrates interactions with the repository layer to retrieve or modify data.
- Handles transactions (if needed).
- Provides a layer of abstraction between the controller and the repository.

Annotations:

- `@Service`: Marks the class as a service component.

Repository Layer:

Purpose:

Interacts with the database or other data sources to perform CRUD (Create, Read, Update, Delete) operations.

Responsibilities:

- Abstracts data access logic from the service layer.
- Provides methods to query and manipulate data.

- Can utilize Spring Data JPA to simplify database interactions.

Annotations:

- **@Repository**: Marks the class as a repository component.
- Spring Data JPA provides additional annotations like **@JpaRepository** to define common repository methods.

Benefits of this pattern:

- **Separation of concerns**:

Each layer has a clear responsibility, making the code more organized, modular, and easier to maintain.

- **Testability**:

Each layer can be tested independently, facilitating unit testing and mocking.

- **Scalability**:

The service layer can be easily scaled to accommodate complex business logic.

- **Flexibility**:

The repository layer can be easily swapped out to use different data sources without impacting the other layers.

Spring Transactions

@Transactional annotation is the metadata used for managing transactions in the Spring Boot application. To configure **Spring Transaction**, this annotation can be applied at the class level or method level. In an enterprise application, a transaction is a sequence of actions performed by the application that together pipelined to perform a single operation. For example, booking a flight ticket is also a transaction where the end user has to enter his information and then make a payment to book the ticket.

What is API?

An **Application Programming Interface (API)** establishes a connection between computers or between computer programs (applications) by providing readily available codes and information pipelines. It is a type of software interface that acts as a mediator among other pieces of software to streamline the interaction with one other.

What is REST API?

Representational State Transfer, also known as **REST**, is basically a standardized Software Architecture Style, or in simple words, a specific type of API used by the industry to establish a connection between Client and Server. REST API is built to guide the development and design of the World Wide Web's architecture.

REST APIs provide a flexible, lightweight way of integrating computer applications. REST APIs are a simple and standardized approach to communication, which means you don't have to worry about how to format your data, it's all standardized and industry use.

What is Spring Boot RESTful service?

RESTful Web Services are **client and server applications that communicate over the WWW**. RESTful Web Services are REST Architecture based Web Services. In REST Architecture, everything is a resource. RESTful Web Services provides communication between software applications running on different platforms and frameworks.

What are examples of RESTful web services?

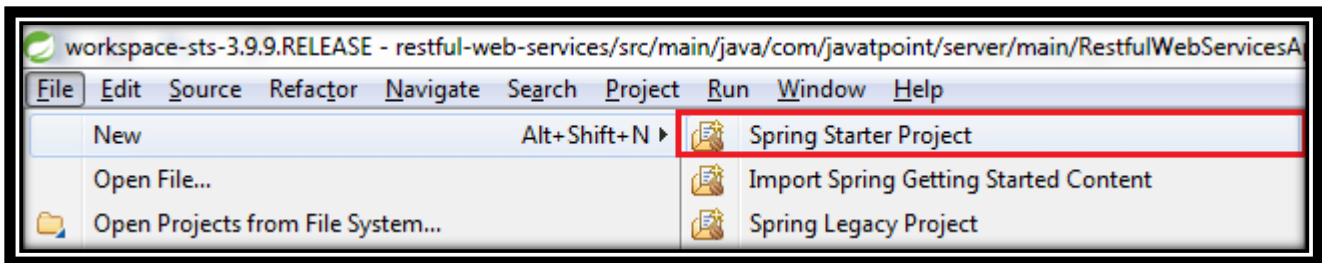
Facebook, Twitter, and Google expose their functionality in the form of Restful web services.

How to create RESTful service

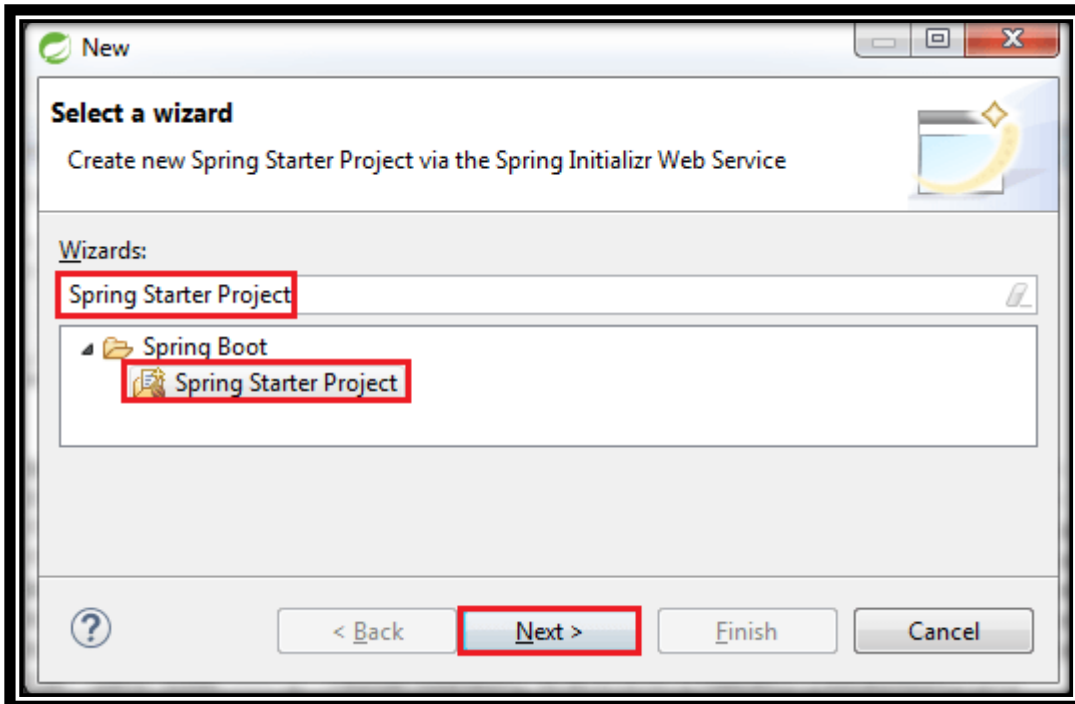
Step 1: Download the **Spring Tool Suite (STS)** from <https://spring.io/tools3/sts/all> and extract it.

Step 2: Launch the **STS**.

Step 3: Click on **File menu -> New -> Spring Starter Project ->**



If the **Spring Starter Project** is not enlisted, then click on **Other** at the bottom of the menu. A dialog box appears on the screen. Type **Spring Starter Project** in the **Wizards** text box and click on the **Next** button.



Step 4: provide the name, group, and package of the project. We have provided:

Name: **restful-web-services**

Group: **com.javatpoint**

Package: **com.javatpoint.server.main**

Click on the **Next** button.

New Spring Starter Project

Service URL:

Name:

☒ Use default location

Location:

Type: Packaging:

Java Version: Language:

Group:

Artifact:

Version:

Description:

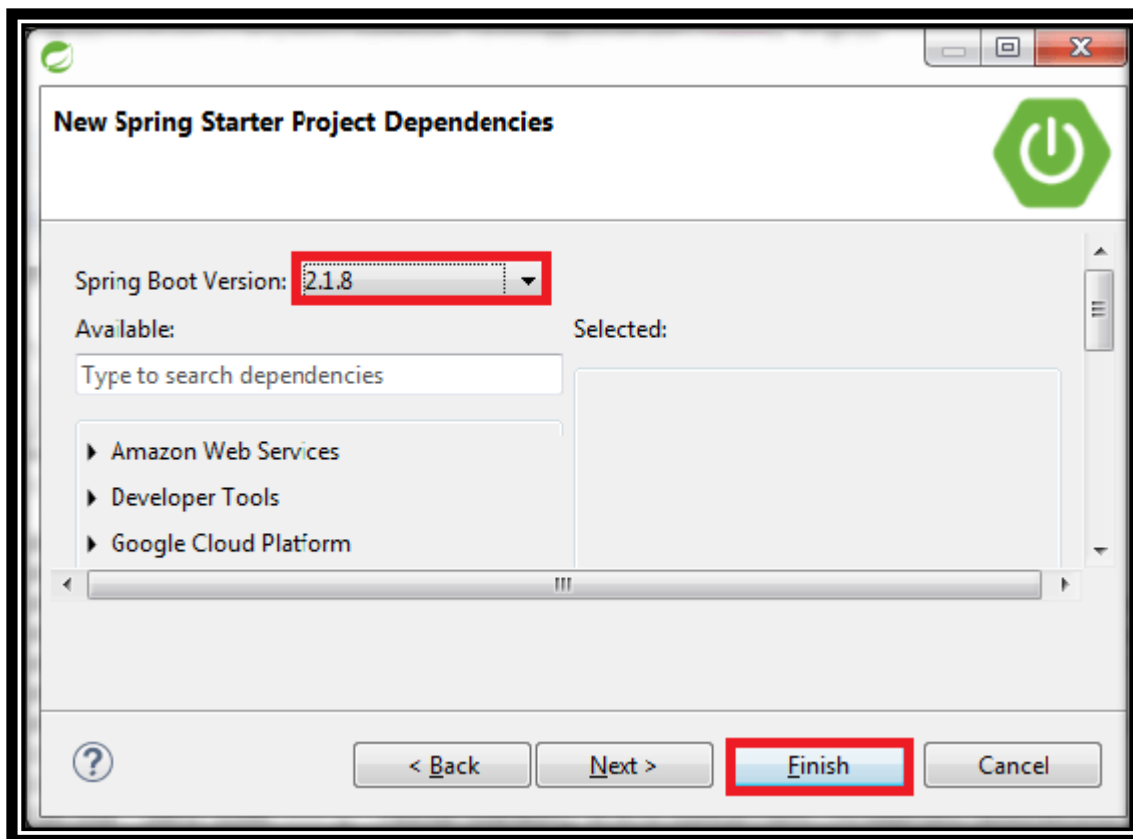
Package:

Working sets

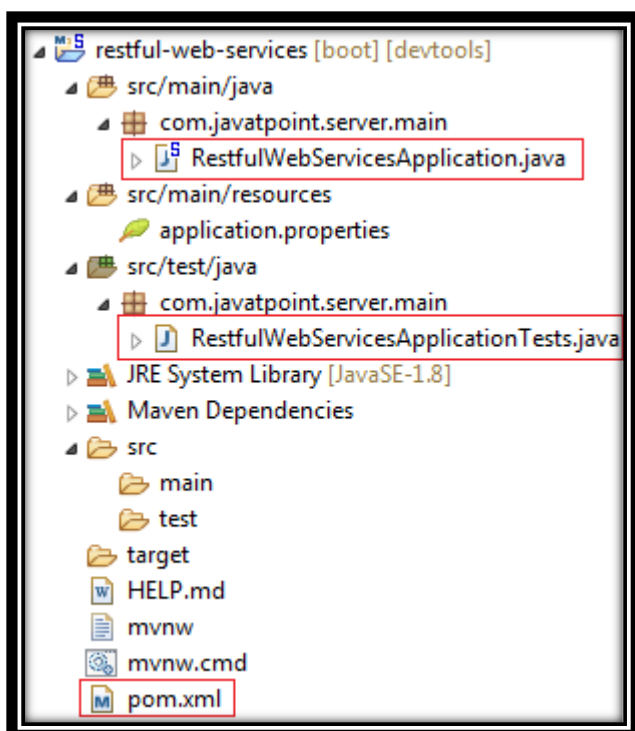
☐ Add project to working sets

Working sets:

Step 5: Choose the Spring Boot Version **2.1.8** and add **Spring Web MVC**, **Spring Boot DevTools**, **JPA**, and **H2** dependencies in the pom.xml.



Step 6: We can see the project structure in the project explorer window.



Step 7: Now open the **RestfulWebServicesApplication.java** file and Run the file as Java Application.

Creating a Hello World Service

Step 1:

Create a new class with the name **HelloWorldController** in the package **com.javatpoint.server.main**.

Step 2:

Whenever we create a web service, we need to define two things **Get** method and the **URI**. Now create the **helloWorld()** method which returns the string "Hello World." If we want to tell the spring MVC that it is going to handle the REST request, we have to add **@RestController** annotation. Now it becomes a rest controller which can handle the Rest request.

```
package com.javatpoint.server.main;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
//Controller
@RestController
public class HelloWorldController
{
//using get method and hello-world as URI
@GetMapping(path="/hello-world")
public String helloWorld()
{
return "Hello World";
}
}
```

Step 3:

Run the **RestfulWebServiceApplication**. It displays the string **Hello World** on the browser.

Enhancing the Hello World Service to Return a Bean

In this section, we are going to generate a bean for the method helloWorld().

Step 1:

Create a **helloWorldBean()** method in **HelloWordController.java** file. Map the URI to **"/hello-world-bean"** and return **HelloWorldBean**.

HelloWorldController.java

```
package com.javatpoint.server.main;
import org.springframework.web.bind.annotation.GetMapping;
```

```

import org.springframework.web.bind.annotation.RestController;
//Controller
@RestController
public class HelloWorldController
{
//using get method and hello-world URI
@GetMapping(path="/hello-world")
public String helloWorld()
{
return "Hello World";
}
@GetMapping(path="/hello-world-bean")
public HelloWorldBean helloWorldBean()
{
return new HelloWorldBean("Hello World"); //constructor of HelloWorldBean
}
}

```

Step 2:

Create a class **HelloWorldBean**.

Step 3:

Generate Getters and **Setters**.

Right-click -> Source -> Generate Getters and Setters -> check the box -> Ok

Step 4:

Generate **toString()**..

Right-click -> Source -> Generate toString().. -> Ok

```

package com.javatpoint.server.main;
public class HelloWorldBean
{
public String message;
//constructor of HelloWorldBean
public HelloWorldBean(String message)
{
this.message=message;
}
}

```

```
//generating getters and setters
public String getMessage()
{
    return message;
}
public void setMessage(String message)
{
    this.message = message;
}
@Override
//generate toString
public String toString()
{
    return String.format ("HelloWorldBean [message=%s]", message);
}
}
```

Step 5:

Launch the **HelloWorldController**. The URL of the browser changes to **localhost:8080/hello-world-bean**.

It returns the message "**Hello World**" in JSON format.

Output:

```
{ message: "Hello World" }
```

Unit 9

Index

1| ACID properties

A.C.I.D properties:

Atomicity, Consistency, Isolation, and Durability

If a database operation has these ACID properties, it can be called an ACID transaction, and data storage systems that apply these operations are called transactional systems.

ACID transactions guarantee that each read, write, or modification of a table has the following properties:

- **Atomicity** - It means if any operation is performed on the data, either it should be performed or executed completely or should not be executed at all.
- **Consistency** – Consistency guarantees that changes made within a transaction are consistent with database constraints.

This includes all rules, constraints, and triggers. If the data gets into an illegal state, the whole transaction fails.

- **Isolation** – Isolation is the property of a database where no data should affect the other one and may occur concurrently.

It means if two operations are being performed on two different databases, they may not affect the value of one another.

- **Durability** - ensures that the data after the successful execution of the operation becomes permanent in the database.

Transaction management and compliance to ACID principles

- A **transaction** is a single logical unit of work that accesses and possibly modifies the contents of a database. Transactions access data using read and write operations.
- In order to maintain consistency in a database, before and after the transaction, certain properties are followed. These are called **ACID** properties.

JUnit Tutorials

Testing: Testing is the process of checking an application that it is working as expected. In other words testing is a process of verification and validation.

Unit testing: Unit testing is the testing of an individual unit (class/method) or group of related units

Types of unit testing:

- **Manual testing:** Manual testing is the process of executing a test case without any tool support.
- **Automated testing:** Automated testing is the process of executing a test case with any tool support.

JUnit: JUnit is an open-source unit testing framework for java programmers. It is only used for unit testing. Integration testing is done by TestNG.

Unit test case:

Unit test case is part of code which executes to check that another part of the code works as expected.

Annotations for Junit testing:

1. **@Test:** It is used to specify the test method.
2. **@BeforeClass:** It is used to specify that method will be called only once, before starting all the test cases.
3. **@AfterClass:** It is used to specify that method will be called only once, after finishing all the test cases.
4. **@Before:** It is used to specify that method will be called before each test case.
5. **@After:** It is used to specify that method will be called after each test case.
6. **@Ignore:** It is used to ignore the test case.

Assert class:

JUnit provides the Assert class to check the certain conditions. Assert class methods compare the output value to the expected value.

Commonly used methods of Assert class:

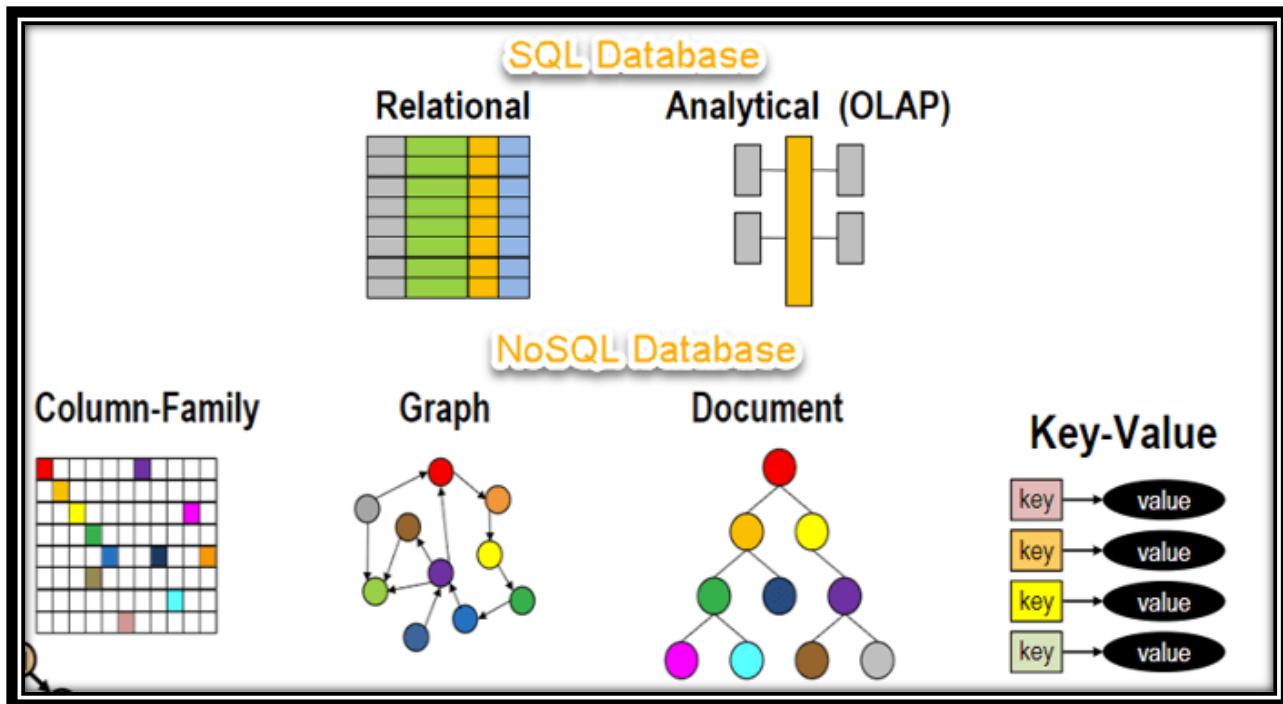
1. **assertTrue(boolean condition):** It assert that the specified boolean condition is true.
2. **assertFalse(boolean condition):** It assert that the specified boolean condition is false
3. **assertNull(Object obj):** It assert that the specified object is null.
4. **assertNotNull(Object obj):** It assert that the specified object is not null.
5. **assertEquals(Object expected, Object actual):** It assert that two objects are equal.
6. **assertSame(Object expected, Object actual):** It assert that two objects refer to the same object.

INTRODUCTION to NOSQL

- NoSQL database stands for “Not Only SQL” or “Not SQL.”
- NoSQL Database is a non-relational Data Management System that does not require a fixed schema.
- It avoids joins, and is easy to scale. The major purpose of using a NoSQL database is for distributed data stores with huge data storage needs.
- NoSQL is used for Big data and real-time web apps. For example, companies like Twitter, Face book and Google collect terabytes of user data every single day.
- NoSQL can store structured, semi-structured, unstructured and polymorphic data

A database Management System provides the mechanism to store and retrieve the data. There are different kinds of database Management Systems:

1. RDBMS (Relational Database Management Systems)
2. OLAP (Online Analytical Processing)
3. NoSQL (Not only SQL)



Features of NoSQL

Non-relational

- NoSQL databases never follow the relational model
- Never provide tables with flat fixed-column records
- Work with self-contained aggregates
- Doesn't require object-relational mapping and data normalization
- No complex features like query languages, query planners, referential integrity joins, ACID

Schema-free

- NoSQL databases are either schema-free or have relaxed schemas
- Do not require any sort of definition of the schema of the data
- Offers heterogeneous structures of data in the same domain

Simple API

- Offers easy to use interfaces for storage and querying data provided
- APIs allow low-level data manipulation & selection methods
- Text-based protocols mostly used with HTTP REST with JSON
- Mostly used no standard based NoSQL query language

- Web-enabled databases running as internet-facing services

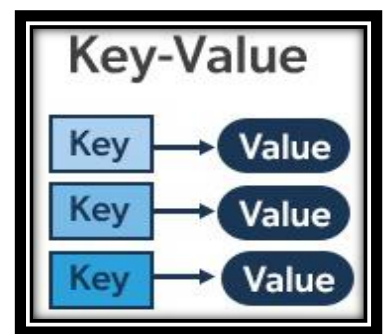
Distributed

- Multiple NoSQL databases can be executed in a distributed fashion
- Offers auto-scaling and fail-over capabilities
- Often ACID concept can be sacrificed for scalability and throughput
- Only providing eventual consistency

Types of NoSQL Databases

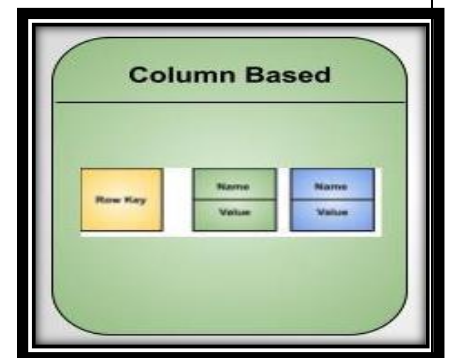
1] Key-value Pair Based:

- Data is stored in key/value pairs. It is designed in such a way to handle lots of data and heavy load.
- Key-value pair storage databases store data as a hash table where each key is unique, and the value can be a JSON, BLOB (Binary Large Objects), string, etc.
- For example, a key-value pair may contain a key like “Website” associated with a value like “Sanpoly”.



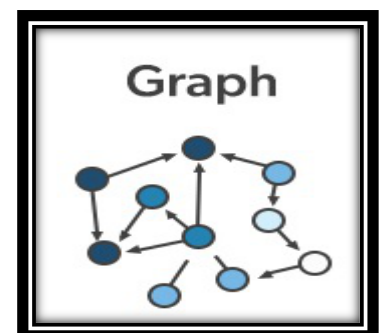
2] Column-oriented Graph

- They store data in tables, rows, and dynamic columns. Here each row doesn't need to have the same columns.
- These were created to store and process very large amounts of data distributed over many machines.



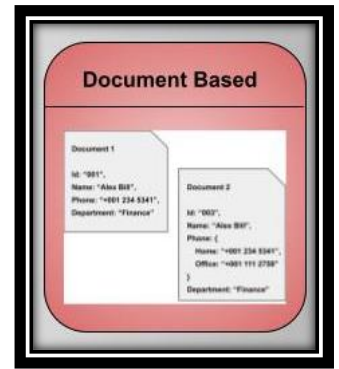
3] Graphs based

- A graph type database stores entities as well the relations amongst those entities. The entity is stored as a node with the relationship as edges. An edge gives a relationship between nodes. Every node and edge has a unique identifier.



4] Document-oriented

- Document-Oriented NoSQL DB stores and retrieves data as a key value pair but the value part is stored as a document.
- The document is stored in JSON or XML formats. The value is understood by the DB and can be queried.



Benefits

1. **High scalability** – NoSQL databases use sharding(dividing a larger part into smaller parts) for horizontal scaling. Partitioning of data and placing it on multiple machines in such a way that the order of the data is preserved is sharding.
2. **High availability** – Auto replication feature in NoSQL databases makes it highly available because in case of any failure data replicates itself to the previous consistent state.

Cap Theorm

The CAP theorem is a belief from theoretical computer science about distributed data stores that claims, in the event of a network failure on a distributed database, it is possible to provide either consistency or availability—but not both.

- **Consistency:**

The data should remain consistent even after the execution of an operation. This means once data is written, any future read request should contain that data. For example, after updating the order status, all the clients should be able to see the same data.

- **Availability:**

The database should always be available and responsive. It should not have any downtime.

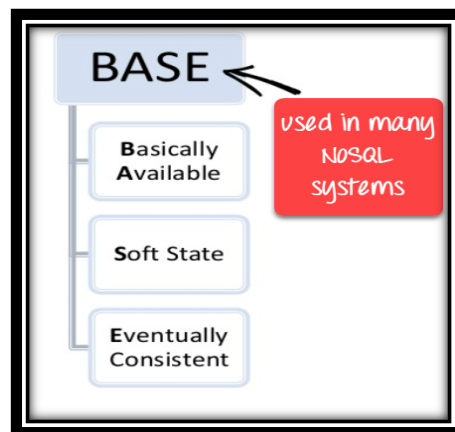
- **Partition Tolerance:**

Partition Tolerance means that the system should continue to function even if the communication among the servers is not stable. For example, the servers can be partitioned

into multiple groups which may not communicate with each other. Here, if part of the database is unavailable, other parts are always unaffected.

BASE: Basically Available, Soft state, Eventual consistency

- Basically, available means DB is available all the time as per CAP theorem
- Soft state means even without an input; the system state may change
- Eventual consistency means that the system will become consistent over time.



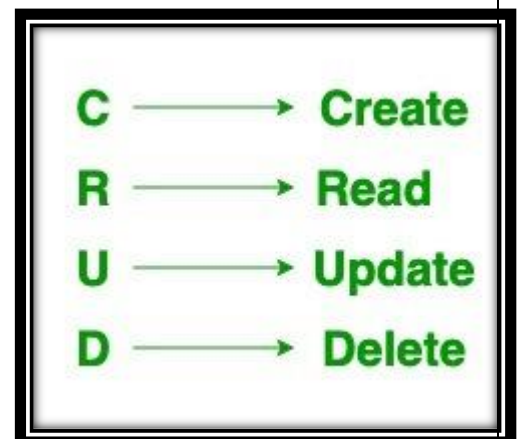
CRUD OPERATIONS

1] Create Operations

Create or insert operations add new [documents](#) to a [collection](#). If the collection does not currently exist, insert operations will create the collection.

MongoDB provides the following methods to insert documents into a collection:

- [db.collection.insertOne\(\)](#)
- [db.collection.insertMany\(\)](#)



In MongoDB, insert operations target a single [collection](#). All write operations in MongoDB are [atomic](#) on the level of a single [document](#).

```
db.users.insertOne(
  {
    name: "sue",
    age: 26,
    status: "pending"
  }
)
```



2] Read Operations

Read operations retrieve [documents](#) from a [collection](#); i.e. query a collection for documents. MongoDB provides the following methods to read documents from a collection:

- [db.collection.find\(\)](#)

```
db.users.find(
  { age: { $gt: 18 } },
  { name: 1, address: 1 }
).limit(5)
```



3] Update Operations

Update operations modify existing [documents](#) in a [collection](#). MongoDB provides the following methods to update documents of a collection:

- [db.collection.updateOne\(\)](#)
- [db.collection.updateMany\(\)](#)
- [db.collection.replaceOne\(\)](#)

In MongoDB, update operations target a single collection. All write operations in MongoDB are [atomic](#) on the level of a single document.

```
db.users.updateMany(
  { age: { $lt: 18 } },
  { $set: { status: "reject" } } )
```



4] Delete Operations

Delete operations remove documents from a collection. MongoDB provides the following methods to delete documents of a collection:

- [db.collection.deleteOne\(\)](#)
- [db.collection.deleteMany\(\)](#)

In MongoDB, delete operations target a single [collection](#). All write operations in MongoDB are [atomic](#) on the level of a single document.

You can specify criteria, or filters, that identify the documents to remove. These [filters](#) use the same syntax as read operations.

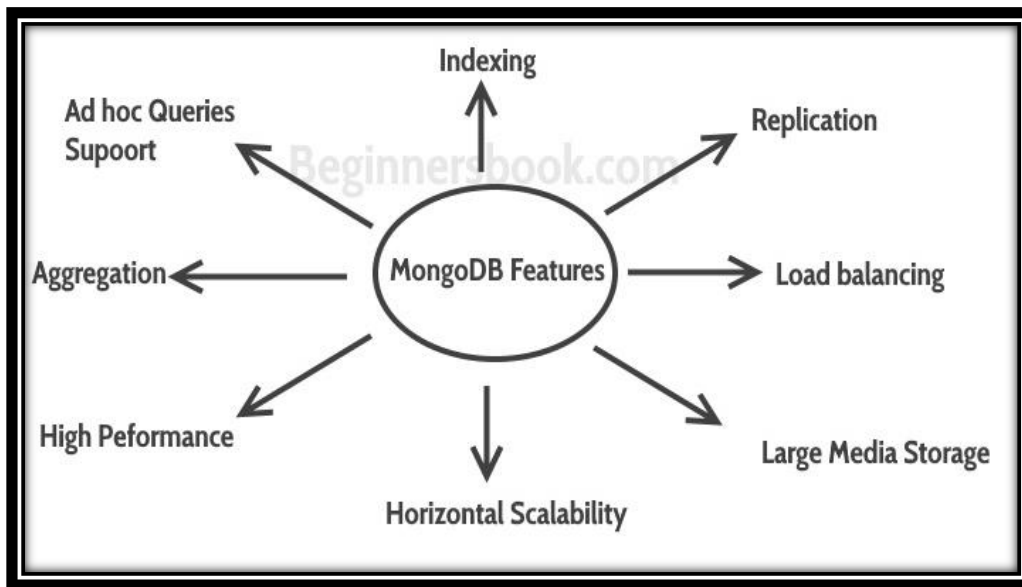
```
db.users.deleteMany(
  { status: "reject" } )
```



Mongo DB

- MongoDB is a document-oriented NoSQL database used for high volume data storage.
- Instead of using tables and rows, MongoDB makes use of collections and documents.
- Documents consist of key-value pairs which are the basic unit of data in MongoDB.
- Collections contain sets of documents and function which is the equivalent of relational database tables.

MongoDB Features



- MongoDB provides **high performance**. Most of the operations in the MongoDB are faster compared to relational databases.
- MongoDB provides **auto replication** feature that allows you to quickly recover data in case of a failure.
- **Horizontal scaling** is possible in MongoDB because of sharing. Sharing is partitioning of data and placing it on multiple machines in such a way that the order of the data is preserved.
- **Vertical scaling** means adding more resources to the existing machine.
- **Load balancing**: horizontal scaling allows MongoDB to balance the load.
- **High Availability**: Auto Replication improves the availability of MongoDB database.
- **Indexing**: Index is a single field within the document. Indexes are used to quickly locate data without having to search every document.

Key Components of MongoDB Architecture

1._id:

- The `_id` field represents a unique value in the MongoDB document.
- The `_id` field is like the document's primary key.

- If you create a new document without an `_id` field, MongoDB will automatically create the field.

2. Collection: This is a grouping of MongoDB documents. A collection is the equivalent of a table.

3. Cursor: This is a pointer to the result set of a query. Clients can iterate through a cursor to retrieve results.

4. Database: This is a container for collections (tables).

5. Document: A record in a MongoDB collection is basically called a document. The document, in turn, will consist of field name and values.

6. Field: A name-value pair in a document. A document has zero or more fields. Fields are analogous to columns in relational databases.

7. JSON: This is known as JavaScript Object Notation. This is a human-readable, plain text format for expressing structured data.

Data Modelling in MongoDB

MongoDB provides two types of data models

- **Embedded data model:** In this model, you can have (embed) all the related data in a single document, it is also known as de-normalized data model

```
{
  _id: , Emp_ID: "10025AE336"
  Personal_details:
  {
    First_Name: "Radhika",
    Date_Of_Birth: "1995-09-26"
  },
  Address:
  {
    city: "Mysore",
  }
}
```

- **Normalized data model:** In this model, you can refer the sub documents in the original document, using references.

Employee:

```
{
  _id: <ObjectId101>,

```

```
Emp_ID: "10025AE336"  
}
```

Personal_details:

```
{  
  _id: <ObjectId102>,  
  empDocID: " ObjectId101",  
  First_Name: "Radhika",  
  Date_Of_Birth: "1995-09-26"  
}
```

Address:

```
{  
  _id: <ObjectId103>,  
  empDocID: " ObjectId101",  
  city: "Mysore"  
}
```

Working with MongoDB

Mongo Compass GUI Setup

Download and Install MongoDB Community Server

- Visit www.mongodb.com and click on MongoDB Community Edition, select a version, platform, and package to download.
- To install follow the instructions.

MongoDB Shell – mongosh

- MongoDB Shell is the quickest way to connect, configure, query, and work with your MongoDB database. It acts as a command-line client of the MongoDB server.

Install mongosh

- Visit www.mongodb.com and click on Product menu -> Tools -> Compass, then click on Download.
- This will take you to a page where you can select a version, platform, and package to download. Zip file will be downloaded.
- Extract the file.

- Open mongosh folder, goto bin, double click on mongosh, press enter
- Type show dbs

MongoDB offers the following query operator types:

- Comparison
- Logical
- Element
- Evaluation
- Geospatial
- Array
- Bitwise
- Comments

Comparison Operators

MongoDB comparison operators can be used to compare values in a document. The following table contains the common comparison operators.

Operator	Description
\$eq	Matches values that are equal to the given value.
\$gt	Matches if values are greater than the given value.
\$lt	Matches if values are less than the given value.
\$gte	Matches if values are greater or equal to the given value.
\$lte	Matches if values are less or equal to the given value.
\$in	Matches any of the values in an array.
\$ne	Matches values that are not equal to the given value.
\$nin	Matches none of the values specified in an array.

Logical Operators

MongoDB logical operators can be used to filter data based on given conditions. These operators provide a way to combine multiple conditions. Each operator equates the given condition to a true or false value.

Operator	Description
\$and	Joins two or more queries with a logical AND and returns the documents that match all the conditions.
\$or	Join two or more queries with a logical OR and return the documents that match either query.
\$nor	The opposite of the OR operator. The logical NOR operator will join two or more queries and return documents that do not match the given query conditions.
\$not	Returns the documents that do not match the given query expression.

Array Operators

MongoDB array operators are designed to query documents with arrays. Here are the array operators provided by MongoDB.

Operator	Description
\$all	Matches arrays that contain all the specified values in the query condition.
\$size	Matches the documents if the array size is equal to the specified size in a query.
\$elemMatch	Matches documents that match specified \$elemMatch conditions within each array element.

Securing REST APIs with Spring Security

In order to add security to our Spring Boot application, we need to add the *security starter dependency*

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

This will also include the *SecurityAutoConfiguration* class containing the initial/default security configuration.

By default, the Authentication gets enabled for the Application. Also, content negotiation is used to determine if basic or formLogin should be used.

There are some predefined properties:

```
spring.security.user.name=root
spring.security.user.password=root
```

If we don't configure the password using the predefined property *spring.security.user.password* and start the application, a default password is randomly generated and printed in the console log:

Using default security password: c8be15de-4488-4490-9dc6-fab3f91435c6

File - new – Project - spring starter project

Name: spring-basic-security

Package: com.example.security

Click Next - Add Dependencies: Spring Web, Spring Security, Spring Boot Dev Tools....

Finish

Name: SpringBasicSecurityApplication

package com.example.security;

SpringBasicSecurityApplication.java

```
import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
```

```
@SpringBootApplication
```

```
@EnableWebSecurity
```

```

public class SpringBasicSecurityApplication {
    public static void main(String[] args) {
        SpringApplication.run(SpringBasicSecurityApplication.class, args);
    }
}

```

Create Package

Name: `com.example.security.controller`

Create Class

Name: `ApplicationController`

```

package com.example.security.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/auth")
public class ApplicationController {
    @GetMapping("/getmsg")
    public String greeting()
    {
        return "Spring Security Example";
    }
}

```

application.properties File

```

spring.security.user.name=niranjana
spring.security.user.password=murthy
server.port=8090

```

Writing Junit test cases for CRUD operations

Download JUnit from <https://junit.org/junit4/>

Goto download & install

Find Plain-old Jar & Download the following

- [junit.jar](#)
- [hamcrest-core.jar](#)
- Create a folder in any drive by giving relevant name, copy and paste both jar files to the folder.
- Create a project in eclipse
- Right click on project select build path, click on configure build path
- Select java build path, Click on Libraries and click on class path in libraries, go to Add External JAR's, select junit.jar and hamcrest-core.jar files, click on apply and then apply and close.
- Goto src/test/java folder find default package and Testclass
- Write the below code

```
package com.example.demo;  
import static org.assertj.core.api.Assertions.assertThat;  
import static org.junit.Assert.assertNotEquals;  
import static org.junit.Assert.assertNotNull;  
import java.util.List;  
import org.junit.jupiter.api.Test;  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.boot.test.context.SpringBootTest;  
import com.example.demo.entity.User;  
import com.example.demo.repository.UserRepository;
```

```
@SpringBootTest
```

```
class SpringbootFirstAppApplicationTests {  
    @Autowired  
    UserRepository userRepo;  
    @Test  
    public void testCreate()  
    {  
        User u=new User();  
        u.setId(3L);  
        u.setFirstname("Kavya");  
        u.setLasttname("shree");  
        userRepo.save(u);  
        assertNotNull(userRepo.findById(902L).get());  
    }  
}
```

```
@Test
public void testReadAll()
{
    List<User> list=userRepo.findAll();
    assertThat(list).size().isGreaterThan(0);
}

@Test
public void testUpdate()
{
    User u=userRepo.findById(902L).get();
    u.setFirstname("Murthy");
    userRepo.save(u);
    assertNotEquals("Niranjan",userRepo.findById(902L).get().getFirstname());
}

@Test
public void testDelete()
{
    userRepo.deleteById(852L);
    assertThat(userRepo.existsById(852L)).isFalse();
}
}
```

UNIT- 10

Create and Drop database

MongoDB use DATABASE_NAME is used to create database. The command will create a new database if it doesn't exist, otherwise it will return the existing database.

Syntax: use DATABASE_NAME

```
>use mydb  
switched to db mydb
```

If you want to check your databases list, use the command show dbs.

```
>show dbs  
local    0.78125GB  
test     0.23012GB
```

MongoDB db.dropDatabase() command is used to drop a existing database.

Syntax db.dropDatabase()

```
>use mydb  
switched to db mydb  
>db.dropDatabase()  
>{ "dropped" : "mydb", "ok" : 1 }
```

Create and Drop Collections

db.createCollection(name, options) is used to create collection.

Syntax: db.createCollection(name, options)

```
>use test  
switched to db test
```

```
>db.createCollection("mycollection")  
{ "ok" : 1 }
```

```
>show collections  
mycollection  
system.indexes
```

db.collection.drop() is used to drop a collection from the database.

Syntax:

```
db.COLLECTION_NAME.drop()
```

```
>db.mycollection.drop()  
true
```

CRUD Operation on MongoDB

1] Creating a Table.

```
db.createCollection("student")  
{ ok: 1 }  
show tables  
student
```

2] insert() Method

To insert data into MongoDB collection, you need to use MongoDB's insert() or save() method.

Syntax: db.COLLECTION_NAME.insert(document)

```
db.student.insert({"id":1,"name":"chandru","mark":300})
```

```
db.student.insertMany([{"id":1,"name":"chandru","mark":300},  
                        {"id":2,"name":"suman","mark":290}])
```

3] View data from Table.

```
db.student.find({})
```

4] Update.

```
db.student.update({"name":"chandru"},{$set:{"name":"sekar",id:5}})
```

5] Delete only one data.

```
db.student.deleteOne({"name":"sekar"})
```

6] Indexing

```
db.indexing.createIndex({"name":1})
```

```
'name_1'
```

7] equal.

```
db.student.find({name:{$eq:"chandru"}})
```

8] GreaterThan.

```
db.student.find({mark:{$gt:300}})
```

9] LessThan.

```
db.student.find({mark:{$lt:400}})
```

10] GreaterThan and LessThan in Between

```
db.student.find({mark:{$gt:290,$lt:400}})
```

11] Sort

```
db.student.find({}, {id:0}).sort({"name":1})
```

12] Count.

```
db.student.countDocuments()
```

```
db.student.countDocuments({"name":"chandru"})
```

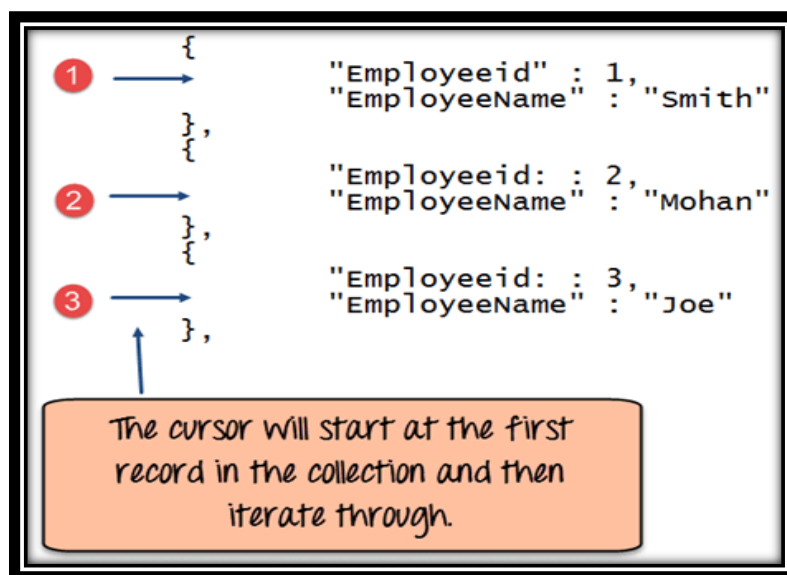
13] Limit: This method limits the number of records or documents that you want.

```
db.student.find().limit(2)
```

14] Cursor:

When the **db.collection.find ()** function is used to search for documents in the collection, the result returns a pointer to the collection of documents returned which is called a cursor.

By default, the cursor will be iterated automatically when the result of the query is returned.



Indexing

- The index stores the value of a specific field or set of fields, ordered by the value of the field as specified in the index.

The createIndex() Method

Syntax: createIndex()

```
>db.COLLECTION_NAME.createIndex({KEY:1})
```

- Here key is the name of the field on which you want to create index and 1 is for ascending order.
- To create index in descending order you need to use -1.

```
>db.mycol.createIndex({"title":1})
{
  "createdCollectionAutomatically" : false,
  "numIndexesBefore" : 1,
  "numIndexesAfter" : 2,
  "ok" : 1
}
```

In **createIndex()** method you can pass multiple fields, to create index on multiple fields.

```
>db.mycol.createIndex({"title":1,"description":-1})
```

The dropIndex() method

Syntax: DropIndex()

```
>db.COLLECTION_NAME.dropIndex({KEY:1})
```

Here, "key" is the name of the file on which you want to remove an existing index.

You can also specify the name of the index directly as:

```
dropIndex("name_of_the_index")
```

```
> db.mycol.dropIndex({"title":1})
{
  "ok" : 0,
  "errmsg" : "can't find index with key: { title: 1.0 }",
  "code" : 27,
  "codeName" : "IndexNotFound"
}
```

The dropIndexes() method

This method deletes multiple (specified) indexes on a collection.

Syntax: db.COLLECTION_NAME.dropIndexes()

```
> db.mycol.createIndex({"title":1,"description":-1})
```

```
>db.mycol.dropIndexes({"title":1,"description":-1})
{ "nIndexesWas" : 2, "ok" : 1 }
```

The getIndexes() method

This method returns the description of all the indexes int the collection.

Syntax: db.COLLECTION_NAME.getIndexes()

```
> db.mycol.getIndexes()
[ {
```

```
    "v" : 2,
    "key" : {
        "_id" : 1
    },
    "name" : "_id_",
    "ns" : "test.mycol"
},
{
    "v" : 2,
    "key" : {
        "title" : 1,
        "description" : -1
    },
    "name" : "title_1_description_-1",
    "ns" : "test.mycol"
}]
```

Aggregation

- Aggregations operations process data records and return computed results.
- Aggregation operations group values from multiple documents together, and can perform a variety of operations on the grouped data to return a single result.
- In SQL count(*) and with group by is an equivalent of MongoDB aggregation.

The aggregate() Method

Syntax: aggregate()

>db.COLLECTION_NAME.aggregate(AGGREGATE_OPERATION)

```
{
  _id: ObjectId(7df78ad8902c)

  by_user: 'tutorials point',
  url: 'http://www.tutorialspoint.com',
  likes: 100
},
{
  _id: ObjectId(7df78ad8902d)
  by_user: 'tutorials point',
  url: 'http://www.tutorialspoint.com',
  likes: 10
},
```

Expression	Description	Example
\$sum	Sums up the defined value from all documents in the collection.	db.mycol.aggregate([{\$group : {_id : "\$by_user", num_tutorial : {\$sum : "\$likes"}}}])
\$avg	Calculates the average of all given values from all documents in the collection.	db.mycol.aggregate([{\$group : {_id : "\$by_user", num_tutorial : {\$avg : "\$likes"}}}])

\$min	Gets the minimum of the corresponding values from all documents in the collection.	db.mycol.aggregate([{\$group : {_id : "\$by_user", num_tutorial : {\$min : "\$likes"}}}]))
\$max	Gets the maximum of the corresponding values from all documents in the collection.	db.mycol.aggregate([{\$group : {_id : "\$by_user", num_tutorial : {\$max : "\$likes"}}}]))
\$push	Inserts the value to an array in the resulting document.	db.mycol.aggregate([{\$group : {_id : "\$by_user", url : {\$push: "\$url"}}}]))
\$addToSet	Inserts the value to an array in the resulting document but does not create duplicates.	db.mycol.aggregate([{\$group : {_id : "\$by_user", url : {\$addToSet : "\$url"}}}]))
\$first	Gets the first document from the source documents according to the grouping. Typically this makes only sense together with some previously applied “\$sort”-stage.	db.mycol.aggregate([{\$group : {_id : "\$by_user", first_url : {\$first : "\$url"}}}]))

Perform CRUD Operations on MongoDB through REST API using Spring Boot Starter Data MongoDB

Step 1: Create a Spring Boot project.

Step 2: Add the following dependency

- Spring Web
- MongoDB
- Lombok
- DevTools

Step 3: Create 3 packages and create some classes and interfaces inside these packages

- entity
- repository
- controller

Step 4: Inside the entity package create a Book.java file.

```
package com.example.security.entity;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;
@Data
@NoArgsConstructor
@AllArgsConstructor
@Document(collection = "Book")
public class Book {
    @Id
    private int id;
    private String bookName;
    private String authorName;
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getBookName() {
        return bookName;
    }
}
```

```

    public void setBookName(String bookName) {
        this.bookName = bookName;
    }
    public String getAuthorName() {
        return authorName;
    }
    public void setAuthorName(String authorName) {
        this.authorName = authorName;
    }
}

```

Step 5: Inside the repository package

Create a simple interface and name the interface as BookRepo. This interface is going to extend the MongoRepository

```

package com.example.security.repository;
import org.springframework.data.mongodb.repository.MongoRepository;
import com.example.security.entity.Book;
public interface BookRepo
    extends MongoRepository<Book, Integer> {
}

```

Step 6: Inside the controller package. Inside the package create one class named as BookController

```

package com.example.security.controller;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import com.example.security.entity.Book;
import com.example.security.repository.BookRepo;

```

```

import java.util.List;

@RestController
public class BookController {

    @Autowired
    private BookRepo repo;

    @PostMapping("/addBook")
    public String saveBook(@RequestBody Book book){
        repo.save(book);
        return "Added Successfully";
    }

    @GetMapping("/findAllBooks")
    public List<Book> getBooks() {
        return repo.findAll();
    }

    @DeleteMapping("/delete/{id}")
    public String deleteBook(@PathVariable int id){
        repo.deleteById(id);
        return "Deleted Successfully";
    }
}

```

Step 7: Below is the code for the application.properties file

```

server.port=8989
spring.data.mongodb.host=localhost
spring.data.mongodb.port=27017
spring.data.mongodb.database=jss

```

Step 8: Inside the MongoDB Compass

Go to your MongoDB Compass and create a Database named BookStore and inside the database create a collection named Book

Testing the Endpoint in Postman

POST – <http://localhost:8989/addBook>

GET – <http://localhost:8989/findAllBooks>

DELETE – <http://localhost:8989/delete/1>

Application Testing

What is Application Testing?

Application Testing is defined as a software testing type conducted through scripts with the motive of finding errors in software. It deals with tests for the entire application.

It helps to enhance the quality of your applications while reducing costs, maximizing ROI, and saving development time.

What is Manual Testing?

Manual testing is testing of the software where tests are executed manually by a QA Analyst. It is performed to discover bugs in software under development.

What is Automation Testing?

In Automated Software Testing, testers write code/test scripts to automate test execution. Testers use appropriate automation tools to develop the test scripts and validate the software. The goal is to complete test execution in a less amount of time.

Manual Testing Vs. Automation Testing

Parameter	Automation Testing	Manual Testing
Definition	Automation Testing uses automation tools to execute test cases.	In manual testing, test cases are executed by a human tester and software.
Processing time	Automated testing is significantly faster than a manual approach.	Manual testing is time-consuming and takes up human resources.
Reliability	Automated testing is a reliable method, as it is performed by tools and scripts. There is no testing Fatigue.	Manual testing is not as accurate because of the possibility of the human errors.

Parameter	Automation Testing	Manual Testing
Investment	Investment is required for testing tools as well as automation engineers	Investment is needed for human resources.
Cost-effective	Not cost effective for low volume regression	Not cost effective for high volume regression.
Test Report Visibility	With automation testing, all stakeholders can login into the automation system and check test execution results	Manual Tests are usually recorded in an Excel or Word, and test results are not readily/ readily available.
Batch testing	You can Batch multiple Test Scripts for nightly execution.	Manual tests cannot be batched.
Programming knowledge	Programming knowledge is a must in automation testing.	No need for programming in Manual Testing.
Deadlines	Automated Tests have zero risks of missing out a pre-decided test.	Manual Testing has a higher risk of missing out the pre-decided test deadline.
When to Use?	Automated Testing is suited for Regression Testing, Performance Testing, Load Testing or highly repeatable functional test cases.	Manual Testing is suitable for Exploratory, Usability and Adhoc Testing. It should also be used where the AUT changes frequently.

Application testing tools

There are various testing tools for Application Testing. Selection of tools depends on what type of testing you want to perform. For different platform different tools are recommended. Application testing tools ensure performance, usability and functionality of applications across a variety of devices.

- Selenium
- IBM Rational Robot

- RFT (Rational Functional Tester)
- Load Runner (HP Performance Tester)
- Apache Jmeter

FUNCTIONAL TESTING

Functional testing is a type of software testing that validates the software system against the functional requirements/specifications. The purpose of Functional tests is to test each function of the software application, by providing appropriate input, verifying the output against the Functional requirements.

How to do Functional Testing:

- Understand the Functional Requirements
- Identify test input or test data based on requirements
- Compute the expected outcomes with selected test input values
- Execute test cases
- Compare actual and computed expected results

What is GUI Testing?

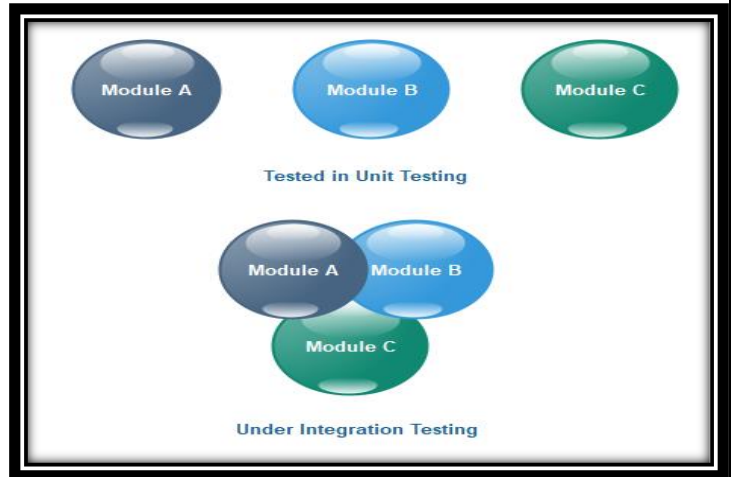
GUI Testing is a software testing type that checks the Graphical User Interface of the Software. The purpose of Graphical User Interface (GUI) Testing is to ensure the functionalities of software application work as per specifications by checking screens and controls like menus, buttons, icons, etc.

What do you Check-in GUI Testing?

- Check all the GUI elements for size, position, width, length, and acceptance of characters or numbers. For instance, you must be able to provide inputs to the input fields.
- Check you can execute the intended functionality of the application using the GUI
- Check Error Messages are displayed correctly
- Check for Clear demarcation of different sections on screen
- Check Font used in an application is readable
- Check the alignment of the text is proper
- Check the Color of the font and warning messages is aesthetically pleasing
- Check that the images have good clarity
- Check that the images are properly aligned
- Check the positioning of GUI elements for different screen resolution.

What is Integration Testing?

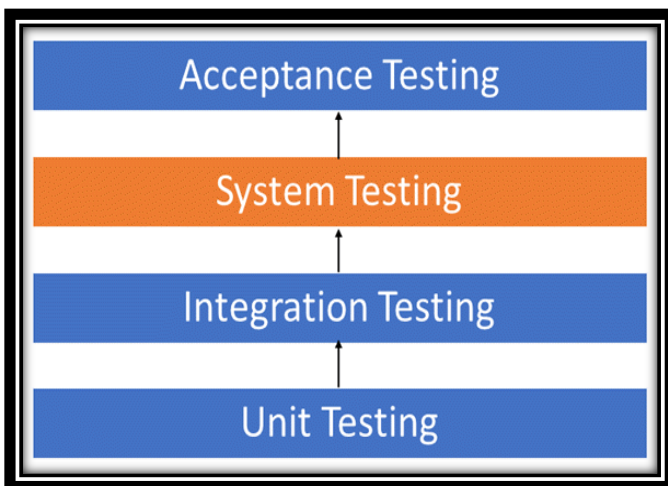
Integration Testing is defined as a type of testing where software modules are integrated logically and tested as a group. A typical software project consists of multiple software modules, coded by different programmers. The purpose of this level of testing is to expose defects in the interaction between these software modules when they are integrated



System testing

System Testing is a level of testing that validates the complete and fully integrated software product. The purpose of a system test is to evaluate the end-to-end system specifications.

Integrate the work of each group and carry out integration testing



Acceptance testing

Acceptance Testing is the last phase of software testing performed after System Testing and before making the system available for actual use.

Acceptance tests and test plan

The acceptance test plan or system test plan is based on the requirement specifications and is required for a formal test environment.

Acceptance evaluates the functionality and performance of the entire application and consists of a variety of tests like.

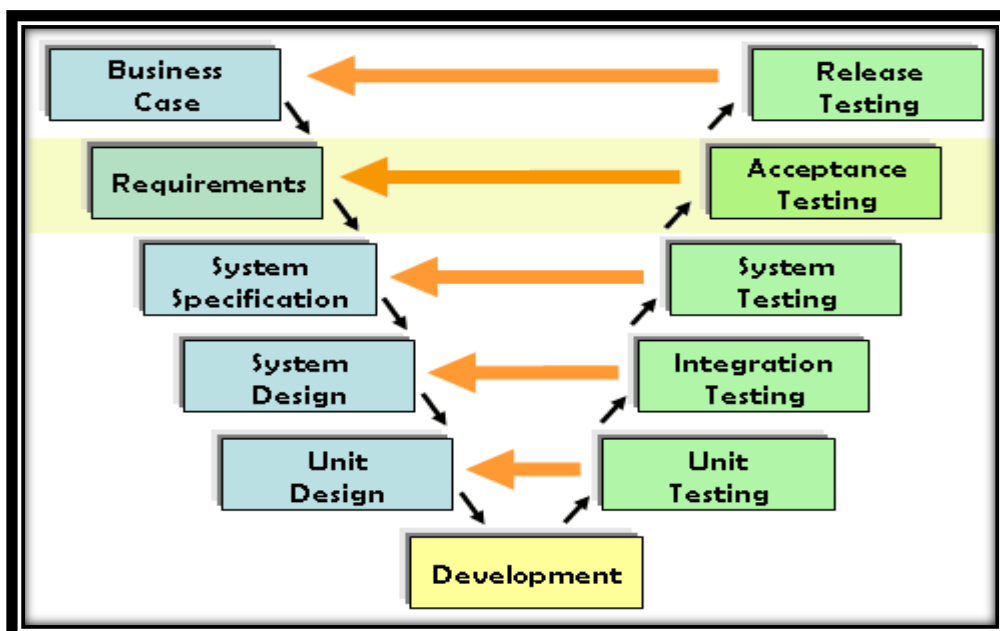
- a) Performance Tests
- b) Usability Tests
- c) Stress Tests
- d) Documentation Tests
- e) Security Tests
- f) Volume Tests
- g) Recovery Tests etc.

User acceptance testing

User Acceptance Testing (UAT) is a type of testing performed by the end user or the client to verify/accept the software system before moving the software application to the production environment. UAT is done in the final phase of testing after functional, integration and system testing is done.

UAT is performed by –

- ☐ Client
- ☐ End users



Deployment process

- Software deployment includes all of the steps, processes, and activities that are required to make a software system or update available to its intended users.
- Today, most IT organizations and software developers deploy software updates, patches and new applications with a combination of manual and automated processes.

Different types of application or software deployment strategies are:

- Basic
- Multi-service
- Rolling
- Blue Green Deployment
- Canary
- A/B Testing

Manual deployment

Manual configuration is intended for advanced users when more control of the application server environment is required. When the configuration utility is run with the manual deployment mode selected, it makes necessary changes to the database, but your application server is not changed. Deployable application files, installation scripts, and properties files are generated in the *<repository installation directory>/toDeploy/<timestamp>* directory.

Automated deployment

Deployment automation is the use of automated processes to move your software between testing and production environments. This process is both reliable and repeatable across the software delivery lifecycle (SDLC).

Some of the best deployment automation tools available include:

- [Jenkins](#)
- [Visual Studio](#)
- [AWS CodeDeploy](#)

Deployment automation benefits

- Anyone can deploy
- Faster, more efficient deployment

- Increased productivity
- Fewer errors
- More frequent releases
- Immediate feedback

Top Deployment tools and their features

1. Bamboo

Key Features:

- Language agnostic integration with many platforms
- Dedicated agents focus on specific build projects
- Full reporting of upcoming code changes and issues

2. TeamCity

Key Features:

- Start within minutes from installation
- Support for third-party tools without plugins or modifications
- Comprehensive version control system
- Customization and extension of the server

3. AWS CodeDeploy

Key Features:

- Automated and repeatable deployments
- Centralized control
- Applications still accessible during deployments

4. Octopus Deploy

Key Features:

- Dashboard showing deployment statuses
- Installation and set up completed quickly
- Automatic deployments for virtual machines on site or in cloud services

5. ElectricFlow

Key Features:

- Build and deploy microservices
- View release status and milestone dates
- Automate and track cloud-native, container, and legacy application deployments

Setup deployment pipeline

In software development, a **deployment pipeline** is a system of automated processes designed to quickly and accurately move new code additions and updates from version control to production.

Step 1: Create a deployment environment. ...

Step 2: Get a copy of the sample code. ...

Step 3: Create your pipeline. ...

Step 4: Activate your pipeline to deploy your code. ...

Step 5: Commit a change and then update your app. ...

Step 6: Clean up your resources.

Continuous deployment

Continuous deployment is a strategy for software releases wherein any code commit that passes the automated testing phase is automatically released into the production environment, making changes that are visible to the software's users. Continuous deployment eliminates the human safeguards against unproven code in live software. It should only be implemented when the development and IT teams rigorously adhere to production-ready development practices and thorough testing, and when they apply sophisticated, real-time monitoring in production to discover any issues with new releases.

Static code analysis

Static analysis is a method of debugging that is done by automatically examining the source code without having to execute the program. This provides developers

with an understanding of their code base and helps ensure that it is compliant, safe, and secure.

When Is Static Code Analysis Performed?

Static code analysis is performed early in development, before [software testing](#) begins. For organizations practicing [DevOps](#), static code analysis takes place during the “Create” phase.

Containers

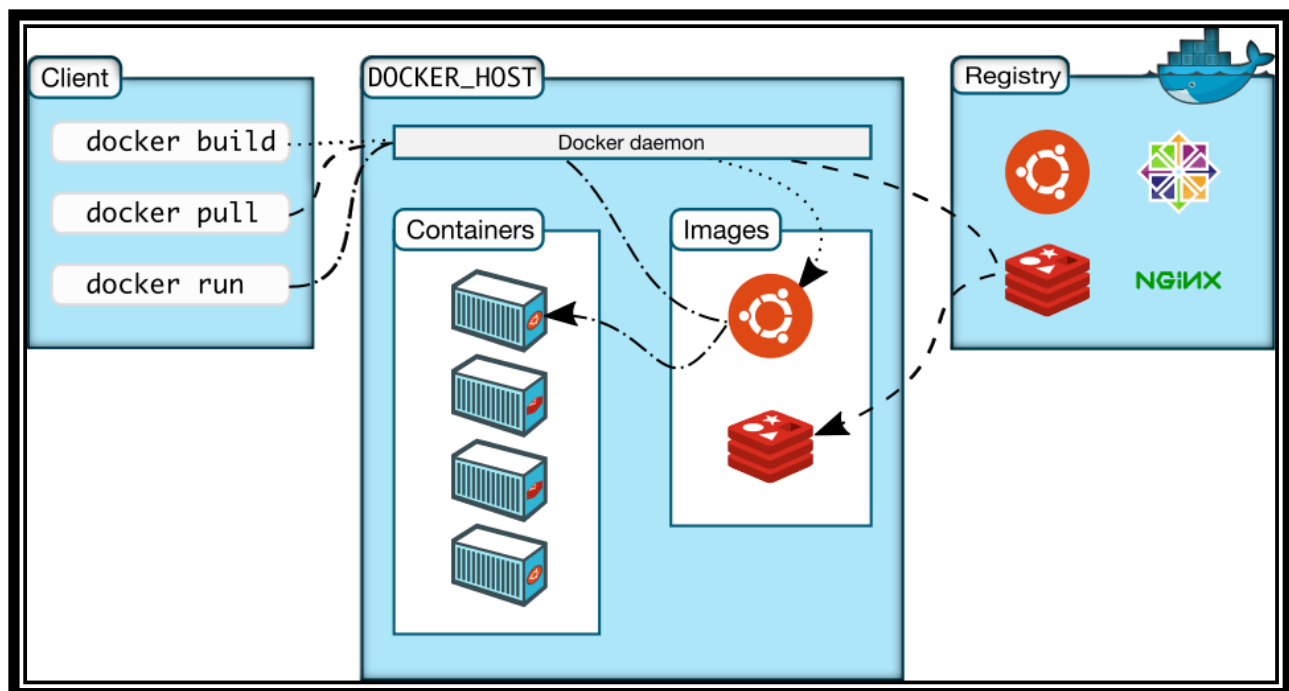
A container is a standard unit of software that packages up code and all its dependencies so the application runs quickly and reliably from one computing environment to another.

Why containers?

Containers require less system resources than traditional or hardware virtual machine environments because they don't include operating system images. Applications running in containers can be deployed easily to multiple different operating systems and hardware platforms.

What is a Docker?

Docker is an open source platform for building, deploying, and managing containerized applications.



How Docker works?

Docker uses a client-server architecture. The Docker client talks to the Docker daemon, which does the heavy lifting of building, running, and distributing your Docker containers. The Docker client and daemon can run on the same system, or you can connect a Docker client to a remote Docker daemon.

Components of Docker

- Docker container

Docker containers are the live, running instances of Docker images. While Docker images are read-only files, containers are live, ephemeral, executable content. Users can interact with them, and administrators can adjust their settings and conditions using Docker commands.

- Docker client

Docker client is an application for Mac or Windows that includes Docker Engine, Docker CLI client, Docker Compose, Kubernetes, and others. It also includes access to Docker Hub.

- Docker daemon

Docker daemon is a service that creates and manages Docker images, using the commands from the client. Essentially Docker daemon serves as the control center of your Docker implementation. The server on which Docker daemon runs is called the Docker host.

- Docker image

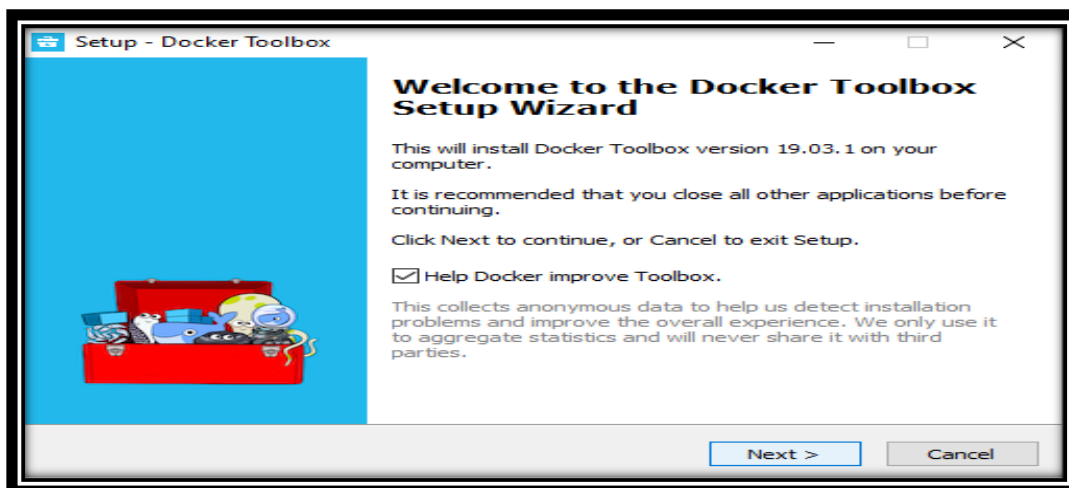
Docker images contain executable application source code as well as all the tools, libraries, and dependencies that the application code needs to run as a container. When you run the Docker image, it becomes one instance (or multiple instances) of the container.

- Docker registry

A *Docker registry* is a scalable open-source storage and distribution system for Docker images. The registry enables you to track image versions in repositories, using tagging for identification. This is accomplished using git, a version control tool.

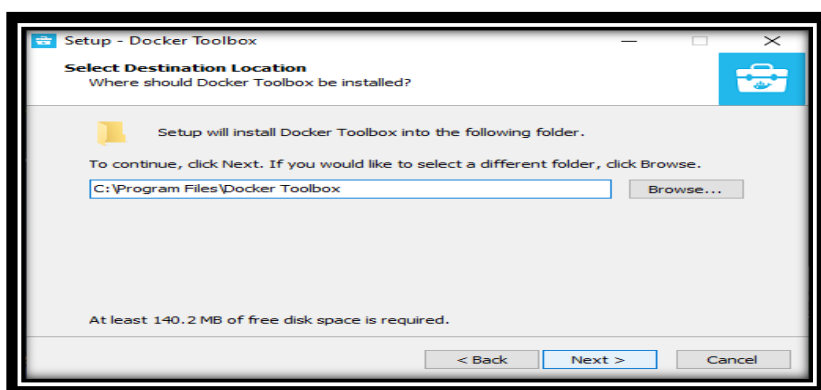
Install docker on desktop and start the docker tool.

- a – Download the Docker Toolbox executable
- b – Run the Docker Toolbox installer for Windows.



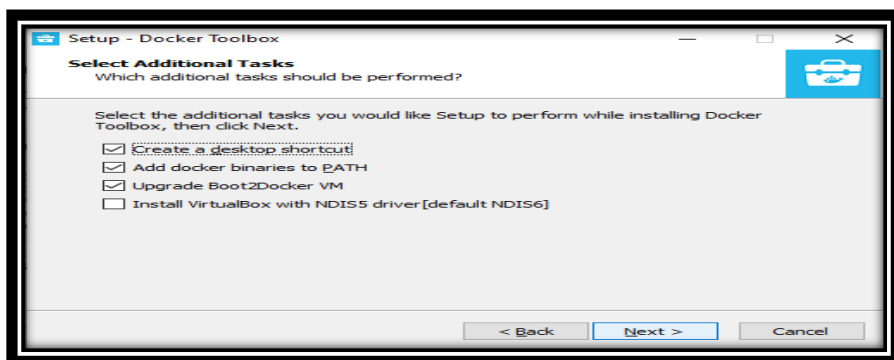
Click on **Next**.

For the second screen, choose a location where to store your Docker files, and click on Next.

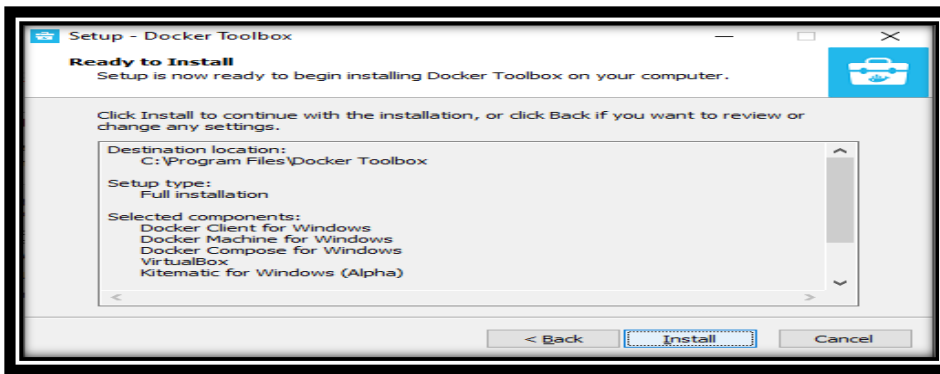


Click on **Next**.

For the additional tasks, you can let the default options and click on Next.



Finally, click on Install to start the Docker Toolbox installation.



After a short while, your installation should complete. Leave the checkbox checked in order to see the shortcuts created by Docker.



To verify your current Docker version, run the following command

```
$ docker version
```

Docker file

Every Docker container starts with a simple text file containing instructions for how to build the Docker container image. *DockerFile* automates the process of Docker image creation. It's essentially a list of command-line interface (CLI) instructions that Docker Engine will run in order to assemble the image. The list of Docker commands is huge, but standardized: Docker operations work the same regardless of contents, infrastructure, or other environment variables.

Docker image

Docker images contain executable application source code as well as all the tools, libraries, and dependencies that the application code needs to run as a container. When you run the Docker image, it becomes one instance (or multiple instances) of the container.

Commands to create docker file.

ADD – Defines files to copy from the Host file system onto the Container

- `ADD ./local/config.file /etc/service/config.file`

- **CMD** – This is the command that will run when the Container starts
 - `CMD ["nginx", "-g", "daemon off;"]`
- **ENTRYPOINT** – Sets the default application used every time a Container is created from the Image. If used in conjunction with CMD, you can remove the application and just define the arguments there
 - `CMD Hello World!`
 - `ENTRYPOINT echo`
- **ENV** – Set/modify the environment variables within Containers created from the Image.
 - `ENV VERSION 1.0`
- **EXPOSE** – Define which Container ports to expose
 - `EXPOSE 80`
- **FROM** – Select the base image to build the new image on top of
 - `FROM ubuntu:latest`
- **LABEL maintainer** – Optional field to let you identify yourself as the maintainer of this image. This is just a label (it used to be a dedicated Docker directive).
 - `LABEL maintainer=someone@xyz.xyz`
- **RUN** – Specify commands to make changes to your Image and subsequently the Containers started from this Image. This includes updating packages, installing software, adding users, creating an initial database, setting up certificates, etc. These are the commands you would run at the command line to install and configure your application. This is one of the most important dockerfile directives.
 - `RUN apt-get update && apt-get upgrade -y && apt-get install -y nginx && rm -rf /var/lib/apt/lists/*`
- **USER** – Define the default User all commands will be run as within any Container created from your Image. It can be either a UID or username
 - `USER docker`
- **VOLUME** – Creates a mount point within the Container linking it back to file systems accessible by the Docker Host. New Volumes get populated with the pre-existing contents of the specified location in the image. It is specially relevant to mention is that defining Volumes in a Dockerfile can lead to issues. Volumes

should be managed with docker-compose or “docker run” commands. Volumes are optional. If your application does not have any state (and most web applications work like this) then you don’t need to use volumes.

- **VOLUME** /var/log

- **WORKDIR** – Define the default working directory for the command defined in the “ENTRYPOINT” or “CMD” instructions

- **WORKDIR** /home

Build docker image with docker file

1. Write a Dockerfile for your application.
2. Build the image with docker build command.
3. Host your Docker image on a registry.
4. Pull and run the image on the target machine.

Refer : <https://codefresh.io/blog/build-docker-image-dockerfiles/>

create docker container from docker image

Step 1: Create a Base Container

The Docker create command will create a new container for us from the command line:

```
~ docker create --name nginx_base -p 80:80 nginx:alpine
```

Step 2: Inspect Images

Step 3: Inspect Containers

```
➔ ~ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED
STATUS        PORTS         NAMES
c365af6303e4   nginx:alpine   "/docker-entrypoint..." 6 minutes ago
Created              nginx
```

Step 4: Start the Container

```
➔ ~ docker start nginx_base
nginx_base
```



Step 5: Modify the Running Container

Let's create a new index.html file and copy it onto the running container. Using an editor on your machine, create an index.html file in the same directory that you have been running Docker commands from.

```
➔ ~ docker cp index.html nginx_base:/usr/share/nginx/html/index.html
```

Step 6: Create an Image From a Container

```
➔ ~ docker commit nginx_base  
sha256:0c17f0798823c7febc5a67d5432b48f525320d671beb2e6f04303f3da2f10432
```

Step 7: Tag the Image

```
➔ ~ docker tag 0c17f0798823 hello_world_nginx
```

Step 8: Create Images With Tags

You can also tag the image as it is created by adding another argument to the end of the command like this:

```
➔ ~ docker commit nginx_base hello_world_nginx
```

Step 9: Delete the Original Container

```
➔ ~ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED
STATUS	PORTS	NAMES	
c365af6303e4	nginx:alpine	"/docker-entrypoint..."	33 minutes ago
25 minutes	0.0.0.		Up

Step 10: Look at Running Containers

```
➔ ~ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
7ca08e03862d	hello_world_nginx	"/docker-entrypoint..."	2 minutes ago	Up	2 minutes	0.0.0.0:80->

Step 11: Consider Your Options

Option A: Set Authorship

Option B: Create Commit Messages

Option C: Commit Without Pause

Option D: Change Configuration

Refer : <https://www.dataset.com/blog/create-docker-image/>

Run the docker container

The basic syntax for the command is:

```
docker run [OPTIONS] IMAGE [COMMAND] [ARG...]
```

To run a container, the only thing you need to include in the command is the image on which it is based:

```
docker run [docker_image]
```

As an example, we used a Dockerfile to create a sample Docker image with the task to echo the message **Hello World**. For us, the image has the ID **e98b6ec72f51**. Your image name will differ depending on the container you want to run.

The command to run our sample container would be:

```
docker run e98b6ec72f51
```

Refer : <https://phoenixnap.com/kb/docker-run-command-with-examples>

Unit-12

Container

[Containers](#) are lightweight, executable application components that combine application source code with all the operating system (OS) libraries and dependencies required to run the code in any environment.

Orchestration

Orchestration is the automated configuration, management, and coordination of computer systems, applications, and services. Orchestration helps IT to more easily manage complex tasks and workflows.

Container orchestration

- Container orchestration automates the provisioning, deployment, networking, scaling, availability, and lifecycle management of containers.
- [Kubernetes](#) is the most popular container orchestration platform, and most leading public cloud providers - including Amazon Web Services (AWS), Google Cloud Platform, IBM Cloud and Microsoft Azure - offer managed Kubernetes services.
- Other container orchestration tools include Docker Swarm and Apache Mesos.

What is container orchestration engine?

Container orchestration is the automation of much of the operational effort required to run containerized workloads and services. This includes a wide range of things software teams need to manage a container's lifecycle, including provisioning, deployment, scaling (up and down), networking, load balancing and more.

Orchestration tools

1. Kubernetes
2. Openshift
3. Nomad
4. Docker Swarm
5. Docker Compose
6. MiniKube
7. Marathon
8. Cloudify

Kubernetes

Kubernetes — also known as “k8s” or “kube” — is a container orchestration platform for scheduling and automating the deployment, management, and scaling of containerized applications.

Why Kubernetes?

- **Service discovery and load balancing:** If traffic to a container is high, Kubernetes is able to load balance and distribute the network traffic so that the deployment is stable.
- **Storage orchestration:** allows you to automatically mount a storage system of your choice, such as local storages, public cloud providers, and more
- **Automated rollouts and rollbacks:** it can change the actual state to the desired state at a controlled rate. For example, you can automate Kubernetes to create new containers for your deployment, remove existing containers and adopt all their resources to the new container.
- **Automatic bin packing:** You provide Kubernetes with a cluster of nodes that it can use to run containerized tasks. You tell Kubernetes how much CPU and memory (RAM) each container needs. Kubernetes can fit containers onto your nodes to make the best use of your resources.
- **Self-healing:** Kubernetes restarts containers that fail, replaces containers, kills containers that don't respond to your user-defined health check, and doesn't advertise them to clients until they are ready to serve.
- **Secret and configuration management:** Kubernetes lets you store and manage sensitive information, such as passwords, OAuth tokens, and SSH keys. You can deploy and update secrets and application configuration without rebuilding your container images, and without exposing secrets in your stack configuration.

Kubernetes configuration

- Deployment

```
curl.exe -LO "https://dl.k8s.io/release/v1.26.0/bin/windows/amd64/kubectl.exe"
```

- Service

- Load balancer/ingress

Deployment strategies:

A Deployment strategy defines how to create, upgrade, or downgrade different versions of Kubernetes applications.

Blue green deployment:

Canary Deployment

Disaster recovery

Disaster recovery is the process by which an organization anticipates and addresses technology-related disasters.

Kubernetes disaster recovery can be broken down into two phases: backup and recovery. Backup is the process of preserving data before any disaster strikes, while recovery entails getting back up after one has occurred.

Disaster recovery and their types:

1. Data Center Disaster Recovery
2. Network Disaster Recovery
3. Virtualized Disaster Recovery
4. Disaster Recovery in the Cloud
5. Disaster Recovery as a Service (DRaaS)

Converting monolithic application to microservices architecture

Step 1: Split the code.

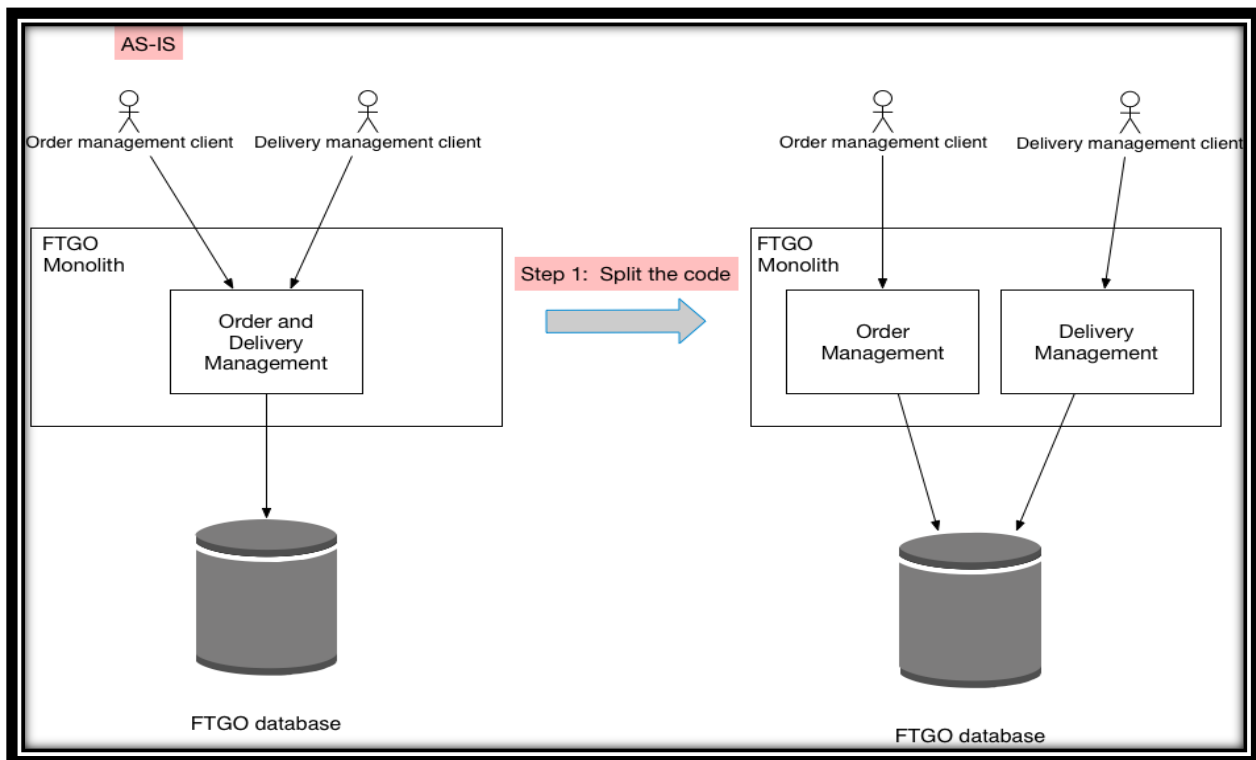
Step 2: Split the database.

Step 3: Define a standalone Delivery Service.

Step 4: Use the standalone Delivery Service.

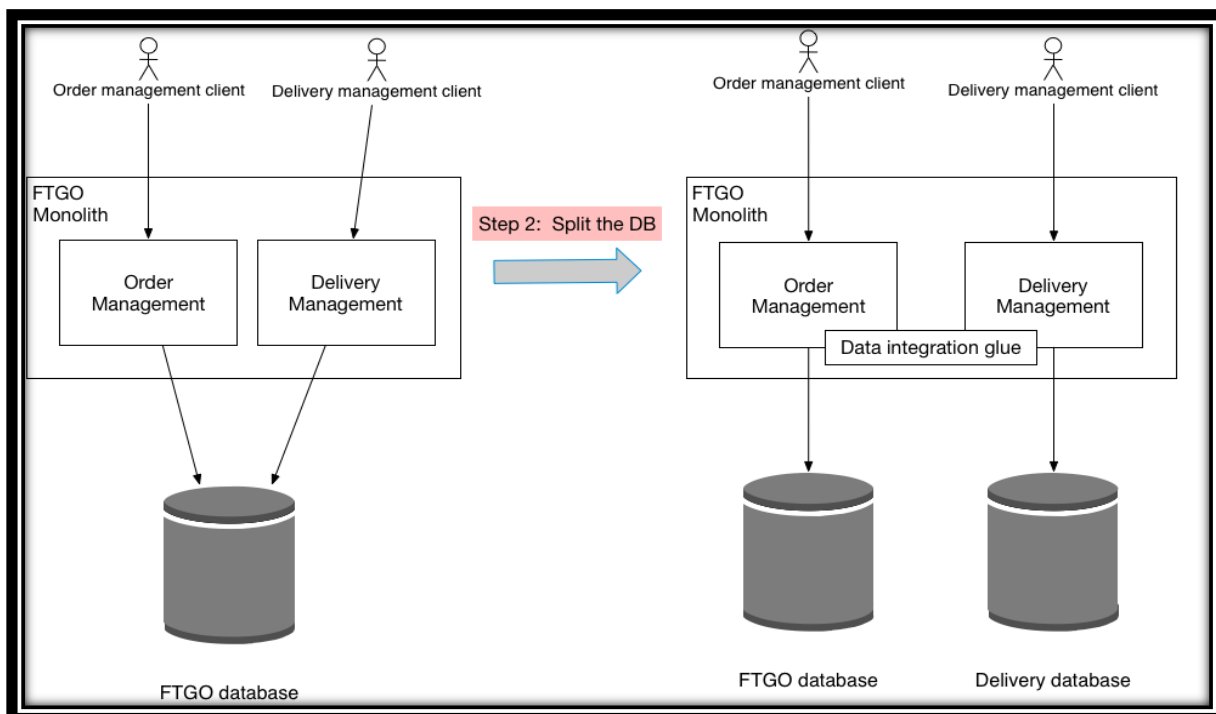
Step 5: Remove the delivery management functionality from the FTGO monolith.

Step 1: Split the code



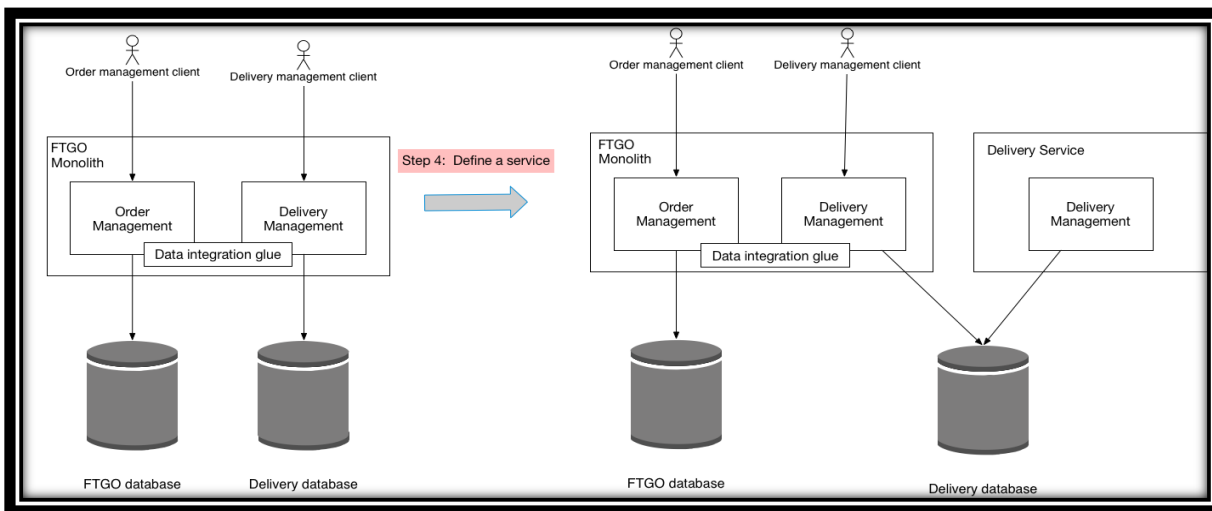
The first step is to split the code and convert delivery management into a separate, loosely coupled module within the monolith.

Step 2: Split the database



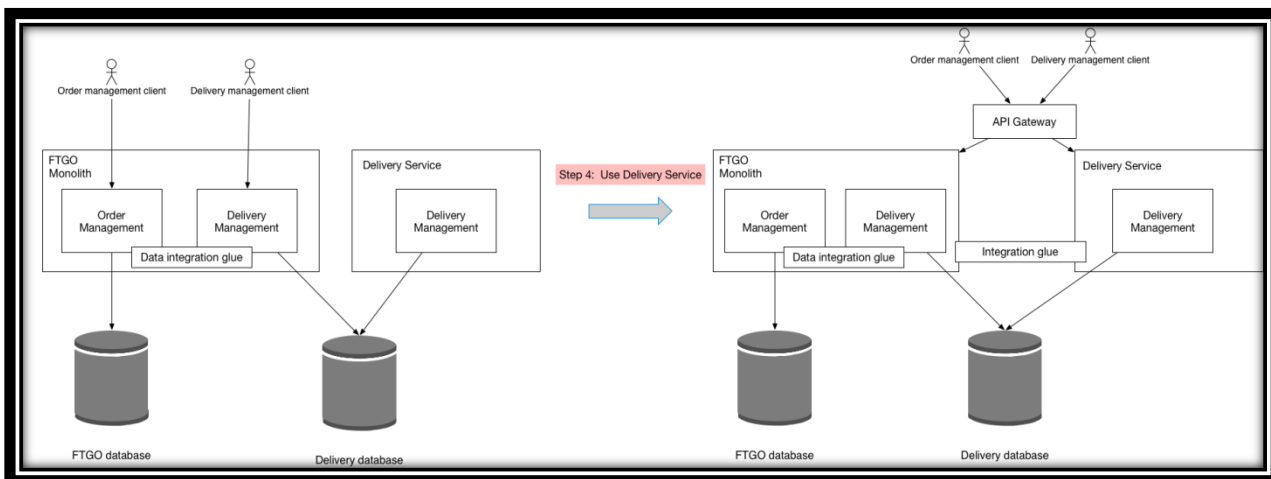
The second step is split the database and define a separate database schema for the `ftgo-delivery-service` module.

Step 3: Define a standalone Delivery Service



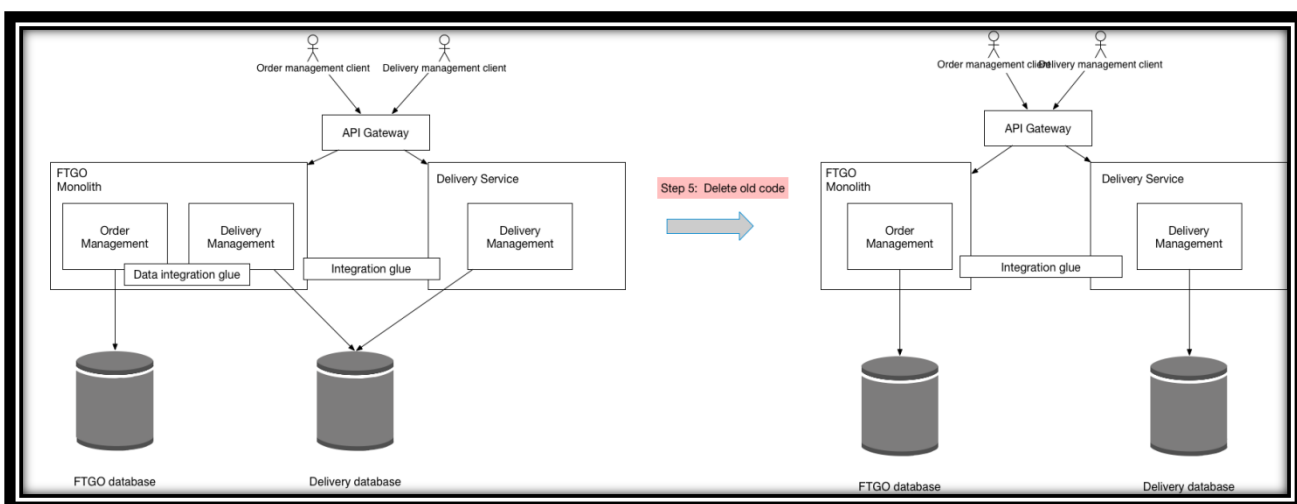
The third step of the refactoring process is to define a standalone **Delivery Service** and deploy it. The service does not, however, handle production traffic. Instead, it can, for example, be tested in production.

Step 4: Use the standalone Delivery Service



The fourth step of the refactoring process is to use the standalone **Delivery Service**.

Step 5: Remove the delivery management functionality from the FTGO monolith



The fifth step of the refactoring process is to remove the now obsolete delivery management logic from the monolith.